# Generative Artificial Intelligence

## Txt2Txt Translation

Dr. Pravin

School of Computer Science, UPES Dehradun

# Txt2Txt Translation: From Autoencoder to Seq2Seq

## Recall the Autoencoder:

Compresses an input, then reconstructs the **same** input:

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} \hat{x} \approx x$$

## New idea:

What if the input and output are both **text**, but the output is a *different* sequence — for example, a translation, a summary, or a paraphrase?

$$x \xrightarrow{\text{Encoder}} z \xrightarrow{\text{Decoder}} y, \qquad y \neq x$$

This reframes the Autoencoder as a more general **sequence-to-sequence** tool.

# Text Representation Basics

**Before text can enter a neural network, it must become numbers.**

**Tokenization:**

Splitting text into smaller units (tokens) — typically words or sub-words.

$$\text{“I love cats”} \rightarrow [\text{“I”}, \text{“love”}, \text{“cats”}]$$

**Vocabulary & Embedding:**

Each token is mapped to a vector, using a lookup table (vocabulary) built from the training data. This vector is what the encoder actually receives.

# Encoder in Detail: What It Does

## The encoder's job:

Read the entire input sequence and compress it into a single fixed-size representation — the **context vector** $z$.

$$x = [x_1, x_2, \ldots, x_n] \xrightarrow{\text{Encoder}} z$$

## Key property: variable length in, fixed length out

The input sequence $x$ can be any length $n$ (a short sentence or a long one), but $z$ always has the **same fixed size** — exactly like the Autoencoder's bottleneck.

# Encoder in Detail: How It Processes the Sequence

## Token by token:

The encoder reads the input one token at a time, updating an internal **running summary** as it goes.

$$x_1 \rightarrow [\text{summary}_1] \rightarrow x_2 \rightarrow [\text{summary}_2] \rightarrow \cdots \rightarrow x_n \rightarrow [\text{summary}_n = z]$$

## The final summary, after seeing every token, becomes $z$:

$z$ is meant to capture everything the decoder will need to know about the input — its meaning, structure, and content — packed into one fixed-size vector.

# Encoder in Detail: Why This Is Hard

**The bottleneck problem:**

Every piece of information about the input sequence — however long — must be squeezed into **one fixed-size vector** $z$.

For a short sentence, this is manageable. For a long paragraph, $z$ has to compress a lot into the same fixed space — information can get lost.

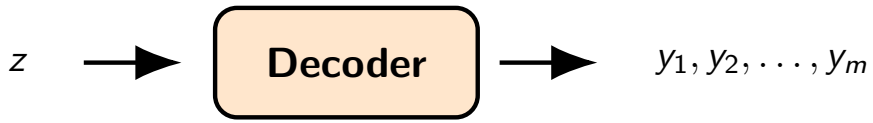
*(This limitation is exactly what later motivates attention mechanisms — letting the decoder look back at all of the encoder's intermediate summaries, not just the final one.)*

# Decoder in Detail: What It Does

## The decoder's job:

Take the context vector $z$ and generate the output sequence, one token at a time.

$$z \xrightarrow{\text{Decoder}} y = [y_1, y_2, \ldots, y_m]$$

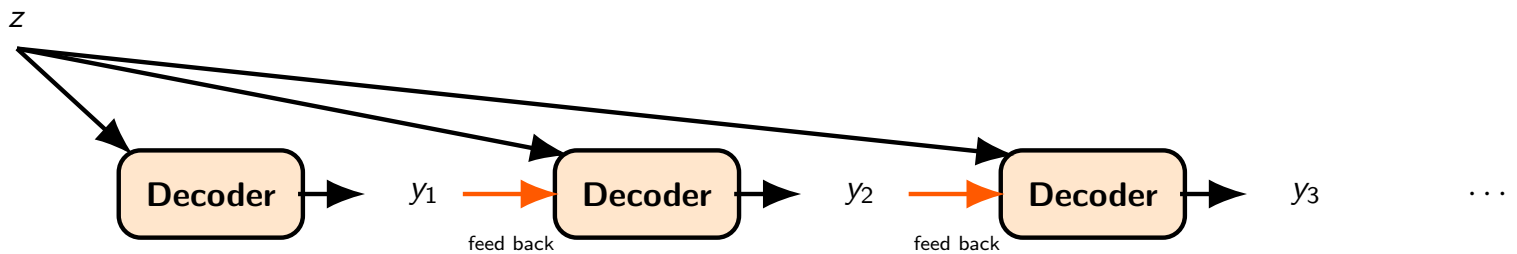



Key difference from the encoder: the decoder **generates** a sequence rather than reading one.

# Decoder in Detail: One Token at a Time

## Autoregressive generation:

Each new token is generated using $z$ **and** the tokens already produced so far.



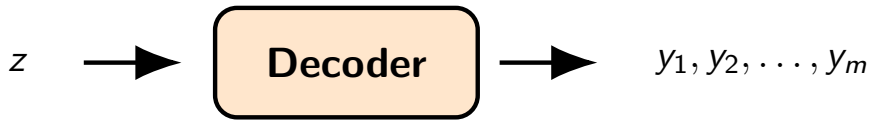

Generation stops when the decoder produces a special "end of sequence" token.

# Decoder in Detail: What It Does

## The decoder's job:

Take the context vector $z$ and generate the output sequence, one token at a time.

$$z \xrightarrow{\text{Decoder}} y = [y_1, y_2, \ldots, y_m]$$

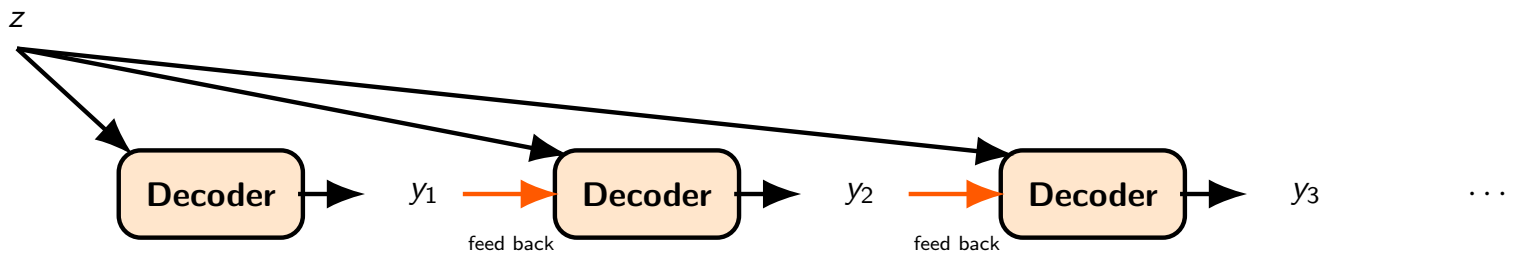



Key difference from the encoder: the decoder **generates** a sequence rather than reading one.

# Decoder in Detail: One Token at a Time

## Autoregressive generation:

Each new token is generated using $z$ **and** the tokens already produced so far.




Generation stops when the decoder produces a special "end of sequence" token.

# Training Objective

## Recall the Autoencoder's loss:

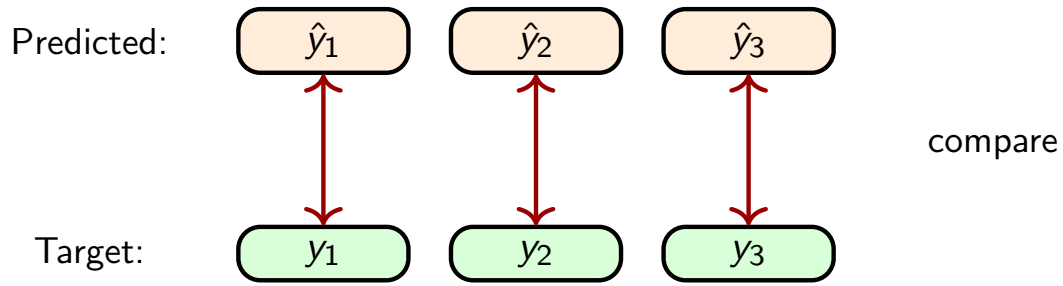
Compares the reconstruction $\hat{x}$ to the **same** input $x$.

## Here, the comparison changes:

The decoder's predicted sequence $\hat{y}$ is compared to the **true target** sequence $y$ — e.g. the correct translation — not the original input $x$.

At each step, the decoder predicts a probability over the vocabulary for the next token; we compare this to the actual correct token.

# Training Objective: Comparing Predicted vs. Target




$$\mathcal{L} = -\sum_{t=1}^{m} \log P(y_t \mid y_{<t}, z)$$

A cross-entropy loss summed over every position in the output sequence.

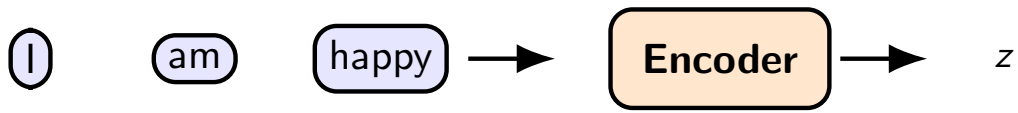
# Worked Example: Setup

## Toy task: paraphrase a short sentence

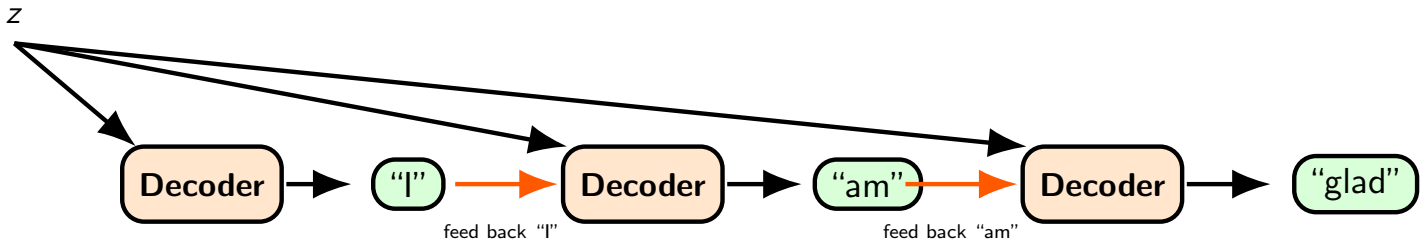
Input: "I am happy" $\longrightarrow$ Target: "I am glad"

## Step 1 — Tokenize the input:

"I am happy" $\rightarrow$ [I, am, happy]



# Worked Example: Decoder Generates Step by Step




Each step uses $z$ plus everything generated so far to predict the next word.

# Worked Example: Comparing to the Target

| Position | Predicted $\hat{y}$ | Target $y$ |
|---|---|---|
| 1 | "I" | "I" |
| 2 | "am" | "am" |
| 3 | "glad" | "glad" |

In this toy case, every predicted token matches the target exactly — so the cross-entropy loss $\mathcal{L}$ is small.

If the decoder had predicted, say, "I am sad" instead, positions 1–2 would match but position 3 would not — raising the loss and pushing training to correct that prediction.

# Recurrent Neural Networks (RNN): Why Now?

## Connecting back to Txt2Txt:

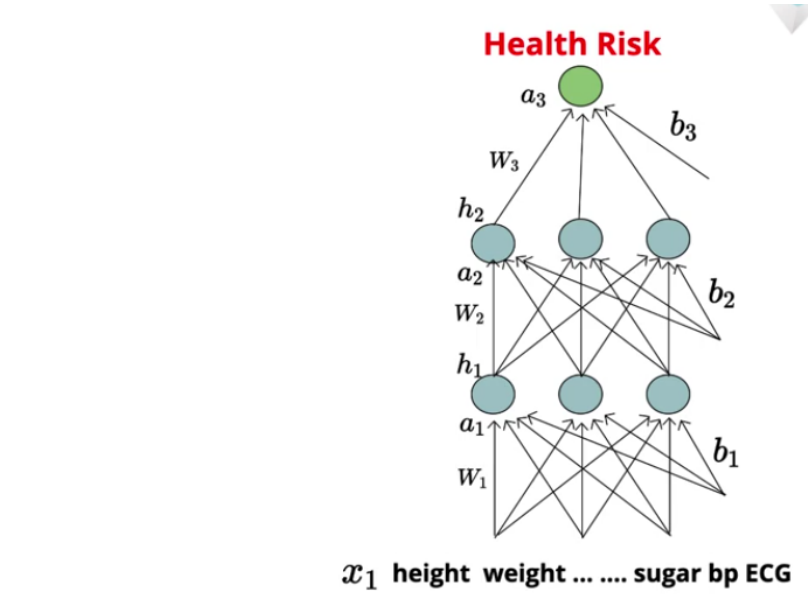
We left the Encoder and Decoder as abstract boxes — "compresses the sequence," "generates the sequence." An **RNN** is the classic architecture that actually sits inside those boxes, processing tokens one at a time while carrying forward a hidden state.

## Connecting forward to Transformers:

RNNs process tokens **sequentially** — one at a time, in order. This makes them slow to train and prone to forgetting information from early in a long sequence.

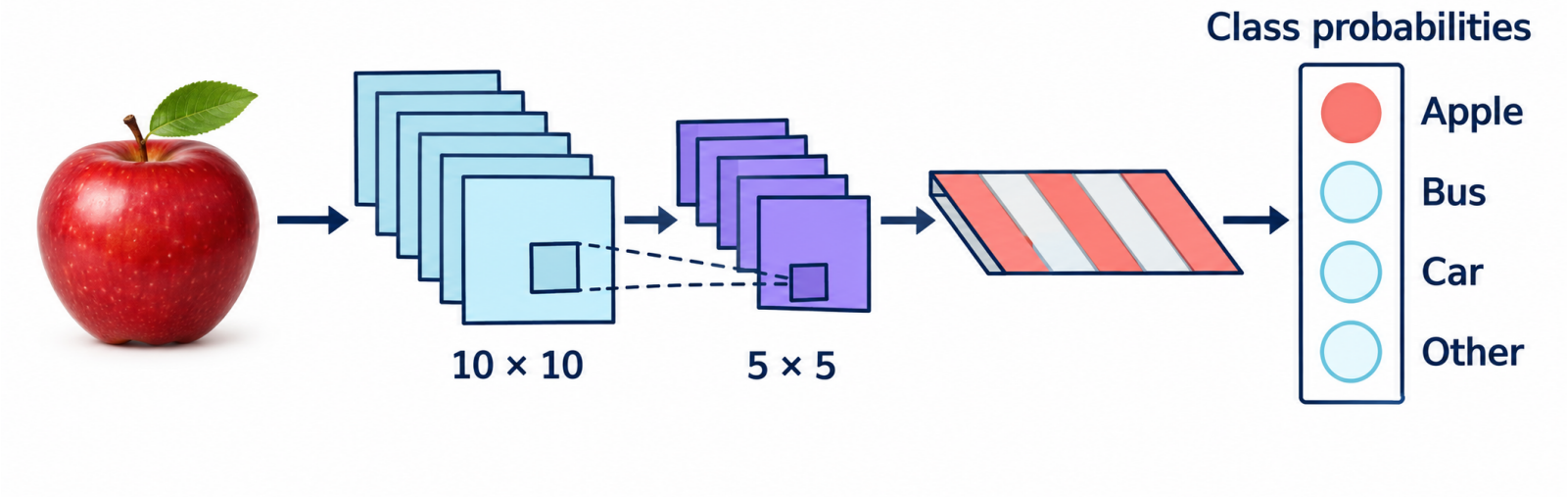
**Transformers** were designed specifically to fix this, by letting the model look at all tokens **at once** instead of one by one. Understanding RNNs first is what makes that motivation clear.

# Recap: Fully Connected Network




- Input features (Height, Weight, Sugar, BP, ECG) feed into the first hidden layer $a_1/h_1$ via weights $W_1$ and bias $b_1$
- Each hidden layer is **fully connected** to the next: $a_1/h_1 \rightarrow a_2/h_2$ via $W_2, b_2$
- Final layer $a_3$ produces the output (Health Risk) via $W_3, b_3$

# Recap: Convolutional Neural Network (CNN)




- Convolution extracts local features from the image, producing feature maps ($10 \times 10$)
- Further convolution/pooling reduces spatial size while retaining important features ($5 \times 5$)
- Flattened features pass through fully connected layers
- Output: class probabilities (Apple, Bus, Car, Other)

# A Shared Limitation: Fixed-Size Input

## Fully Connected Network:

$$x \in \mathbb{R}^n$$

Input must be a vector of a **fixed length** $n$.

## Convolutional Neural Network (CNN):

$$x \in \mathbb{R}^{30 \times 30}$$

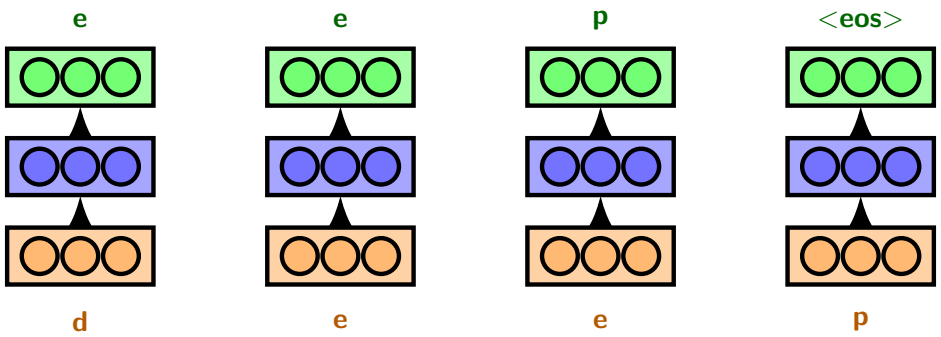
Input must be an image of a **fixed size**.

✓ Outputs are independent of previous inputs
✓ Input is of a fixed length

Neither architecture is built to handle sequences of **varying** length, or outputs that depend on what came before — this is exactly the gap RNNs are designed to fill.

# Sequence Learning Problems

*Let's look at a different class of problems*




$$h = \sigma(Wx + b), \qquad y = \text{softmax}(Vh + c)$$

# Sequence Learning Problems: Key Properties

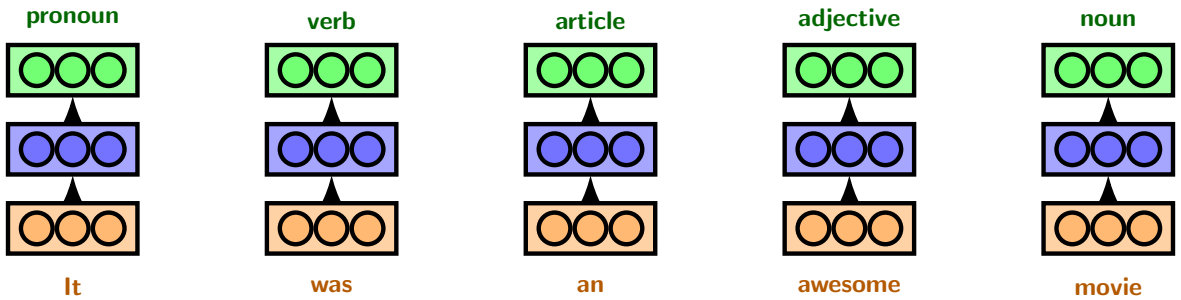
✓ Outputs depend on previous inputs also

✓ The length of the input is not fixed

This is the exact opposite of the FCN/CNN limitation we just saw — and it's precisely what RNNs are built to handle.

# Sequence Learning Problems

*What about a sequence of words?*




Task: label each word in the sentence "It was an awesome movie" with its part of speech.

# How Words Are Represented: One-Hot Encoding

## Step 1 — Build a vocabulary (index every word):

$$\text{a} = 1, \quad \text{an} = 2, \quad \text{the} = 3, \quad \text{movie} = 4, \quad \ldots$$

## Step 2 — Represent each word as a one-hot vector:

The vector has one entry per vocabulary word: a 1 at that word's index, 0 everywhere else.

$$\text{“an”} \rightarrow x = [0,\ 1,\ 0,\ 0,\ 0,\ \ldots]$$

$$\text{“movie”} \rightarrow x = [0,\ 0,\ 0,\ 1,\ 0,\ \ldots]$$

This one-hot vector $x$ is what actually enters the network at each time step.

# Feedforward: From Word to Prediction

**The same two equations as before, now with one-hot input:**

$$h = \sigma(Wx + b), \qquad y = \text{softmax}(Vh + c)$$

**Example — predicting the tag for "It" (true tag: pronoun):**

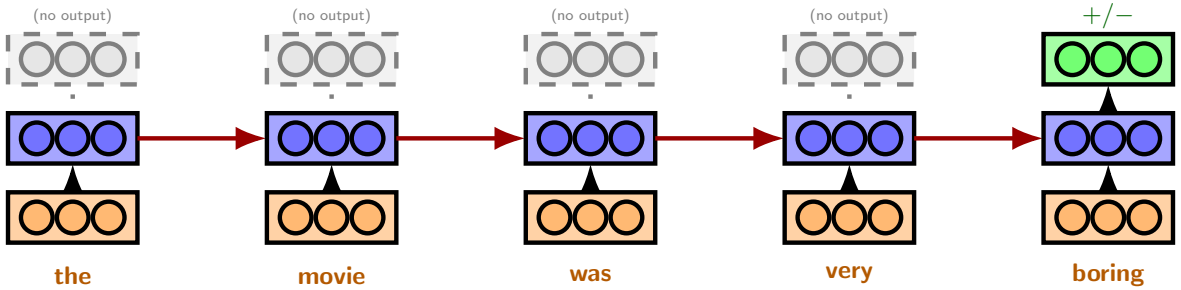
$$y = [1,\ 0,\ 0,\ 0,\ 0,\ \ldots] \quad \text{(true one-hot label)}$$

$$\hat{y} = [0.7,\ 0.1,\ 0.1,\ 0.05,\ 0.05,\ \ldots] \quad \text{(predicted probabilities)}$$

$\hat{y}$ is the network's predicted probability over all possible tags; $y$ is the true one-hot target. The gap between them is what the network learns to close — exactly as with the character-prediction example.

# Sequence Learning Problems

*Do we always need to produce an output at every time step?*




The hidden state carries forward at every step, but the output is only produced **once**, at the end — after the whole sentence has been read.

# Many-to-One: Not Every Step Needs an Output

## Example: sentiment classification

"the movie was very boring" $\longrightarrow$ negative $(-)$

## Why wait until the end?

The sentiment of a sentence usually can't be judged from a single word in isolation — it depends on the **whole sequence**. The network reads every word, updating its hidden state each time, and only makes its final decision once it has seen the entire input.

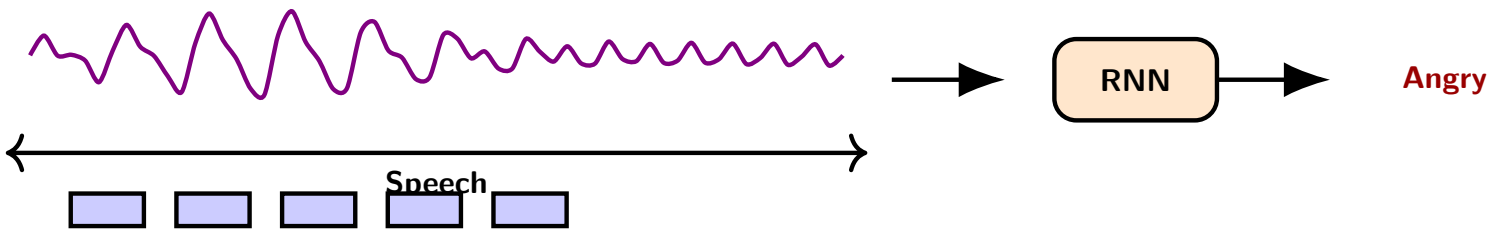
This is called a **many-to-one** architecture — many inputs (words), one output (the overall sentiment). Contrast this with the earlier examples, which were **many-to-many** (one output per input step).

# Sequence Learning Problems

*What are some other types of sequences that we encounter?*

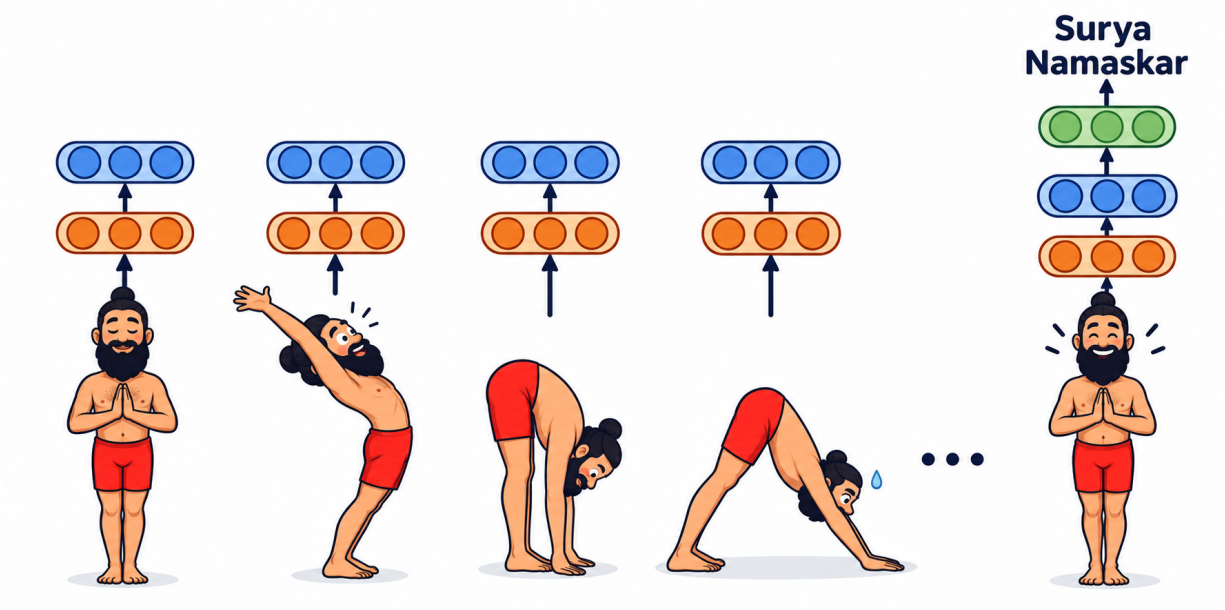
## Speech / Audio

A speech signal is also a sequence — just not one made of words or characters, but of **audio samples over time**.




The waveform is broken into small time segments (frames), read one at a time — just like words in a sentence — to classify the overall emotion.

# Sequence Learning Problems: Video Sequences




A sequence of video frames (each pose in the routine) is read one at a time; the final frame's output classifies the overall action: **Surya Namaskar**.

# Why Is the Input Length Variable Here?

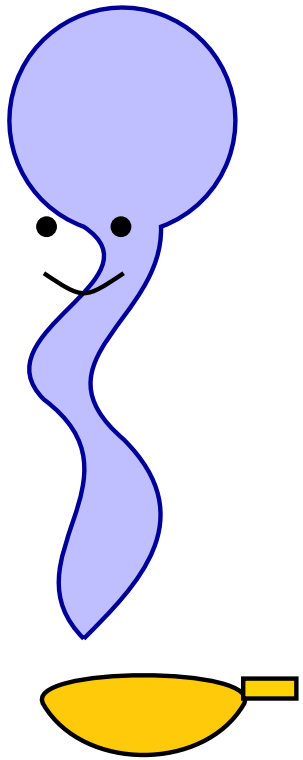
## The same routine, performed at different speeds:

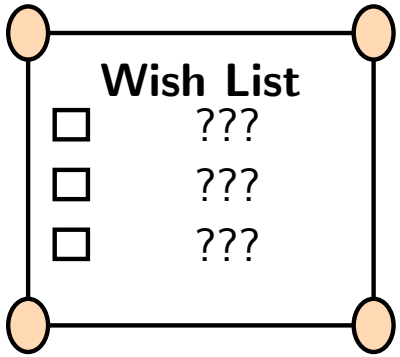
Two people doing Surya Namaskar don't take the same amount of time — one may move slowly through each pose, another quickly.

## More time $\rightarrow$ more frames captured:

A slower performance produces **more** video frames for the same routine; a faster one produces **fewer**. The number of input frames is not fixed — it depends on how fast the person moves.

This is exactly the "length of input is not fixed" property from earlier — and exactly why an RNN, not a fixed-size FCN/CNN, is the right tool here.






# A Wish List for Modelling Sequence Learning Problems

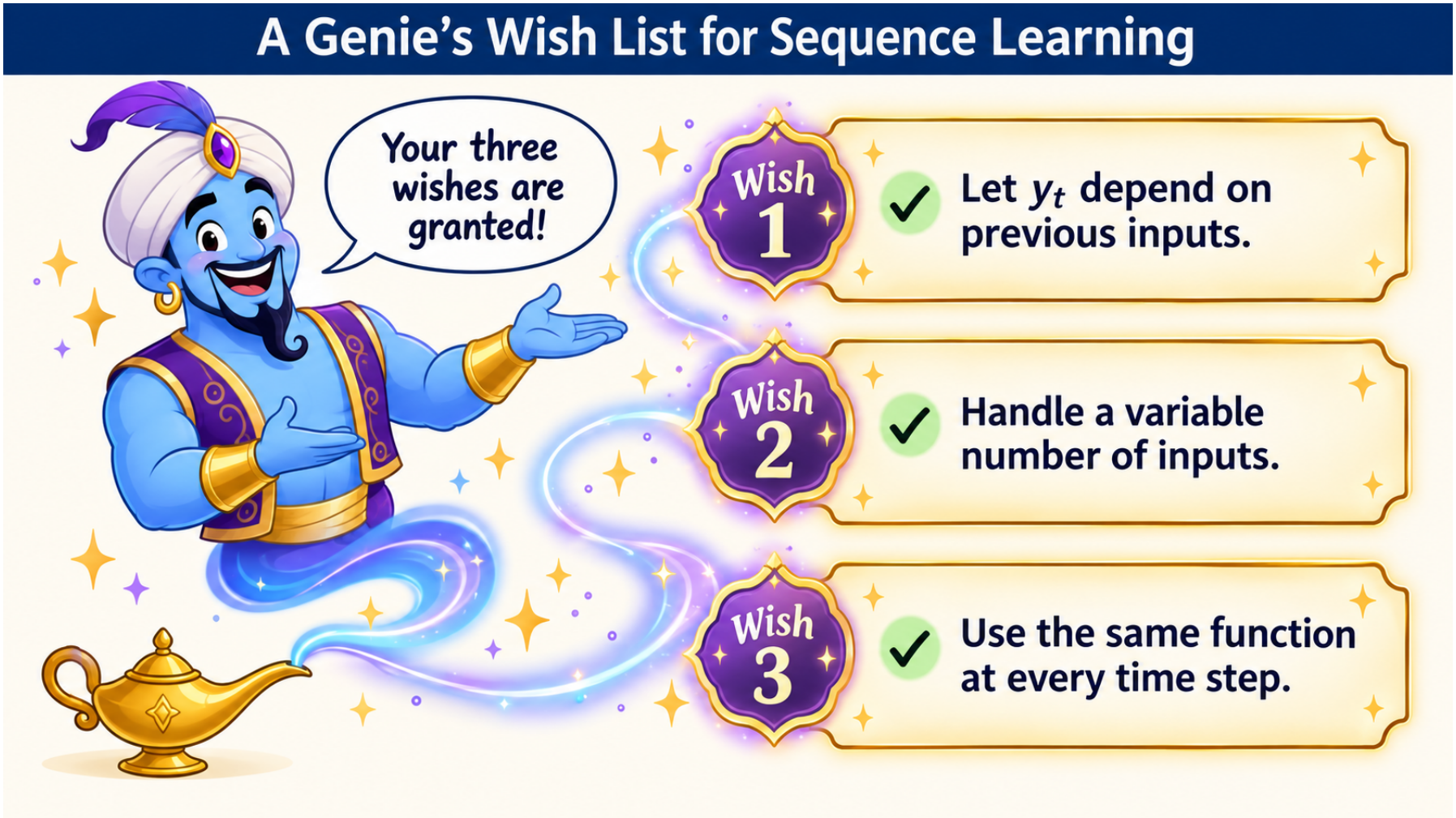
# A Wish List for Modelling Sequence Learning Problems

✓ Ensure that $y_t$ is dependent on previous inputs also

✓ Ensure that the function can deal with variable number of inputs

✓ Ensure that the function executed at each time step is the same

# A Genie's Wish List for Sequence Learning



# Sequence Models

*What is the function being executed at each time step?*

✓ Ensure that $y_t$ is dependent on previous inputs also

✓ Ensure that the function can deal with variable number of inputs

✓ **Ensure that the function executed at each time step is the same**

$$h_i = \sigma(W_1 x_i + b_1), \qquad y_i = O(W_2 h_i + b_2), \qquad i = \text{timestep}$$

# Sequence Models: The Same Function, Every Time Step

**At each timestep $i$, the same equations are applied:**

$$y_1 = O(W_2 h_1 + b_2)$$

$$y_2 = O(W_2 h_2 + b_2)$$

Notice: $W_2$ and $b_2$ are **identical** in both equations — only $h_i$ changes from step to step.

$$y_i = f(x_i,\ W_1, W_2, b_1, b_2)$$

The same function $f$, with the **same parameters**, is reused at every single timestep.

# Sequence Models: Parameter Sharing

## Equivalently, written with common RNN notation:

$$s_i = \sigma(Ux_i + b), \qquad y_i = O(Vs_i + c)$$

✓ **Parameter Sharing**

The same weights $U, V$ and biases $b, c$ are used at every timestep, regardless of the input sequence's length — this is what makes the RNN able to handle sequences of **any** length with a **fixed** number of parameters.

*How does parameter sharing help here?*

✓ Ensure that $y_t$ is dependent on previous inputs also

✓ **Ensure that the function can deal with variable number of inputs**

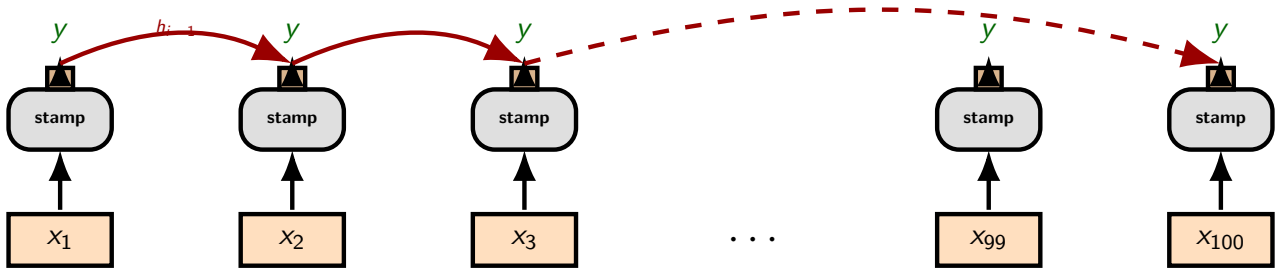
✓ Ensure that the function executed at each time step is the same

$$h_i = \sigma(Ux_i + Wh_{i-1} + b), \qquad y_i = O(Vh_i + c)$$

$$y_i = f(x_i,\ h_{i-1},\ U, V, W, b, c)$$

The extra term $Wh_{i-1}$ carries the *previous* hidden state forward. Since $U, V, W, b, c$ never change, the **same** $f$ works for $x_1, x_2, \ldots$ no matter how many there are — even $x_{99}$, $x_{100}$, and beyond.

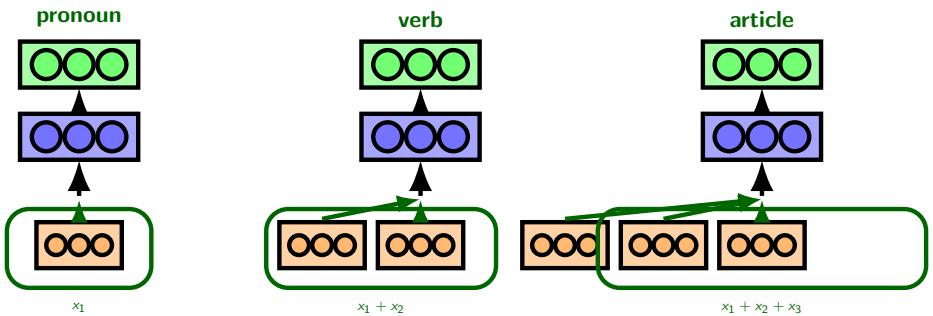
# Why This Matters: One Stamp, Any Length




The **same stamp** (weights $U, V, W, b, c$) is used at every step, but each stamp also receives the previous hidden state $h_{i-1}$ (red arrows) — so it doesn't matter if the sequence has 3 tokens or 300: no new parameters are needed, and information still flows all the way back to $x_1$.

# Sequence Models

*What have we not accounted for so far?*




At each step, the naive approach feeds in *more and more* concatenated inputs — the input to the hidden layer keeps growing.

# Sequence Models

*What have we not accounted for so far?*

## Recall this equation had no memory of previous inputs:

$$s_i = \sigma(Ux_i + b), \qquad y_i = O(Vs_i + c)$$

## A naive fix: concatenate all previous inputs together

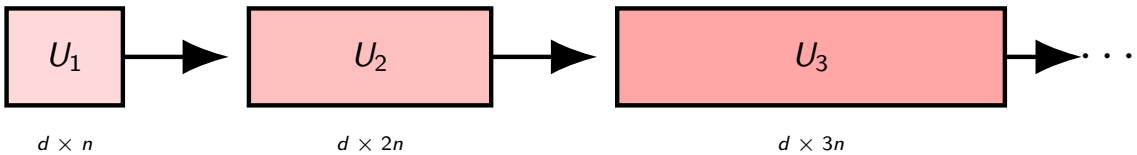
$$\hat{y}_1 = \hat{f}(x_1), \quad \hat{y}_2 = \hat{f}(x_1, x_2), \quad \hat{y}_3 = \hat{f}(x_1, x_2, x_3), \quad \ldots$$
$$\hat{y}_n = \hat{f}(x_1, x_2, x_3, \ldots, x_n)$$

At each step, simply feed in *all* the words seen so far, concatenated into one long vector.

# Why This Approach Fails

## Look at the input size to $\hat{f}$ at each step:

- Step 1: input is $x_1 \in \mathbb{R}^n$
- Step 2: input is $[x_1; x_2] \in \mathbb{R}^{2n}$
- Step 3: input is $[x_1; x_2; x_3] \in \mathbb{R}^{3n}$
- Step $t$: input is $[x_1; \ldots; x_t] \in \mathbb{R}^{tn}$




A **different, ever-growing** weight matrix is needed at every step — this breaks **parameter sharing**, and the model can never handle a sequence longer than whatever it was built for.

# This Is Why We Need RNNs

**The naive concatenation approach violates our own wishlist:**

- ✗ The function executed at each time step is **not** the same — $U_1 \neq U_2 \neq U_3 \neq \ldots$
- ✗ It cannot truly handle a **variable** number of inputs — it only works up to whatever maximum length it was built for

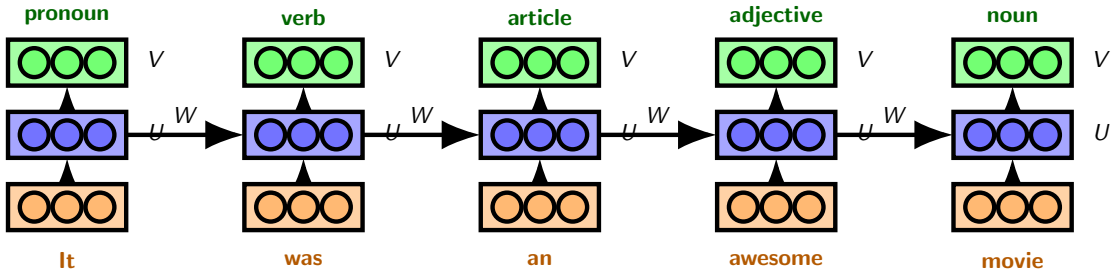
The recurrent hidden state

$$h_i = \sigma(Ux_i + Wh_{i-1} + b)$$

solves both problems at once — a **fixed-size** $U, W, b$ that works for **any** sequence length, by summarizing history through $h_{i-1}$ instead of concatenating it.

# Recurrent Neural Networks (RNNs)

*So what is the solution?*




$$s_i = \sigma(Ux_i + Ws_{i-1} + b), \qquad y_i = O(Vs_i + c)$$

$$y_i = \hat{f}(x_i,\ s_{i-1},\ W, U, V, b, c)$$

# RNN Toy Example: Setup

## Sentence: "It was an awesome movie"

## Word embeddings ($x_i \in \mathbb{R}^2$):

$$x_{\text{It}} = [1, 0], \quad x_{\text{was}} = [0, 1], \quad x_{\text{an}} = [1, 1]$$

$$x_{\text{awesome}} = [1, -1], \quad x_{\text{movie}} = [-1, 1]$$

## Weights and initial hidden state:

$$U = W = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}, \qquad b = [0, 0], \qquad s_0 = [0, 0]$$

**Question: Compute $s_1, s_2, s_3, s_4, s_5$ by hand, using $s_i = \sigma(Ux_i + Ws_{i-1} + b)$.**

# RNN Toy Example: Step 1 — "It"

**$x_{\text{It}} = [1, 0], \quad s_0 = [0, 0]$**

$$Ux_1 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = [0.5,\ 0]$$

$$Ws_0 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = [0,\ 0]$$

$$z_1 = Ux_1 + Ws_0 + b = [0.5,\ 0]$$

$$s_1 = \sigma(z_1) = [\sigma(0.5),\ \sigma(0)] = \boxed{[0.6225,\ 0.5000]}$$

# RNN Toy Example: Step 2 — "was"

$x_{\text{was}} = [0, 1], \quad s_1 = [0.6225,\ 0.5000]$

$$Ux_2 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = [0,\ 0.5]$$

$$Ws_1 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.6225 \\ 0.5000 \end{bmatrix} = [0.3112,\ 0.2500]$$

$$z_2 = Ux_2 + Ws_1 + b = [0.3112,\ 0.7500]$$

$$s_2 = \sigma(z_2) = \boxed{[0.5772,\ 0.6792]}$$

# RNN Toy Example: Step 3 — "an"

$x_{\text{an}} = [1, 1], \quad s_2 = [0.5772,\ 0.6792]$

$$Ux_3 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = [0.5,\ 0.5]$$

$$Ws_2 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.5772 \\ 0.6792 \end{bmatrix} = [0.2886,\ 0.3396]$$

$$z_3 = Ux_3 + Ws_2 + b = [0.7886,\ 0.8396]$$

$$s_3 = \sigma(z_3) = \boxed{[0.6875,\ 0.6984]}$$

# RNN Toy Example: Step 4 — "awesome"

$x_{\text{awesome}} = [1, -1], \quad s_3 = [0.6875,\ 0.6984]$

$$Ux_4 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = [0.5,\ -0.5]$$

$$Ws_3 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.6875 \\ 0.6984 \end{bmatrix} = [0.3438,\ 0.3492]$$

$$z_4 = Ux_4 + Ws_3 + b = [0.8438,\ -0.1508]$$

$$s_4 = \sigma(z_4) = \boxed{[0.6993,\ 0.4624]}$$

# RNN Toy Example: Step 5 — "movie"

$x_{\text{movie}} = [-1, 1], \quad s_4 = [0.6993,\ 0.4624]$

$$Ux_5 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = [-0.5,\ 0.5]$$

$$Ws_4 = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 0.6993 \\ 0.4624 \end{bmatrix} = [0.3496,\ 0.2312]$$

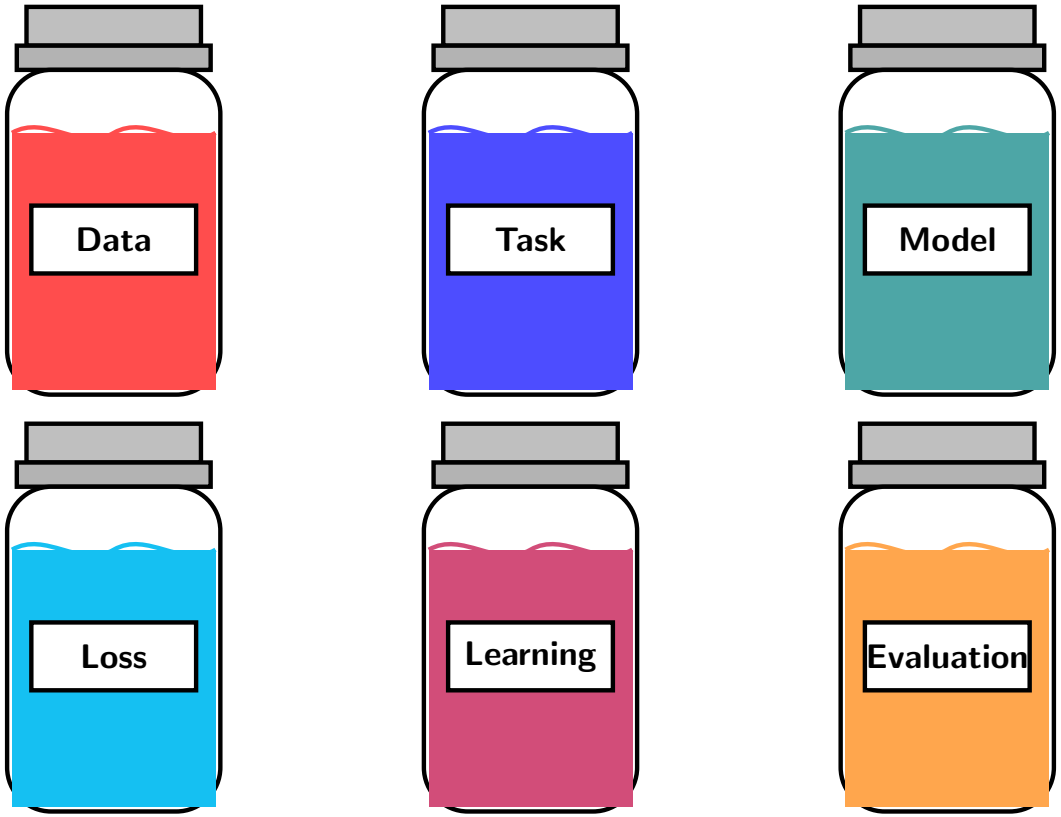
$$z_5 = Ux_5 + Ws_4 + b = [-0.1504,\ 0.7312]$$

$$s_5 = \sigma(z_5) = \boxed{[0.4625,\ 0.6751]}$$

Notice how $s_5$ still carries traces of every earlier word — through the chain

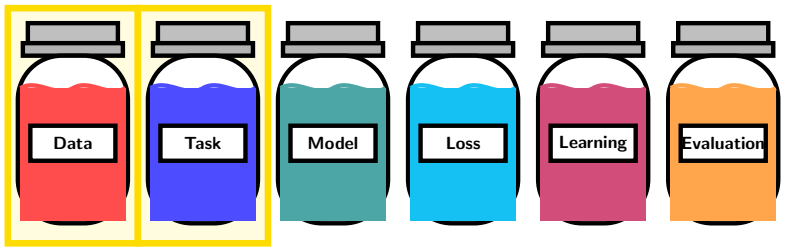
$$s_1 \to s_2 \to s_3 \to s_4 \to s_5.$$

# Six Jars



# Data and Tasks

*What are the main tasks for which RNNs are used for?*




$\underline{\text{Ram}}_{\text{NE}}$ went to $\underline{\text{Delhi}}_{\text{NE}}$ yesterday

"Ram" and "Delhi" are tagged as Named Entities (NE) — an example of a per-token sequence task.

# Data and Tasks: Main Task Categories

## ✓ Sequence Classification

(sentiment classification, video classification)

## ✓ Sequence Labelling

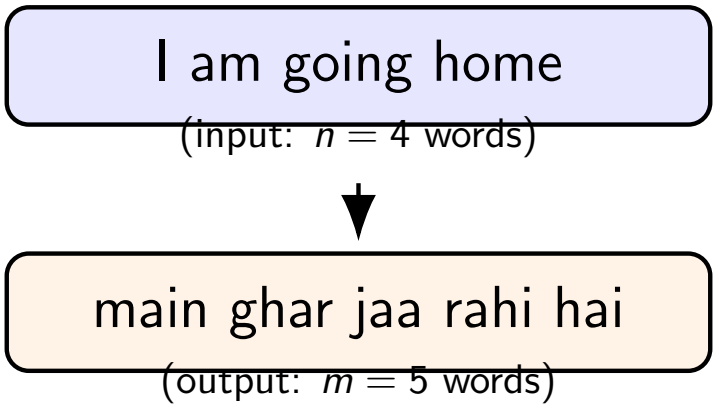
(part of speech tagging, named entity recognition)

## ✓ Sequence Generation

(machine translation, transliteration)

# Sequence Generation: A New Requirement

✓ **Sequence Generation** (machine translation, transliteration)

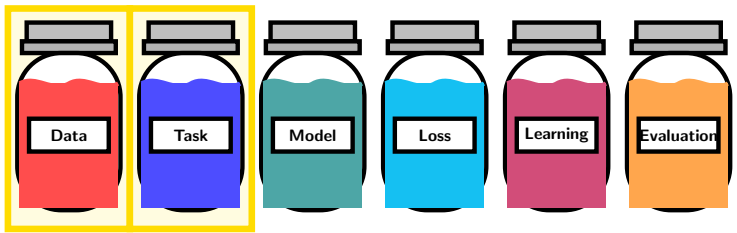



**The output sequence length is not the same as the input sequence length: $n \neq m$.**

All our earlier examples (POS tagging, sentiment classification) had output length tied to input length — Sequence Generation needs **independent** input/output lengths.

# Data and Tasks

*What does the data look like in the case of sequence classification?*




| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ | $y$ |
|---|---|---|---|---|---|---|---|---|
| The | first | half | was | very | boring | . | | 0 |
| Great | performance | by | all | the | lead | actors | . | 1 |
| The | visual | effects | were | stunning | . | | | 1 |
| The | movie | was | a | waste | of | time | . | 0 |

Each row is one review; blanks mean that review simply has fewer words — rows don't all have the same length.

# Data and Tasks: Sequence Classification

## ✓ Variable number of inputs (time steps)

**The problem:** different reviews have different numbers of words — the input length is not fixed.

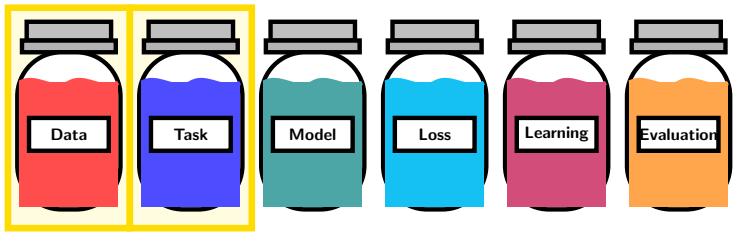
**The good news:** for sequence classification, the **output** is not variable — it's always a single label ($y = 0$ or $y = 1$), no matter how long the input sentence is.

## ✓ Words need to be converted to numbers

*(more on this next)*

# Data and Tasks

*How do you pre-process the data before feeding to the model?*




| $x_0$ | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ | $x_9$ | $y$ |
|---|---|---|---|---|---|---|---|---|---|---|
| <sos> | The | first | half | was | very | boring | . | <eos> | <pad> | 0 |
| <sos> | Great | performance | by | all | the | lead | actors | . | <eos> | 1 |
| <sos> | The | visual | effects | were | stunning | . | <eos> | <pad> | <pad> | 1 |
| <sos> | The | movie | was | a | waste | of | time | . | <eos> | 0 |

- ✓ Define special symbols: <sos>, <eos>, <pad>
- ✓ Prepend/Append <sos>, <eos> to each sequence
- ✓ Find maximum input length across all sequences (say, 10)
- ✓ Add special word <pad> to all shorter sequences so that they become of the same length (10, in this case)

# End of Sequence: <eos>

## Why do we need it?

The model needs an explicit signal for **"the sequence ends here."** Without it, especially for generation tasks (like translation), the model wouldn't know when to stop producing more output.

## A subtlety: punctuation can sometimes act like a natural boundary

Marks like "." or "!" often already signal the end of a sentence to a human reader — but the model still needs its own **explicit** <eos> token, since punctuation can appear *inside* a sequence too (e.g. "Wow! That was great."), not just at the very end.

<eos> is always placed at the true end of the sequence, regardless of what punctuation appears within it.

# Padding: <pad>

## Why do we need it?

Neural networks (and RNN training in batches) require all sequences in a batch to have the **same length**. But real sentences vary in length.

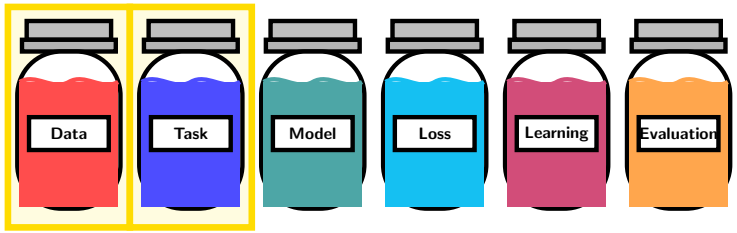
## The fix:

- Find the maximum sequence length in the dataset (e.g. 10)
- Fill every shorter sequence with the special token <pad> until it also reaches that length

<pad> carries no real meaning — it's just a placeholder so every sequence in a batch lines up to the same length. (Its contribution to the loss is typically masked out during training.)

# Data and Tasks

*How do you convert words to numbers? What are 1-hot encodings?*




| word | id |
|---|---|
| <sos> | 1 |
| <eos> | 2 |
| <pad> | 3 |
| the | 4 |
| first | 5 |
| half | 6 |
| ... | ... |
| time | 24 |

# Data and Tasks: Building the One-Hot Encoding

- ✓ Lower case all words
- ✓ Compute the total number of unique words across all sentences (say, $L \rightarrow 24$ in the above case)
- ✓ Assign a unique id to each word (between 1 to $L$)
- ✓ Represent each word using an $L$-dimensional binary vector with only the bit corresponding to the word id set to 1

# One-Hot Encoding: Worked Example

**With $L = 24$, "the" has id 4:**

$$x_{\text{the}} = [\underbrace{0, 0, 0}_{3}, \ \mathbf{1}, \ \underbrace{0, 0, \ldots, 0}_{20}] \in \mathbb{R}^{24}$$

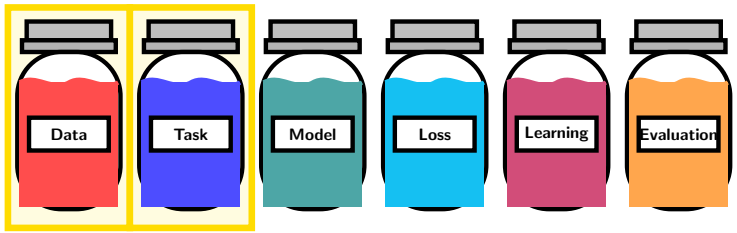
**"first" has id 5:**

$$x_{\text{first}} = [\underbrace{0, 0, 0, 0}_{4}, \ \mathbf{1}, \ \underbrace{0, 0, \ldots, 0}_{19}] \in \mathbb{R}^{24}$$

Every word becomes a 24-dimensional vector, with exactly one "1" at its own id position — and zeros everywhere else.

# Data and Tasks

*What does the final input look like?*




**Every sequence now has exactly 10 tokens (after padding), each a 24-dim one-hot vector:**

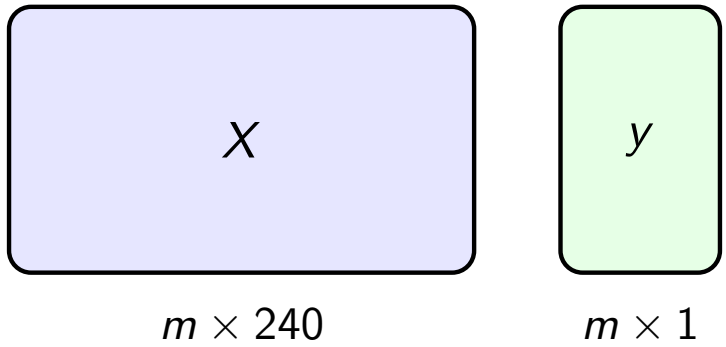
$$10 \text{ tokens} \times 24 \text{ dims} = 240$$

Concatenating all 10 one-hot vectors end-to-end gives one flat 240-dimensional vector per sequence.

# Data and Tasks: The Dataset as a Matrix

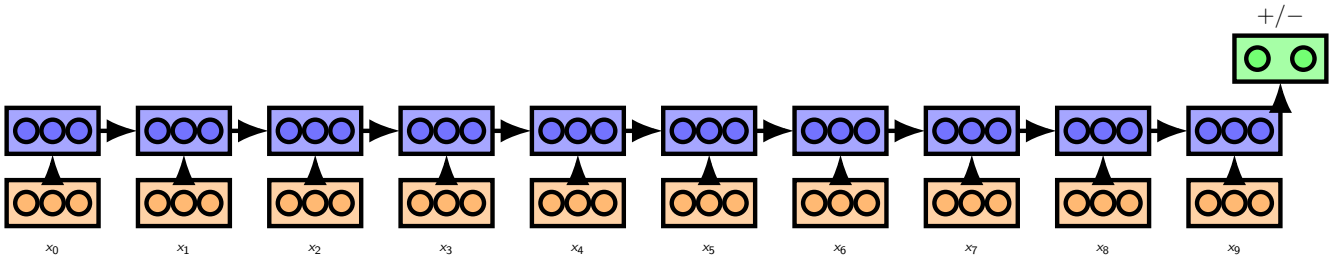
**Stacking every example's flattened 240-dim vector into rows:**

$$X \in \mathbb{R}^{m \times 240}, \qquad y \in \mathbb{R}^{m}$$




$m$ = number of sentences in the dataset; each row of $X$ is one sentence's flattened one-hot representation, paired with its label in $y$.

# But This Is Not How the RNN Reads It

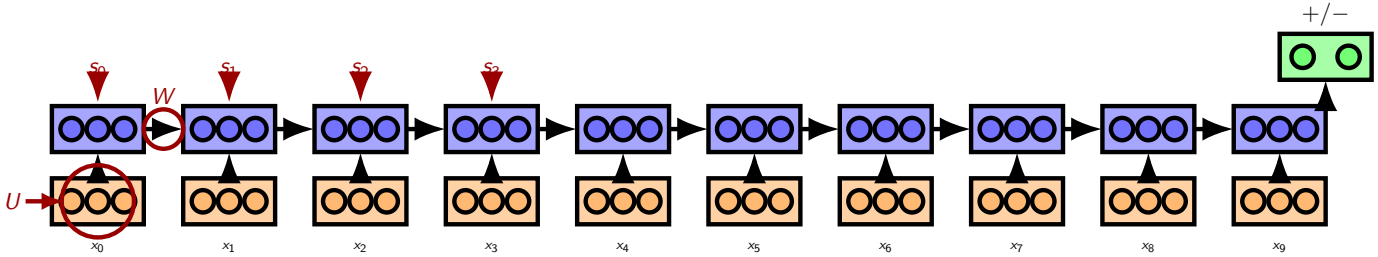



**The RNN reads one 24-dim one-hot vector at a time — not the whole 240-dim vector at once:**

- 10 separate inputs, $x_0, x_1, \ldots, x_9$, each $\in \mathbb{R}^{24}$
- **Not** one flattened input $\in \mathbb{R}^{240}$

240 describes how the data is *stored*; the RNN still *consumes* it sequentially, 24 dimensions at a time — this is exactly what preserves parameter sharing and avoids the naive-concatenation problem.

# Data and Tasks: This Is How the Computation Happens

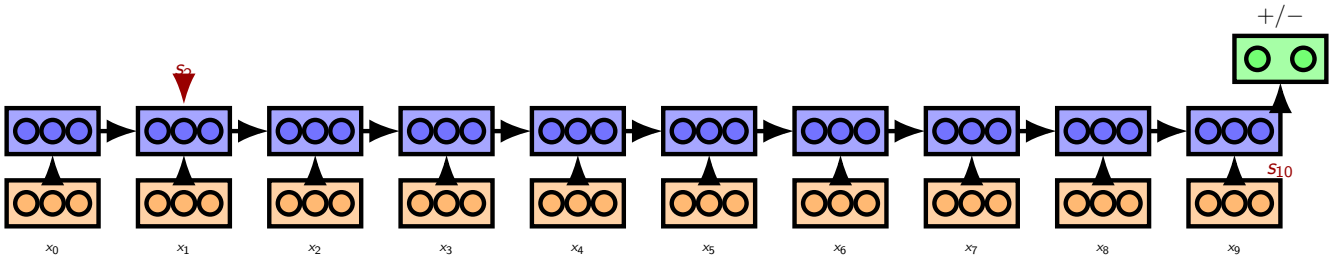



**Starting from $s_0 = [0, 0, \ldots, 0]$, each step applies the same recurrence:**

$$s_1 = \sigma(Ux_1 + Ws_0 + b)$$

Then $s_2 = \sigma(Ux_2 + Ws_1 + b)$, $s_3 = \sigma(Ux_3 + Ws_2 + b)$, and so on — the **same** $U, W, b$ every time, now applied to real 24-dimensional one-hot inputs.

# Data and Tasks: The Recurrence Continues

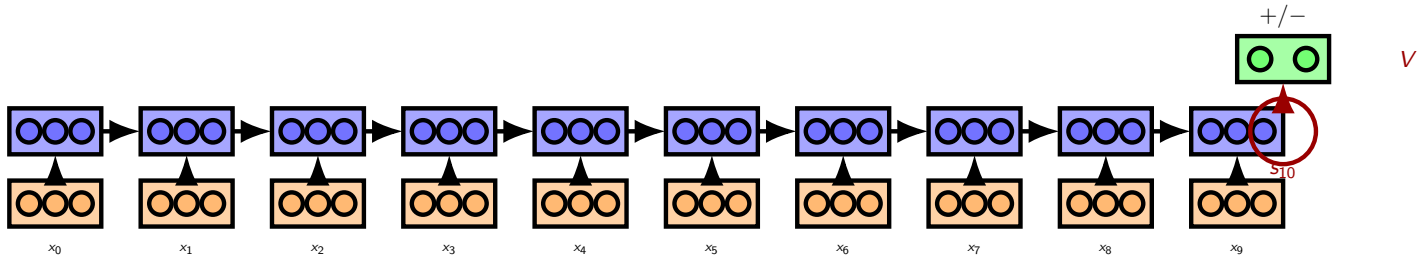



**The same recurrence, now for the second timestep:**

$$s_2 = \sigma(Ux_2 + Ws_1 + b)$$

$U, W, b$ are still exactly the same as in the $s_1$ step — the only things that change are $x_2$ (the current input) and $s_1$ (the hidden state carried forward). This continues all the way to $s_{10}$, whose output feeds the final $+/-$ classification.

# Data and Tasks: Computing the Final Prediction



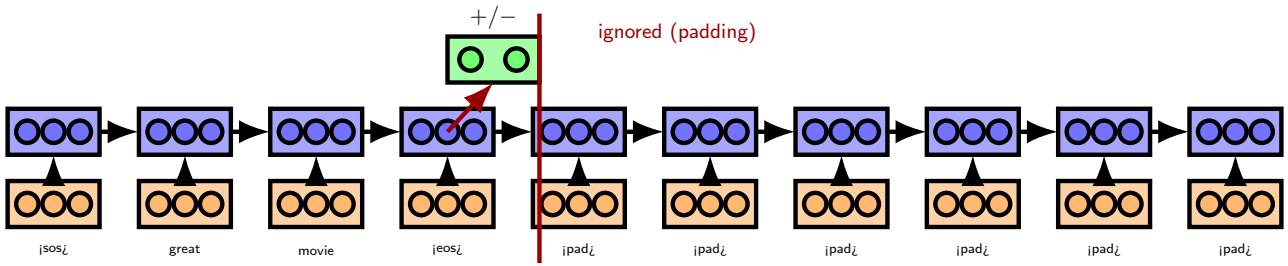

**Only the final hidden state $s_{10}$ is used to produce the output:**

$$\hat{y} = O(Vs_{10} + c)$$

This is the many-to-one pattern from earlier: all 10 intermediate hidden states pass information forward, but only $s_{10}$ — which has now seen the **entire** sentence — is used to compute the sentiment prediction $\hat{y}$.

# Data and Tasks

*Aren't we corrupting the inputs by adding <pad>s?*

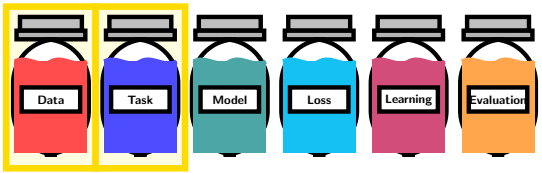



- ✓ Padding was only to ensure the input matrix is of uniform size
- ✓ Computations are done only for the required number of time steps

The output is read at the <eos> position — here, $s_3$ — **not** at the very last position $s_{10}$. The <pad> steps that follow are never used to compute $\hat{y}$.

# Data and Tasks

*What does the data look like in the case of sequence labeling?*



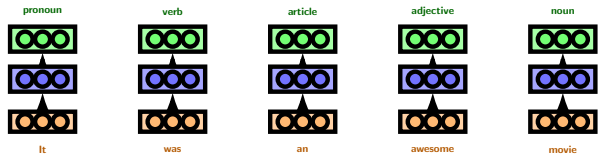

| $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ |
|---|---|---|---|---|---|---|---|
| The | first | half | was | very | boring | . | |
| Great | performance | by | all | the | lead | actors | . |
| The | background | music | was | awesome | . | | |
| The | movie | was | a | waste | of | time | . |

| $y_1$ | $y_2$ | $y_3$ | $y_4$ | $y_5$ | $y_6$ | $y_7$ | $y_8$ |
|---|---|---|---|---|---|---|---|
| DT | AJ | NN | VB | JJ | JJ | PC | |
| AJ | NN | PP | PN | DT | JJ | NN | PC |
| DT | JJ | BB | VB | JJ | PC | | |
| DT | NN | VB | DT | NN | PP | NN | PC |

Now **every** word gets its own output label — not just one label per sentence.

# Data and Tasks: Sequence Labeling

- ✓ Variable number of inputs (time steps)
- ✓ **Variable number of outputs (time steps)**
- ✓ Words need to be converted to numbers
- ✓ **Labels/outputs need to be converted to numbers**




Unlike classification, here **both** the number of inputs **and** outputs vary together — one output per input, whatever the sentence length.

# Data and Tasks

*How do you pre-process the data before feeding to the model?*

$X$ (input)

| $x_0$ | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ | $x_7$ | $x_8$ | $x_9$ |
|---|---|---|---|---|---|---|---|---|---|
| <sos> | The | first | half | was | very | boring | . | <eos> | <pad> |
| <sos> | Great | performance | by | all | the | lead | actors | . | <eos> |

$Y$ (output)

| $y_0$ | $y_1$ | $y_2$ | $y_3$ | $y_4$ | $y_5$ | $y_6$ | $y_7$ | $y_8$ | $y_9$ |
|---|---|---|---|---|---|---|---|---|---|
| <sos> | DT | AJ | NN | VB | JJ | JJ | PC | <eos> | <pad> |
| <sos> | AJ | NN | PP | PN | DT | JJ | NN | PC | <eos> |

- ✓ Define special symbols/labels: <sos>, <eos>, <pad>
- ✓ Prepend/Append <sos>, <eos> to each **input/output** sequence
- ✓ Find maximum input length across all sequences (say, 10)
- ✓ Add special word/label <pad> to all shorter sequences so they become the same length

Both $X$ and $Y$ are now $m \times 10$ matrices.

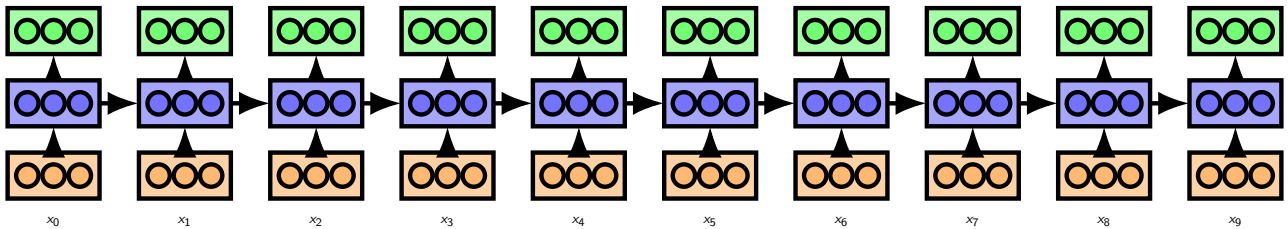
# Data and Tasks: One-Hot Encoding for Words *and* Labels

*How do you convert words/labels to numbers? What are 1-hot encodings?*

| | **word** | | **label** |
|---|---|---|---|
| word | id | label | id |
| <sos> | 1 | <sos> | 1 |
| <eos> | 2 | <eos> | 2 |
| <pad> | 3 | <pad> | 3 |
| the | 4 | DT | 4 |
| first | 5 | JJ | 5 |
| half | 6 | NN | 6 |
| ... | ... | ... | ... |

Both words *and* labels get their own vocabulary and their own one-hot encoding — since now the **output** is also a sequence of discrete symbols, just like the input.

# Data and Tasks: Computation — One Output Per Step



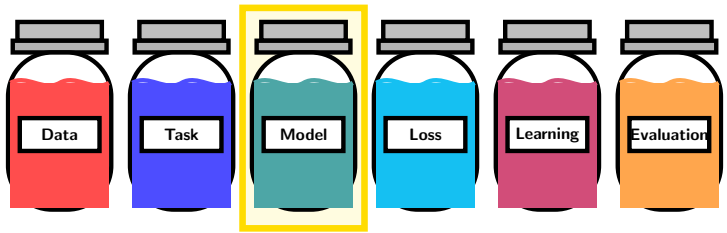

**Unlike sequence classification, an output is computed at every timestep:**

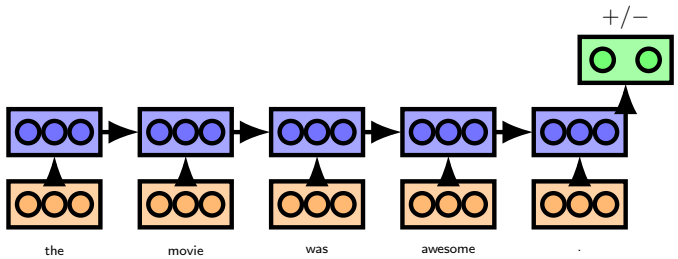
$$s_i = \sigma(Ux_i + Ws_{i-1} + b), \qquad y_i = O(Vs_i + c)$$

$y_1 = O(Vs_1 + c)$, $y_2 = O(Vs_2 + c)$, $\ldots$, $y_9 = O(Vs_9 + c)$ — the same $U, V, W, b, c$ every time, but now the output layer is used at **every** position, not just the last one.

# Model

*Can we write the output as a function of the input for* **sequence classification***?*




$$s_i = \sigma(Ux_i + Ws_{i-1} + b)$$

$$\hat{y} = O(Vs_T + c)$$

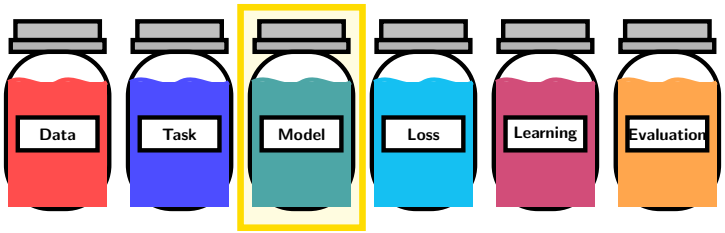
$T$ = sequence length

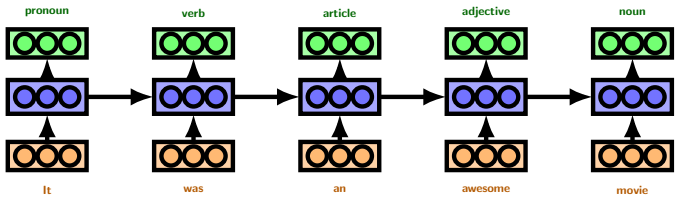
$$\hat{y} = \hat{f}(x), \qquad x = [x_1, x_2, \ldots, x_n]$$

The entire recurrence collapses into one function of the whole input sequence.

# Model

*Can we write the output as a function of the input for* **sequence labeling***?*




$$s_i = \sigma(Ux_i + Ws_{i-1} + b)$$

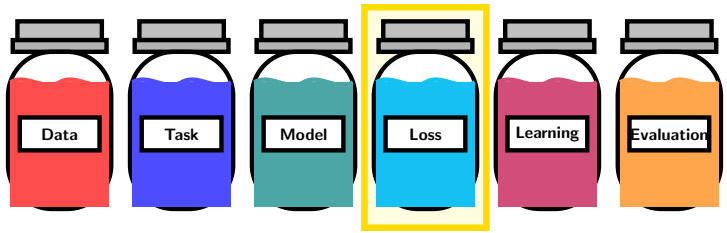
$$\hat{y}_i = O(Vs_i + c)$$

$$\hat{y}_i = \hat{f}(x_1, x_2, \ldots, x_i)$$

Unlike classification, each output $\hat{y}_i$ depends only on the inputs seen **up to that point** — not the entire sequence.

# Loss Function

*What is the loss function for sequence classification tasks?*




## Cross-entropy loss (one example):

$$\mathcal{L}(\theta) = -\sum_{i=0}^{1} y[i] \log \hat{y}[i]$$

## Given (for one sentence, e.g. "the movie was awesome ."):

$$y = [1, 0] \qquad \text{(true class: class 0)}$$

$$\hat{y} = [0.7, 0.3] \qquad \text{(predicted probabilities)}$$

# Loss Function: Expanding the Sum, Term by Term

## Write out both terms of the sum explicitly:

$$\begin{aligned}\mathcal{L}(\theta) &= -\big[\, y[0]\log\hat{y}[0] + y[1]\log\hat{y}[1] \,\big] \\ &= -\big[\, (1)\log(0.7) + (0)\log(0.3) \,\big] \\ &= -\big[\, \log(0.7) + 0 \,\big] \\ &= -\log(0.7)\end{aligned}$$

Since $y$ is one-hot, every term except the one matching the **true class** $c$ is multiplied by 0 and vanishes. What survives is just $-\log\hat{y}[c]$ — written compactly as $-\log y_c$.

# Loss Function: The Numeric Value

## Plugging in $\hat{y}[c] = \hat{y}[0] = 0.7$:

$$\mathcal{L}(\theta) = -\log y_c = -\log(0.7) \approx 0.357$$

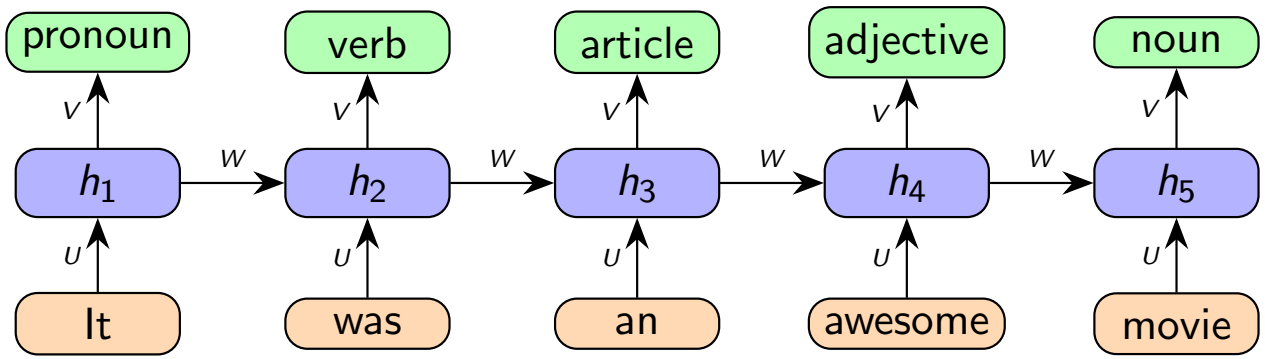
## Averaged over the whole dataset ($m$ examples):

$$\mathcal{L}(\theta) = -\frac{1}{m}\sum_{i=1}^{m} \log y_{ic}$$

Each example contributes $-\log$(predicted probability of its own true class); the final loss is simply the average of these values across all $m$ examples.

# Problem: Sequence Labeling

**Sentence:** "It was an awesome movie"
**Task:** assign a POS tag to every word, one output per timestep.




## At each timestep $j$

True label $y_j$ is a one-hot vector; the network predicts a probability distribution $\hat{y}_j$ over all tag classes.

# Targets vs. Predictions (per timestep)

## True label $y_j$ (one-hot)

$$y_1 = [1, 0, 0, 0, 0] \quad \text{(pronoun)}$$
$$y_2 = [0, 1, 0, 0, 0] \quad \text{(verb)}$$
$$y_3 = [0, 0, 1, 0, 0] \quad \text{(article)}$$
$$y_4 = [0, 0, 0, 1, 0] \quad \text{(adjective)}$$
$$y_5 = [0, 0, 0, 0, 1] \quad \text{(noun)}$$

## Predicted $\hat{y}_j$ (softmax output)

$$\hat{y}_1 = [0.2, 0.5, 0.1, 0.1, 0.1]$$
$$\hat{y}_2 = [0.3, 0.4, 0.1, 0.1, 0.1]$$
$$\hat{y}_3 = [0.1, 0.1, 0.3, 0.4, 0.1]$$
$$\hat{y}_4 = [0.2, 0.2, 0.1, 0.3, 0.1]$$
$$\hat{y}_5 = [0.2, 0.1, 0.2, 0.1, 0.3]$$

## Key idea

For each timestep $j$, only the probability the model assigned to the **true** class $c$ matters for the loss at that step, e.g. for $y_2$ the true class is "verb" so we read off $\hat{y}_{2,\text{verb}} = 0.4$.

# Loss Function for Sequence Labeling

## Cross-entropy loss, summed over time, averaged over samples

$$\mathcal{L}(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{T}\log \hat{y}_{i,j,c}$$

## Symbols

- $m$ — number of training sentences (samples) in the batch
- $T$ — number of timesteps (words) in a sentence
- $c$ — index of the true tag class at position $j$
- $\hat{y}_{i,j,c}$ — predicted probability of the true class $c$ at timestep $j$ of sample $i$

## How to read it

- Inner sum ($j = 1$ to $T$): add up the per-word loss across the whole sentence
- Outer sum ($i = 1$ to $m$): add up over every sentence in the batch
- $-\log(\cdot)$: penalizes low confidence on the correct tag
- Averaging by $\frac{1}{m}$ keeps loss comparable across batch sizes

# Worked Calculation (single sentence, $m = 1$)

For this sentence $T = 5$, so the loss is the average of five per-word terms:

$$\mathcal{L}(\theta) = -\frac{1}{5}\Big[\log(0.5) + \log(0.4) + \log(0.3) + \log(0.3) + \log(0.3)\Big]$$

## Per-word terms

- $j = 1$ (It $\rightarrow$ pronoun): $\hat{y}_{1,\text{pronoun}} = 0.5 \Rightarrow -\log(0.5) \approx 0.693$
- $j = 2$ (was $\rightarrow$ verb): $\hat{y}_{2,\text{verb}} = 0.4 \Rightarrow -\log(0.4) \approx 0.916$
- $j = 3$ (an $\rightarrow$ article): $\hat{y}_{3,\text{article}} = 0.3 \Rightarrow -\log(0.3) \approx 1.204$
- $j = 4$ (awesome $\rightarrow$ adjective): $\hat{y}_{4,\text{adjective}} = 0.3 \Rightarrow -\log(0.3) \approx 1.204$
- $j = 5$ (movie $\rightarrow$ noun): $\hat{y}_{5,\text{noun}} = 0.3 \Rightarrow -\log(0.3) \approx 1.204$

## Result

$$\mathcal{L}(\theta) \approx \frac{0.693 + 0.916 + 1.204 + 1.204 + 1.204}{5} \approx 1.044$$

# Can We Use the Same Gradient Descent Algorithm?

## Learning algorithm (unchanged in structure)

- Initialise parameters
- Iterate over data:
  - compute $\hat{y}$
  - compute $\mathcal{L}$
  - update each parameter: $\text{param} = \text{param} - \eta\,\Delta(\text{param})$
- repeat till satisfied

## Earlier (single neuron)

- Parameters: $w, b$
- Loss: $\mathcal{L}(w, b)$

## Now (sequence labeling)

- Parameters: $W, U, V$ (matrices, shared across timesteps)
- Loss: $\mathcal{L}(W, U, V)$

# Parameters of the Sequence Labeling Network

## Each weight is now a matrix, not a scalar

- $U \in \mathbb{R}^{d \times n}$ — input-to-hidden weights; $n$ = input (one-hot vocab) size, $d$ = hidden size
- $W \in \mathbb{R}^{d \times d}$ — hidden-to-hidden (recurrent) weights
- $V \in \mathbb{R}^{k \times d}$ — hidden-to-output weights; $k$ = number of output classes (tags)

## Vectors at each timestep

- $x \in \mathbb{R}^{n \times 1}$ — input word (one-hot)
- $h \in \mathbb{R}^{d \times 1}$ — hidden state
- $\hat{y} \in \mathbb{R}^{k \times 1}$ — predicted tag distribution

## Old vs. new notation

- Earlier: $w, b$
- Now:
  $w_{11}, w_{12}, \ldots, u_{11}, u_{12}, \ldots, v_{11}, v_{12}, \ldots$
- Every entry of $W, U, V$ is updated by gradient descent

# Why This Matters: Parameter Count

## Total trainable parameters

$$\underbrace{n \cdot d}_{U} + \underbrace{d \cdot d}_{W} + \underbrace{k \cdot d}_{V} \quad \text{vs.} \quad \underbrace{d}_{w} + \underbrace{k}_{b}$$

## Takeaway

- A single neuron has $d + k$ parameters (roughly linear in the input size)
- The sequence labeling network has $nd + d^2 + kd$ parameters — quadratic growth from $W$ alone
- Same gradient descent recipe applies, but $\Delta w$ becomes $\Delta W$, $\Delta U$, $\Delta V$ — full matrix gradients computed via backpropagation through time

# Learning Algorithm: Element-Wise Updates

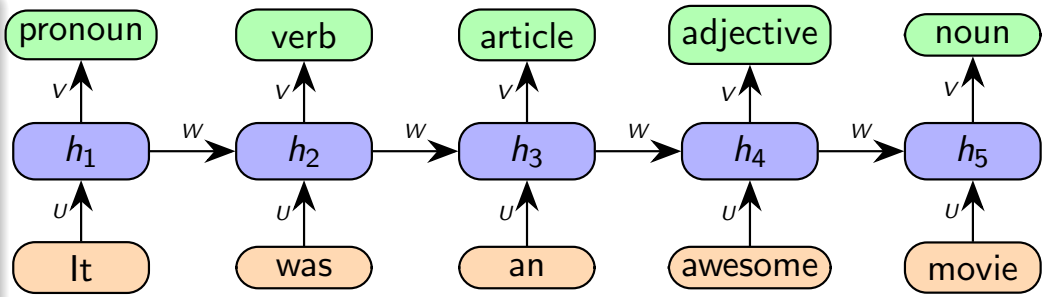
## Algorithm

**Initialise** $w, b$

**Iterate over data:**

- compute $\hat{y}$
- compute $\mathcal{L}(w, b)$
- $w_{11} = w_{11} - \eta \Delta w_{11}$
- $u_{12} = u_{12} - \eta \Delta u_{12}$
- ...
- $v_{13} = v_{13} - \eta \Delta v_{13}$

**till satisfied**




## Idea

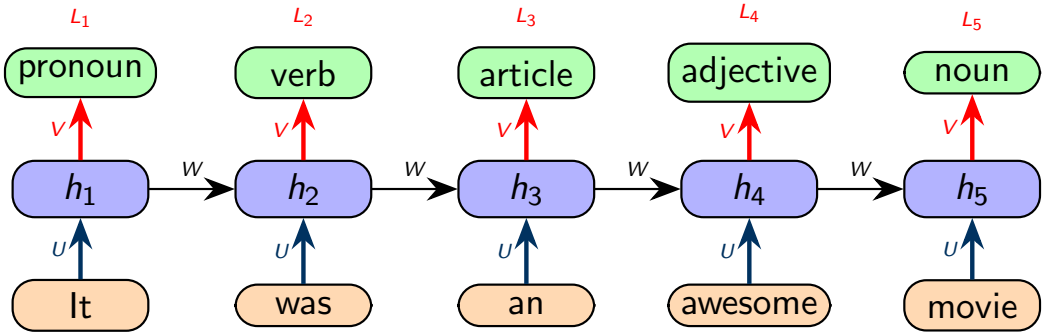
Every individual entry $w_{ij}, u_{ij}, v_{ij}$ of $W, U, V$ gets its own gradient descent update — same rule as before, just applied element by element across every matrix.

## Notation shift

Earlier: $w, b \longrightarrow$ Now: $w_{11}, w_{12}, \ldots, u_{11}, u_{12}, \ldots, v_{11}, v_{12}, \ldots$

# Derivative of the Loss w.r.t. $V$

*How do you compute the derivative of the loss function w.r.t. $V$?*




## $V$ is shared across every timestep

- The **same** $V$ is used to produce $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_T$, so every timestep's loss $L_t$ depends on $V$
- Total loss is a sum of per-timestep losses: $\mathcal{L} = L_1 + L_2 + \cdots + L_T$
- By the sum rule of differentiation, the gradient is also a sum:

$$\frac{\partial \mathcal{L}}{\partial V} = \frac{\partial L_1}{\partial V} + \frac{\partial L_2}{\partial V} + \cdots + \frac{\partial L_T}{\partial V} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial V}$$

# Chain Rule at a Single Timestep

## Full loss (recap)

$$\mathcal{L}(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{T}\log \hat{y}_{i,j,c}$$

## At timestep $t$: expand $\frac{\partial L_t}{\partial V}$ via chain rule

Let $a_t = Vh_t$ (pre-softmax logits) and $\hat{y}_t = \text{softmax}(a_t)$. Then:

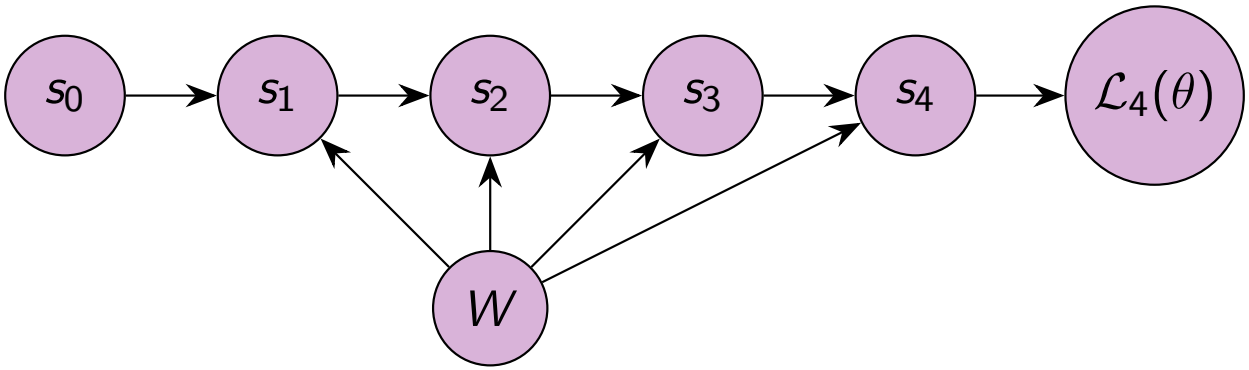
$$\frac{\partial L_t}{\partial V} = \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial a_t} \cdot \frac{\partial a_t}{\partial V}$$

## Putting it together

$$\frac{\partial \mathcal{L}}{\partial V} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial \hat{y}_t} \cdot \frac{\partial \hat{y}_t}{\partial a_t} \cdot \frac{\partial a_t}{\partial V}$$

# Derivative of the Loss w.r.t. $W$

*How do you compute the derivative of the loss function w.r.t. $W$?*




## In simple words

- $V$ only touches the output at its own timestep, so its gradient at each step is separate
- $W$ is reused to move from **every** hidden state to the next: $s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$
- So $s_4$ was shaped by $W$ not once, but four times over: directly, and through $s_3, s_2, s_1$ as well
- We must add up **all** of $W$'s paths of influence, not just the most recent one

# Adding Up Every Path $W$ Took

## Step 1: how did $W$ reach $s_4$?

$W$ shaped $s_4$ through four different routes:

$$\frac{\partial s_4}{\partial W} = \underbrace{\frac{\partial s_4}{\partial W}}_{\text{direct}} + \underbrace{\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial W}}_{\text{via } s_3} + \underbrace{\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial W}}_{\text{via } s_3, s_2} + \underbrace{\frac{\partial s_4}{\partial s_3}\frac{\partial s_3}{\partial s_2}\frac{\partial s_2}{\partial s_1}\frac{\partial s_1}{\partial W}}_{\text{via } s_3, s_2, s_1}$$

## Step 2: write it as one tidy sum

$$\frac{\partial s_4}{\partial W} = \sum_{k=1}^{4} \frac{\partial s_4}{\partial s_k}\frac{\partial s_k}{\partial W}$$

## Step 3: bring back the loss

$$\frac{\partial \mathcal{L}_4(\theta)}{\partial W} = \frac{\partial \mathcal{L}_4(\theta)}{\partial s_4} \sum_{k=1}^{4} \frac{\partial s_4}{\partial s_k}\frac{\partial s_k}{\partial W}$$

# The General Rule: Backpropagation Through Time

## Same idea for any timestep $t$

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial W} = \frac{\partial \mathcal{L}_t(\theta)}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial s_k}{\partial W}$$

## What $\partial s_t / \partial s_k$ really is

$$\frac{\partial s_4}{\partial s_1} = \frac{\partial s_4}{\partial s_3} \cdot \frac{\partial s_3}{\partial s_2} \cdot \frac{\partial s_2}{\partial s_1}$$

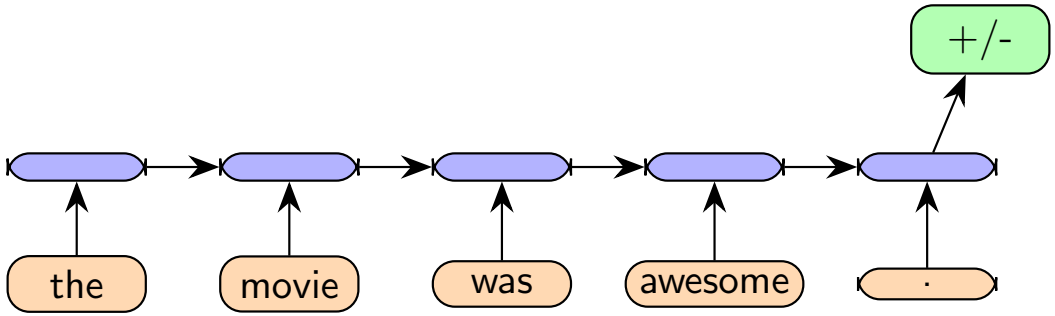
A chain of one-step-back derivatives, hopping backward from $s_4$ all the way to $s_1$

## Why the name

- We walk backward through the timesteps to collect $W$'s influence
- This repeated backward walk is called **Backpropagation Through Time (BPTT)**
- Total gradient: sum this over every timestep's loss, $t = 1, \ldots, T$

# Evaluation: Sequence Classification

*How do you check the performance for sequence classification tasks?*



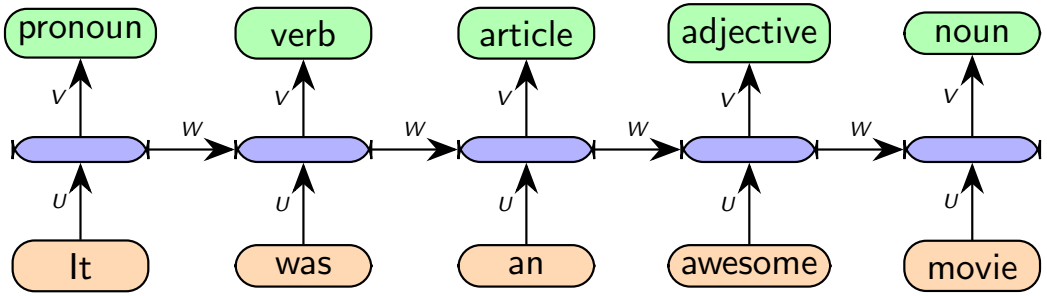

## In simple words

- Only the **last** timestep produces an output, one label for the whole sentence (e.g. sentiment: $+$ or $-$)
- This is just an ordinary classification problem in disguise, one prediction, one ground truth, per example
- So the standard metric applies directly:

$$\text{Accuracy} = \frac{\text{number of sentences correctly classified}}{\text{total number of sentences}}$$

# Evaluation: Sequence Labeling

*How do you check the performance for sequence labeling tasks?*




## Why one number is not enough

- Every single word gets its own predicted label, so there are many predictions to score per sentence, not just one
- ✓ **Overall Accuracy** — fraction of **all** word-level tags correct, across every word in every sentence
- ✓ **Accuracy Per Class** — accuracy computed separately for each tag (how good are we at "noun" vs. "verb" vs. "article"?)
- ✓ **Confusion Matrix** — a table showing exactly which tags get mistaken for which other tags

# Confusion Matrix

## What it is

A grid with **true labels** as rows and **predicted labels** as columns. Each cell counts how many times a word with that true tag was predicted as that column's tag.

| | **P** | **V** | **Ar** | **Ad** | **N** |
|---|---|---|---|---|---|
| **P** | 10 | 0 | 0 | 0 | 0 |
| **V** | 0 | 10 | 0 | 0 | 0 |
| **Ar** | 0 | 1 | 9 | 0 | 0 |
| **Ad** | 0 | 0 | 0 | 10 | 0 |
| **N** | 0 | 0 | 0 | 0 | 10 |

## How to read it

- Rows = true tag, Columns = predicted tag (P = pronoun, V = verb, Ar = article, Ad = adjective, N = noun)
- Shaded diagonal = correct predictions; the model got these right every time except one

*Solving the problem of vanishing gradients*

# Long Short Term Memory (LSTM) and Gated Recurrent Units (GRU)

# Exploding / Vanishing Gradients

*Taking a closer look at the derivative w.r.t. W*

## Recall

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial W} = \frac{\partial \mathcal{L}_t(\theta)}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial s_k}{\partial W} \qquad \begin{aligned} a_j &= Ux_j + Ws_{j-1} + b \\ s_j &= \sigma(a_j) \end{aligned}$$

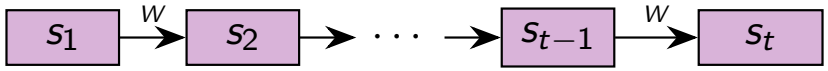
## The term that decides everything: $\frac{\partial s_t}{\partial s_k}$

This one factor tells us how strongly an early hidden state $s_k$ still echoes inside a later state $s_t$. To understand training on long sequences, we need to look closely at what this term actually is.

# $\partial s_t / \partial s_k$ Is Not One Step, It's Many

## $s_t$ only talks to $s_k$ through every state in between

$$\frac{\partial s_t}{\partial s_k} = \frac{\partial s_t}{\partial s_{t-1}} \cdot \frac{\partial s_{t-1}}{\partial s_{t-2}} \cdot \frac{\partial s_{t-2}}{\partial s_{t-3}} \cdots \frac{\partial s_{k+1}}{\partial s_k} = \prod_{j=k+1}^{t} \frac{\partial s_j}{\partial s_{j-1}}$$




## In simple words

Every arrow between two neighbouring states is one small derivative. Going from $s_k$ to $s_t$ means multiplying all of these small derivatives together, one for every step in between.

# A Story: What If $t = 20$ and $k = 1$?

## The question

Suppose word 20 in a sentence needs to remember something about word 1. How many terms is $\frac{\partial s_{20}}{\partial s_1}$ actually made of?

## Count the hops

$$\frac{\partial s_{20}}{\partial s_1} = \frac{\partial s_{20}}{\partial s_{19}} \cdot \frac{\partial s_{19}}{\partial s_{18}} \cdots \frac{\partial s_2}{\partial s_1}$$

That is **19 hops**, so **19 numbers multiplied together**, one for every state that sits between $s_1$ and $s_{20}$.

## Why this is dangerous

- Each hop is roughly $\sigma'(a_j) \cdot W$, and $\sigma'(\cdot) \leq 0.25$ for the sigmoid
- If most hops are **less than 1**: multiplying 19 small numbers together shrinks the product toward **0** — the gradient **vanishes**, and word 1 is

# Takeaway: Why Plain RNNs Struggle

## The core problem

- A plain RNN reuses the same $W$ at every step, so remembering something from far in the past means multiplying **many** small Jacobians together
- Long sequences $\Rightarrow$ long products $\Rightarrow$ gradients either vanish (early words forgotten) or explode (training blows up)
- This is exactly the problem the next section addresses

**Next: LSTM and GRU**

*architectures designed to control this product, not fight it*

# Exploding / Vanishing Gradients

*Taking a closer look at the derivative w.r.t. W*

## Recall

$$\frac{\partial \mathcal{L}_t(\theta)}{\partial W} = \frac{\partial \mathcal{L}_t(\theta)}{\partial s_t} \sum_{k=1}^{t} \frac{\partial s_t}{\partial s_k} \frac{\partial s_k}{\partial W} \qquad \begin{aligned} a_j &= Ux_j + Ws_{j-1} + b \\ s_j &= \sigma(a_j) \end{aligned}$$

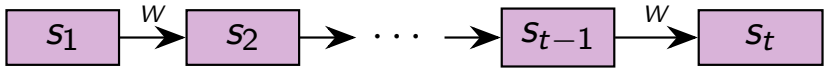
## The term that decides everything: $\frac{\partial s_t}{\partial s_k}$

This one factor tells us how strongly an early hidden state $s_k$ still echoes inside a later state $s_t$. To understand training on long sequences, we need to look closely at what this term actually is.

# $\partial s_t / \partial s_k$ Is Not One Step, It's Many

## $s_t$ only talks to $s_k$ through every state in between

$$\frac{\partial s_t}{\partial s_k} = \frac{\partial s_t}{\partial s_{t-1}} \cdot \frac{\partial s_{t-1}}{\partial s_{t-2}} \cdot \frac{\partial s_{t-2}}{\partial s_{t-3}} \cdots \frac{\partial s_{k+1}}{\partial s_k} = \prod_{j=k+1}^{t} \frac{\partial s_j}{\partial s_{j-1}}$$




## In simple words

Every arrow between two neighbouring states is one small derivative. Going from $s_k$ to $s_t$ means multiplying all of these small derivatives together, one for every step in between.

# A Story: What If $t = 20$ and $k = 1$?

## The question

Suppose word 20 in a sentence needs to remember something about word 1. How many terms is $\frac{\partial s_{20}}{\partial s_1}$ actually made of?

## Count the hops

$$\frac{\partial s_{20}}{\partial s_1} = \frac{\partial s_{20}}{\partial s_{19}} \cdot \frac{\partial s_{19}}{\partial s_{18}} \cdots \frac{\partial s_2}{\partial s_1}$$

That is **19 hops**, so **19 numbers multiplied together**, one for every state that sits between $s_1$ and $s_{20}$.

## Why this is dangerous

- Each hop is roughly $\sigma'(a_j) \cdot W$, and $\sigma'(\cdot) \leq 0.25$ for the sigmoid
- If most hops are **less than 1**: multiplying 19 small numbers together shrinks the product toward **0** — the gradient **vanishes**, and word 1 is

# Takeaway: Why Plain RNNs Struggle

## The core problem

- A plain RNN reuses the same $W$ at every step, so remembering something from far in the past means multiplying **many** small Jacobians together
- Long sequences $\Rightarrow$ long products $\Rightarrow$ gradients either vanish (early words forgotten) or explode (training blows up)
- This is exactly the problem the next section addresses

**Next: LSTM and GRU**

*architectures designed to control this product, not fight it*

# The Whiteboard Analogy

*Over time the whiteboard becomes so messy that you can't extract any information*

# The Whiteboard Analogy

*Selectively read, write and forget*

$a = 1 \quad b = 3 \quad c = 5 \quad d = 11$ **Compute** $ac(bd + a) + ad$

1. $ac$
2. $bd$
3. $bd + a$
4. $ac(bd + a)$
5. $ad$
6. $ac(bd + a) + ad$

$$ac = 5$$
$$bd = 33$$
$$bd + a = 34$$
$$ac(bd + a) = 170$$
$$ad = 11$$

## Strategy

- ✓ Selectively **write** on the board
- ✓ Selectively **read** the already written content
- ✓ Selectively **forget** (erase) some content

## Why this matters

Step 1 writes $ac = 5$; step 4 **reads** it back. Once step 4 is done, $bd + a$ is no longer needed and can be **forgotten**. This is exactly what an LSTM's gates let the network do with its cell state, instead of overwriting everything at every timestep.

# Dealing with Longer Sequences

*Can we use a similar strategy in RNNs?*

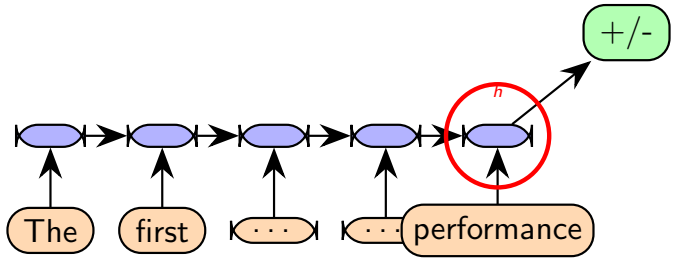
Since the RNN also has a finite state size, we need to figure out a way to allow it to selectively read, write and forget

**Strategy**

- ✓ Selectively **write** to the state
- ✓ Selectively **read** the already written content
- ✓ Selectively **forget** (erase) some content

# Dealing with Longer Sequences

*What is our wishlist?*

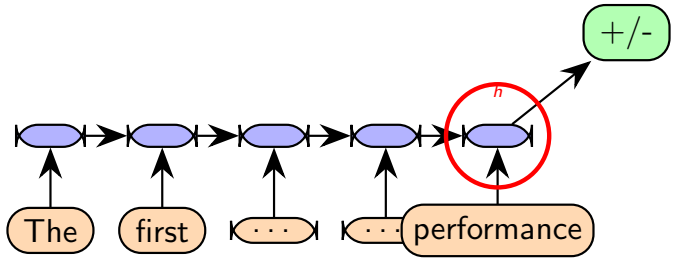



## Wishlist, point 1: Selective write

Every new word is a candidate to update the finite-sized state vector $h$ — the network should choose **how much** of the current word's information actually gets written in, rather than overwriting $h$ wholesale at every step.

# Dealing with Longer Sequences

*What is our wishlist?*

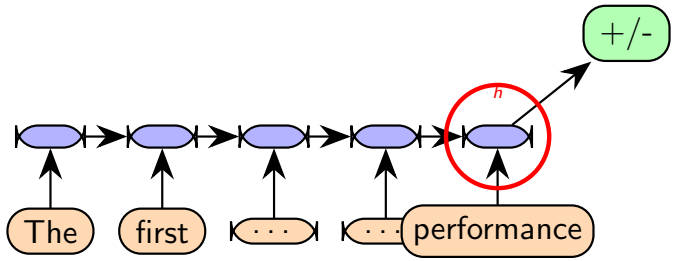



## Wishlist, point 2: Selective read

When it's time to predict, the network should be able to pull out exactly the useful parts of $h$ that were written earlier, not just whatever happens to be sitting there after being repeatedly morphed at every step.

# Dealing with Longer Sequences

*What is our wishlist?*




## Wishlist, point 3: Selective forget

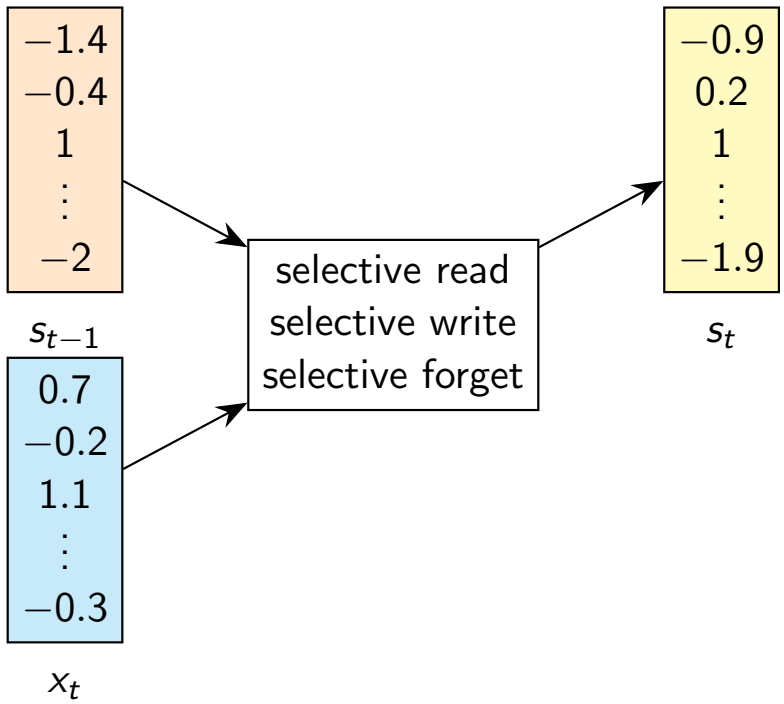
*h* has a fixed, finite size — it cannot keep growing to hold every word forever. The network should be able to actively erase parts of *h* that are no longer useful, freeing up capacity for what matters next.

## Putting it all together

Selective write, selective read and selective forget together ensure this finite-sized state vector *h* is used effectively.

# Long Short Term Memory Cells

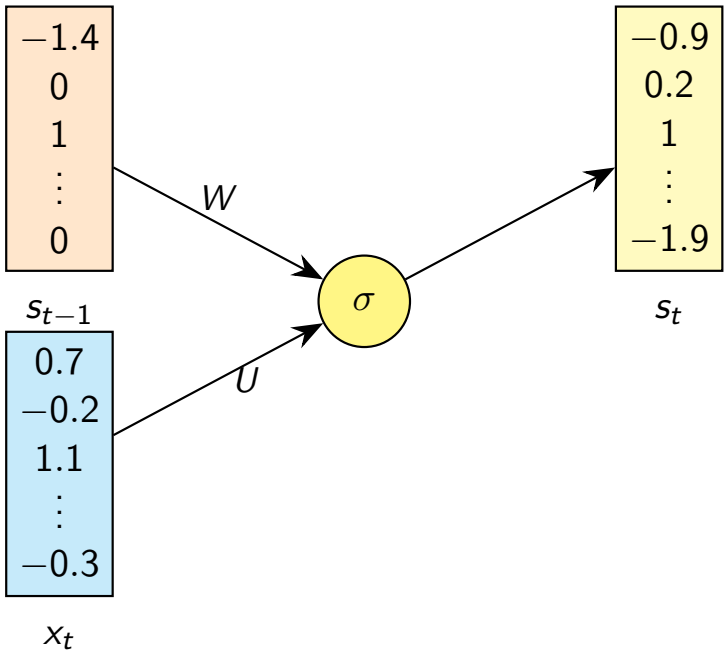
*How do you implement selective read, write and forget?*




While computing $s_t$ from $s_{t-1}$, we want to make sure that we use selective write, selective read and selective forget so that only important information is retained in $s_t$.

# Long Short Term Memory Cells

*Selective write*




## The naive way (plain RNN)

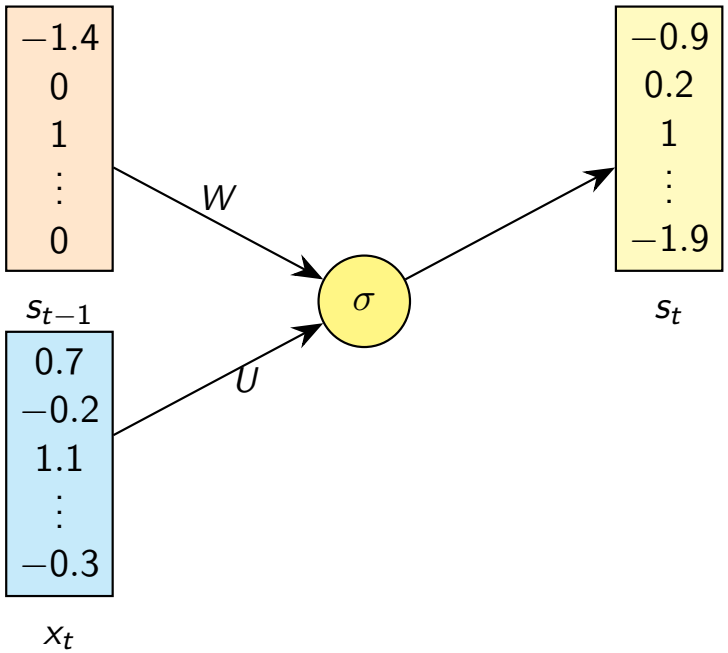
$$s_t = \sigma(Ux_t + W\,\mathbf{s_{t-1}} + b)$$

The **entire** $s_{t-1}$ gets passed forward and mixed in, every single time.

- ✓ Instead of passing $s_{t-1}$ as it is to $s_t$, we want to pass (write) only **some portions** of it to the next state
- ✓ A reasonable way of doing this: assign a value between 0 and 1 which determines what **fraction** of the current state to pass on

# Long Short Term Memory Cells

*Selective write*




## The naive way (plain RNN)

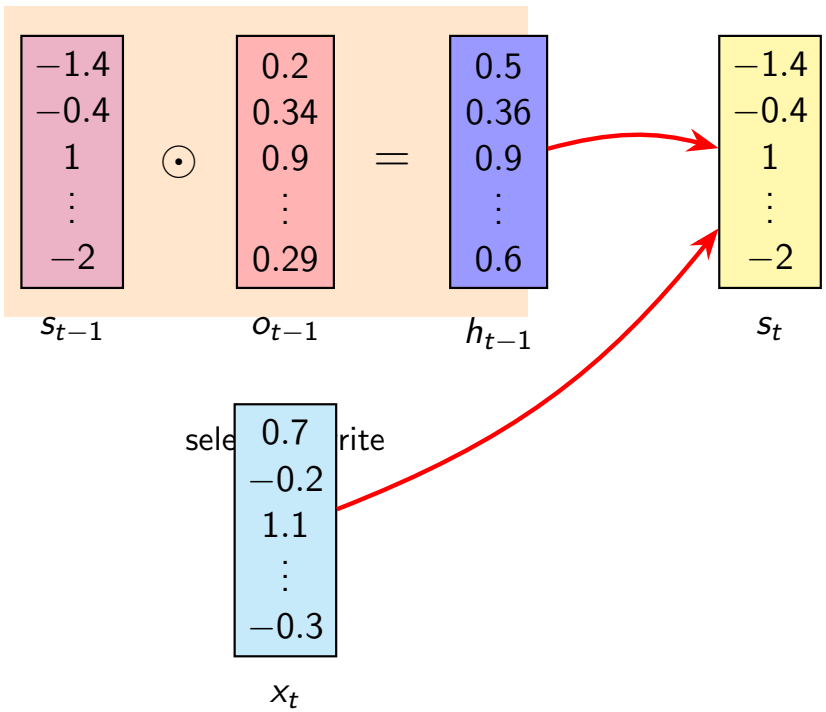
$$s_t = \sigma(Ux_t + W\,\mathbf{s_{t-1}} + b)$$

The **entire** $s_{t-1}$ gets passed forward and mixed in, every single time.

- ✓ Instead of passing $s_{t-1}$ as it is to $s_t$, we want to pass (write) only **some portions** of it to the next state
- ✓ A reasonable way of doing this: assign a value between 0 and 1 which determines what **fraction** of the current state to pass on

# Long Short Term Memory Cells

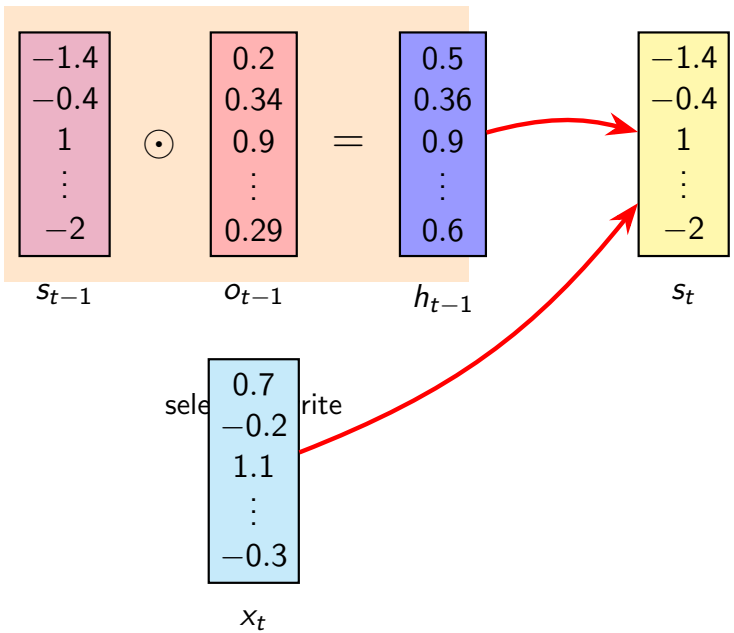
*Selective write*




But how do we compute $o_{t-1}$? How does the RNN know what fraction of the state to pass on?

# Long Short Term Memory Cells

*Selective write*

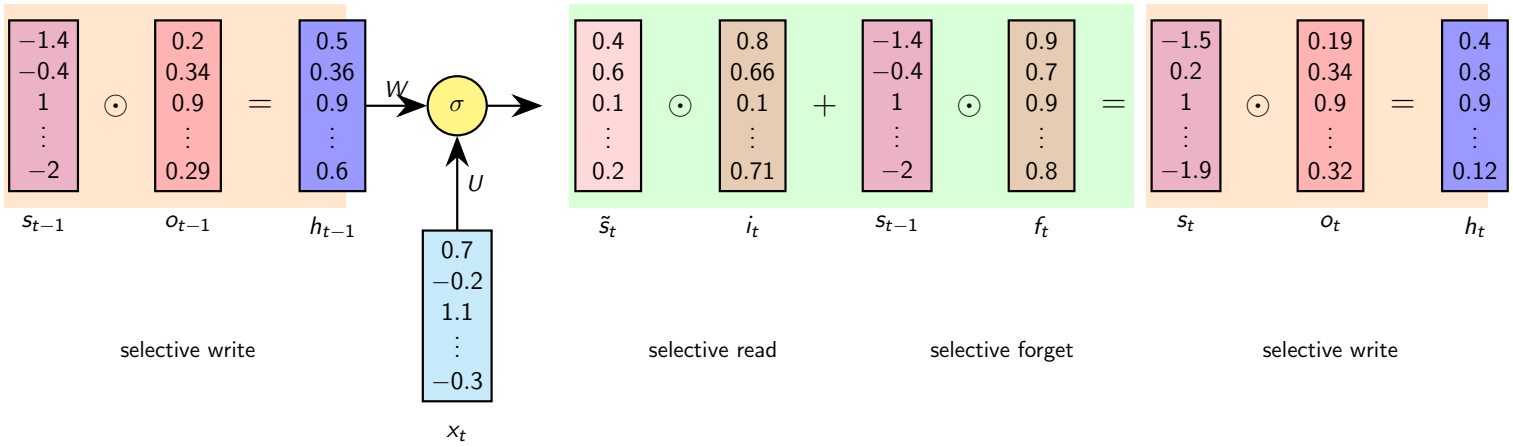



But how do we compute $o_{t-1}$? How does the RNN know what fraction of the state to pass on?

- ✓ learn $o_{t-1}$ from data
- ✓ the only thing that we learn from data is **parameters**
- ✓ **Solution:** express $o_{t-1}$ using parameters

# Long Short Term Memory Cells

*What does the full set of equations look like?*




## Gates

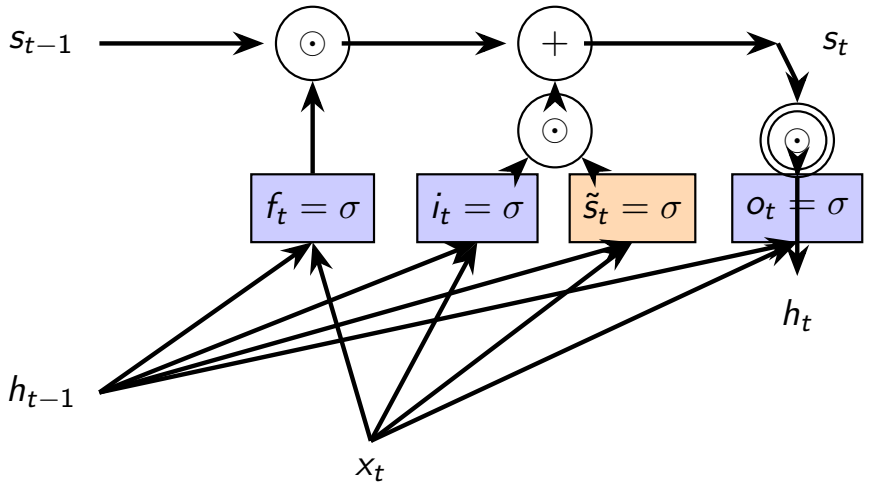
$$o_t = \sigma(W_o h_{t-1} + U_o x_t + b_o)$$
$$i_t = \sigma(W_i h_{t-1} + U_i x_t + b_i)$$
$$f_t = \sigma(W_f h_{t-1} + U_f x_t + b_f)$$

## States

$$\tilde{s}_t = \sigma(W h_{t-1} + U x_t + b)$$
$$s_t = f_t \odot s_{t-1} + i_t \odot \tilde{s}_t$$
$$h_t = o_t \odot \sigma(s_t)$$

# Long Short Term Memory Cells

*Putting it all together: one LSTM cell*




## Reading the flowchart

- Top highway carries the cell state $s_{t-1} \rightarrow s_t$ largely untouched, gated only by $\odot$ and $+$
- $f_t$ decides what fraction of $s_{t-1}$ survives; $i_t \odot \tilde{s}_t$ decides what new content gets added
- $o_t$ decides what fraction of (a squashed) $s_t$ becomes the visible output $h_t$

# Language Modeling

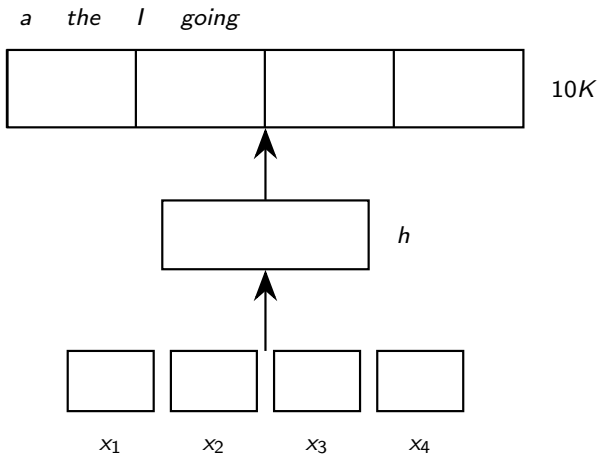
$$\underbrace{\text{I am going to}}_{t-1} \quad \underbrace{\text{the}}_{t}$$

- We will start by revisiting the problem of language modeling
- Informally, given '$t-1$' words we are interested in predicting the $t^{th}$ word

# Language Modeling

More formally, given $y_1, y_2, \ldots, y_{t-1}$ we want to find

$$y^* = \text{argmax } P(y_t \mid y_1, y_2, \ldots, y_{t-1})$$




## Output layer: a softmax over the whole vocabulary

$$\text{softmax}(Vh + c) \qquad \mathcal{O}(|V|h + c)$$

$$P(y_t = a), \quad P(y_t = \text{the}), \quad \ldots$$

The network produces one probability per vocabulary word (here, $10K$ of them); $y^*$ is simply the word with the highest probability.

# Language Modeling: Why $y$, not $x$?

## The question

The words $y_1, y_2, \ldots, y_{t-1}$ are fed **into** the network as inputs — so why call them $y$ instead of $x$?

## Every word plays two roles

- At timestep $t-1$, that word was itself the **prediction target**: $y_{t-1}$
- At timestep $t$, the very same word is now **fed in** to help predict the next word: it acts as an input

## Why the notation stays $y$

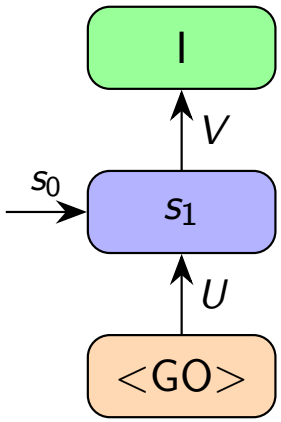
- Language modeling is fundamentally about modeling the sequence of **labels** $y_1, y_2, \ldots, y_T$ (the joint probability of the whole sentence)
- There is no separate, independent "$x$" in this task — unlike ordinary classification, the input at each step **is** a previously predicted (or ground-truth) label

# Language Modeling: Shorthand Notation

We will refer to $P(y_t \mid y_1, y_2, \ldots, y_{t-1})$ by shorthand notation:

$$P(y_t \mid y_1^{t-1})$$

# Generating the First Word




## Kick-starting generation

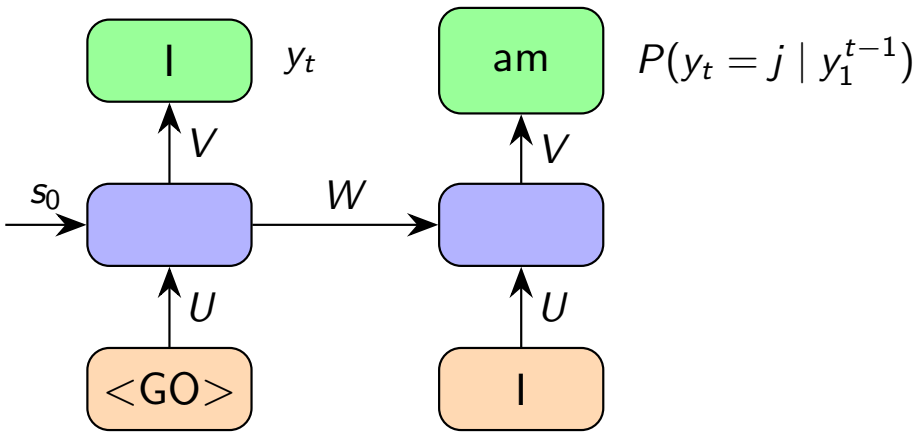
Generation begins from a special start token <**GO**> (playing the role of $x_0$) together with an initial state $s_0$:

$$s_1 = \sigma(Ws_0 + Ux_0 + b)$$

$$y = \text{softmax}(Vs_1 + c)$$

The predicted distribution over the vocabulary gives us the first generated word, here "I".

# Generating the Second Word



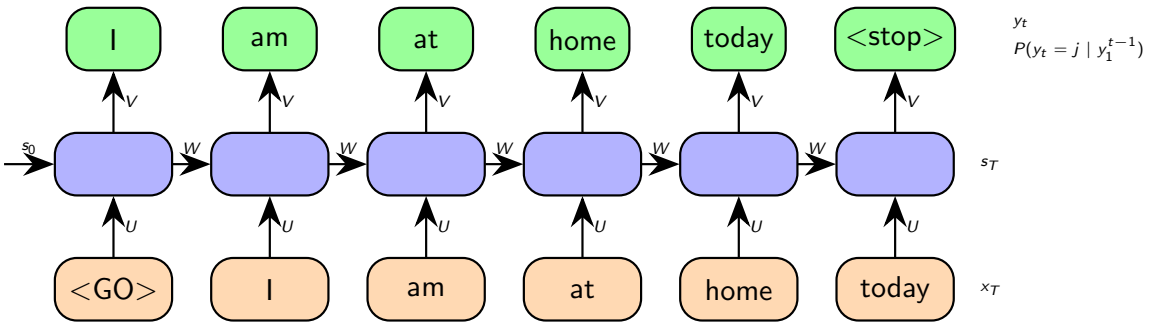

**The key idea: generation is autoregressive**

The word predicted at one step, "I", is fed straight back in as the **input** for the next step, producing $s_2$ and then the next word.

$$y_2 = \text{softmax}(Vs_2 + c)$$

Same output layer as before, applied to the new hidden state $s_2$; here it predicts "am".

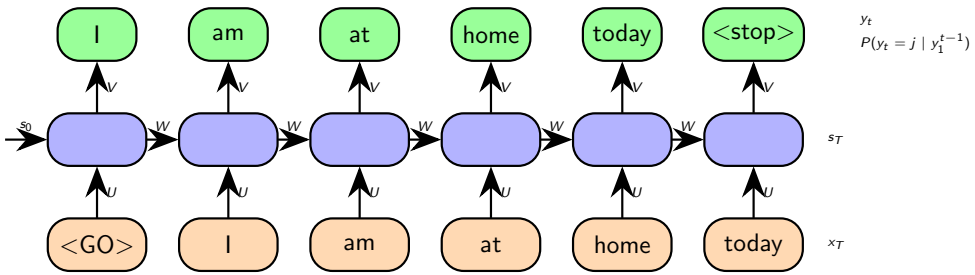
# Generating a Full Sentence, One Word at a Time




## The story so far, continued

- Start with <**GO**>, generate one word, feed it back as the next input, repeat
- Each new word only depends on the words generated before it, exactly as $P(y_t \mid y_1^{t-1})$ says
- Generation stops the moment the model predicts a special <**stop**> token

# Language Modeling with an RNN




## We are interested in

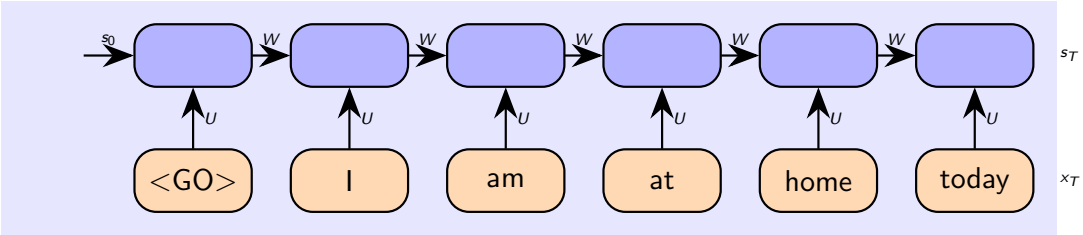
$$P(y_t = j \mid y_1, y_2 \ldots y_{t-1})$$

where $j \in V$ and $V$ is the set of all vocabulary words.

## Using an RNN we compute this as

$$P(y_t = j \mid y_1^{t-1}) = \text{softmax}(Vs_t + c)_j$$

# The Encoder

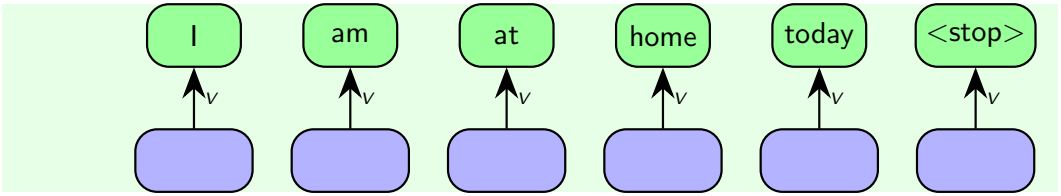



**Encoder**

## What the encoder does

- Reads the input tokens $x_0, x_1, \ldots, x_T$ one at a time
- Combines each new input with the running hidden state via $U$ and $W$: $s_t = \sigma(Ws_{t-1} + Ux_t + b)$
- Produces a hidden state $s_t$ that summarizes everything seen **so far**

# The Decoder




**Decoder**

## What the decoder does

- Takes each hidden state $s_t$ produced by the encoder
- Turns it into a probability distribution over the vocabulary: $P(y_t = j \mid y_1^{t-1}) = \text{softmax}(Vs_t + c)_j$
- Reads off the predicted word $y_t$, e.g. "I", "am", ..., until <**stop**> is produced

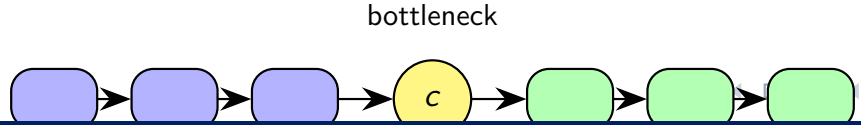
# Encoder-Decoder: Where's the Latent?

## This example (language model)

- One shared RNN plays **both** roles, at **every** timestep
- $s_t$ is a running, evolving state — updated (encode) and immediately read out via $V$ (decode), over and over
- There is no single moment where "encoding stops and decoding begins"
- No fixed-size bottleneck vector, unlike the autoencoder / VAE

## True encoder-decoder (seq2seq)

- **Two separate** RNNs: one encoder, one decoder
- Encoder reads the **entire** input sequence, then stops
- Only at the end does it hand off **one** final context vector $c$ (typically its last hidden state)
- Decoder generates the **whole** output sequence starting from $c$ alone



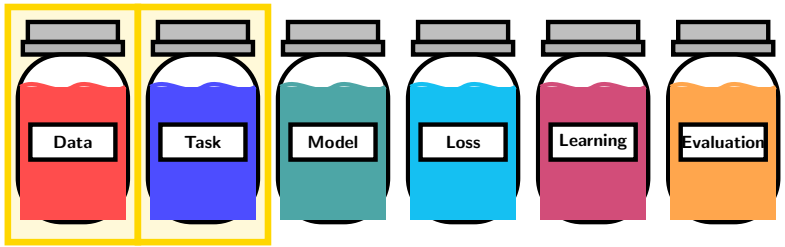
# Txt2Txt

## Task: Machine translation

i/p: I am going home

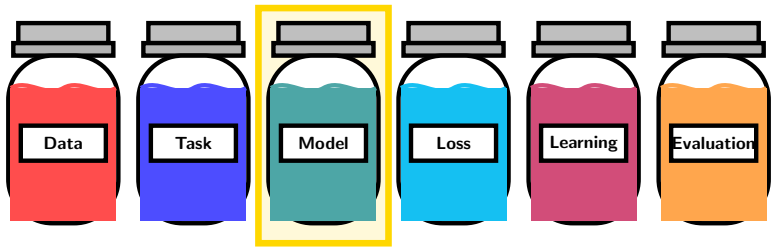
o/p: Mein ghar ja raha hoon

# Txt2Txt




- **Task:** Machine translation
- **Data:** $\{x_i = source_i,\ y_i = target_i\}_{i=1}^{N}$

# Txt2Txt

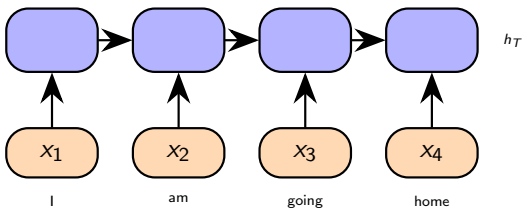



## Model (Option 1):

**Encoder:**

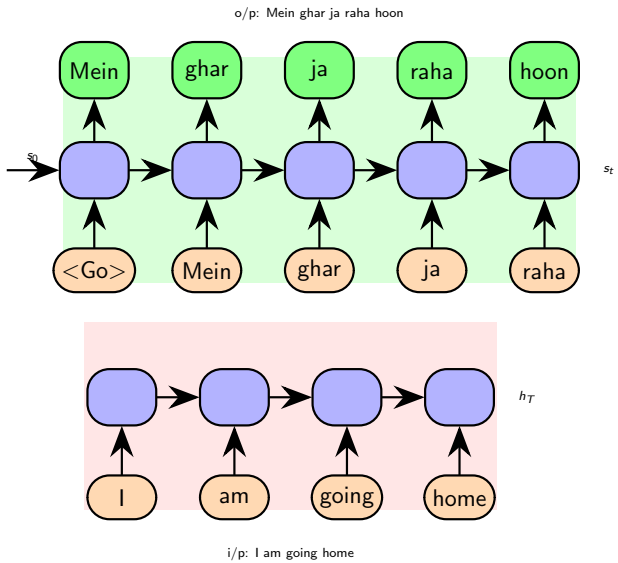
$$h_t = RNN(h_{t-1}, x_{it})$$

$$s_1 = \sigma(Ws_0 + Ux_1 + b)$$




i/p: I am going home

# Txt2Txt




- **Task:** Machine translation
- **Data:** $\{x_i = source_i, y_i = target_i\}_{i=1}^{N}$
- **Model (Option 1):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:**

$$s_0 = h_T \quad (T \text{ is length of input})$$

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

$$P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$$

# Encoder: Worked Example (continued)

**Same $W, U, b$ as before.** $x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ("am")

**Compute** $s_2 = \sigma(Ws_1 + Ux_2 + b)$, using $s_1 = \begin{bmatrix} 0.6225 \\ 0.4013 \end{bmatrix}$

1. $Ws_1$
2. $Ux_2$
3. $Ws_1 + Ux_2$
4. $+ b$
5. $\sigma(\cdot)$ elementwise

$$Ws_1 = \begin{bmatrix} 0.2310 \\ 0.1826 \end{bmatrix}$$

$$Ux_2 = \begin{bmatrix} 0.6 \\ 0.2 \end{bmatrix}$$

$$\text{sum} = \begin{bmatrix} 0.8310 \\ 0.3826 \end{bmatrix}$$

$$+ b = \begin{bmatrix} 0.9310 \\ 0.2826 \end{bmatrix}$$

$$s_2 \approx \begin{bmatrix} 0.717 \\ 0.570 \end{bmatrix}$$

# Encoder: Worked Example (continued)

**Same $W, U, b$ as before.** $x_3 = \begin{bmatrix}1\\1\end{bmatrix}$ ("going")

**Compute** $s_3 = \sigma(Ws_2 + Ux_3 + b)$, using $s_2 = \begin{bmatrix}0.717\\0.570\end{bmatrix}$

1. $Ws_2$
2. $Ux_3$
3. $Ws_2 + Ux_3$
4. $+ b$
5. $\sigma(\cdot)$ elementwise

$$Ws_2 = \begin{bmatrix}0.2445\\0.2427\end{bmatrix}$$

$$Ux_3 = \begin{bmatrix}1.0\\-0.1\end{bmatrix}$$

$$\text{sum} = \begin{bmatrix}1.2445\\0.1427\end{bmatrix}$$

$$+ b = \begin{bmatrix}1.3445\\0.0427\end{bmatrix}$$

$$s_3 \approx \begin{bmatrix}0.793\\0.511\end{bmatrix}$$

# Encoder: Worked Example (continued)

**Same $W, U, b$ as before.** $x_4 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$ ("home")

**Compute** $s_4 = \sigma(Ws_3 + Ux_4 + b)$, using $s_3 = \begin{bmatrix} 0.793 \\ 0.511 \end{bmatrix}$

1. $Ws_3$
2. $Ux_4$
3. $Ws_3 + Ux_4$
4. $+ b$
5. $\sigma(\cdot)$ elementwise

$$Ws_3 = \begin{bmatrix} 0.2943 \\ 0.2326 \end{bmatrix}$$

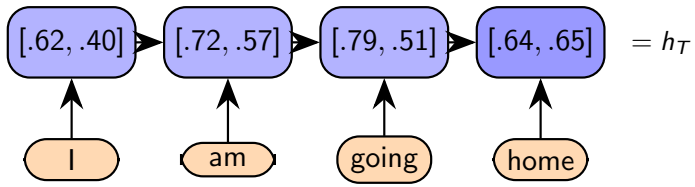
$$Ux_4 = \begin{bmatrix} 0.2 \\ 0.5 \end{bmatrix}$$

$$\text{sum} = \begin{bmatrix} 0.4943 \\ 0.7326 \end{bmatrix}$$

$$+ b = \begin{bmatrix} 0.5943 \\ 0.6326 \end{bmatrix}$$

$$s_4 \approx \begin{bmatrix} 0.644 \\ 0.653 \end{bmatrix}$$

# Encoder: Worked Example — Summary




i/p: I am going home $\longrightarrow$ $h_T \approx [0.644,\ 0.653]$

# Txt2Txt: Input and Output Lengths Can Differ

$$x_1, x_2, \ldots, x_T \longrightarrow y_1, y_2, \ldots, y_L$$

## Two separate lengths, on purpose

- $T$ = length of the **input** sequence (source sentence)
- $L$ = length of the **output** sequence (target sentence)
- Unlike sequence labeling (one tag per word, so output length = input length), here $T$ and $L$ need **not** be equal

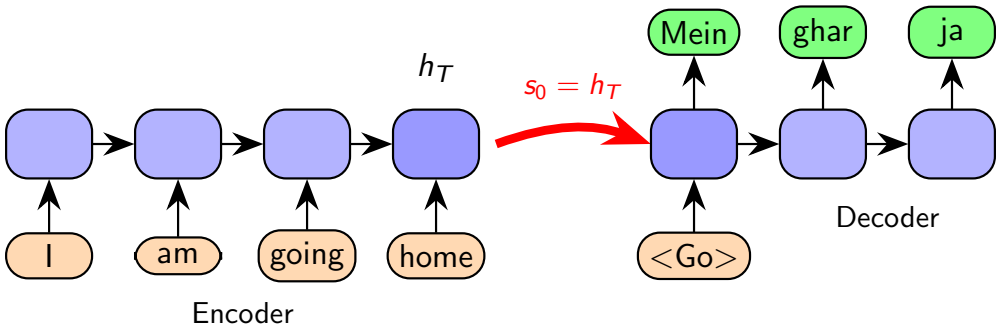
## Our example

i/p: I am going home $(T = 4)$
o/p: Mein ghar ja raha hoon $(L = 5)$

## The key idea

$$y_t = f(x_1, x_2, \ldots, x_T)$$

# Txt2Txt: The Bridge Between Encoder and Decoder




## Why this matters

- The encoder's very **last** hidden state, $h_T$, is the **only** thing handed over to the decoder
- The decoder's starting state is literally set to it: $s_0 = h_T$
- This single vector is the **bottleneck** — everything the decoder ever knows about the input sentence has to be packed into it

# Txt2Txt: Tracing the First Decoding Step

## Start from the general formula

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

## Plug in $t = 1$

- $s_0$ is already known: it's $h_T$ from the encoder
- There is no "previous predicted word" yet, so we feed in the start token instead: $\hat{y}_0 = \langle Go \rangle$

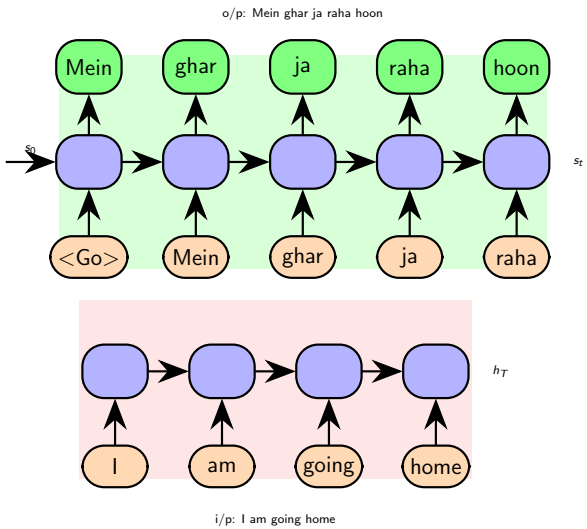
$$s_1 = RNN(s_0, e(\langle Go \rangle)) = RNN(h_T, e(\langle Go \rangle))$$

## Then decode it

$$P(y_1 \mid x) = \text{softmax}(Vs_1 + b)$$

Reading off the highest-probability word gives us the first output: **"Mein"**.

# Txt2Txt



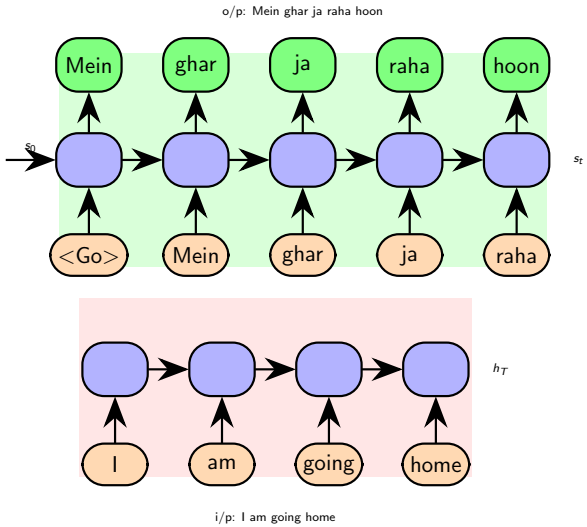

- **Task:** Machine translation
- **Data:** $\{x_i = source_i, y_i = target_i\}_{i=1}^{N}$
- **Model (Option 1):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:**

    $s_0 = h_T$ ($T$ is length of input)

    $s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$

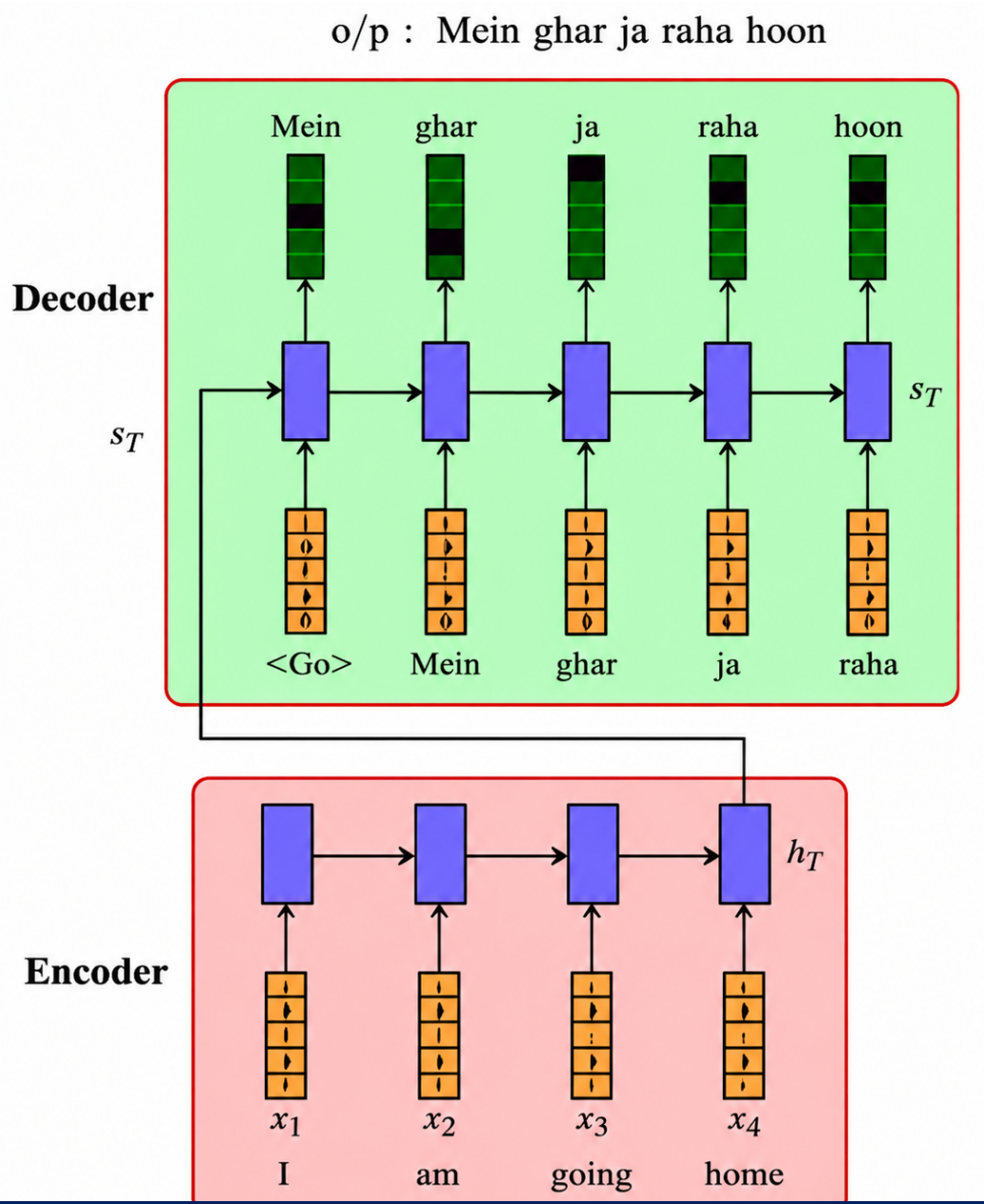
# Txt2Txt




- **Task:** Machine translation
- **Model (Option 1):**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$, $s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$, $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$
- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^t$$

# Txt2Txt


o/p : Mein ghar ja raha hoon
Decoder
Mein
ghar
ja
raha
hoon
$s_T$
$s_T$
<Go>
Mein
ghar
ja
raha
Encoder
$h_T$
$x_1$
$x_2$
$x_3$
$x_4$
I
am
going
home

# Txt2Txt

- **Task:** Machine translation
- **Model (Option 1):**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$,

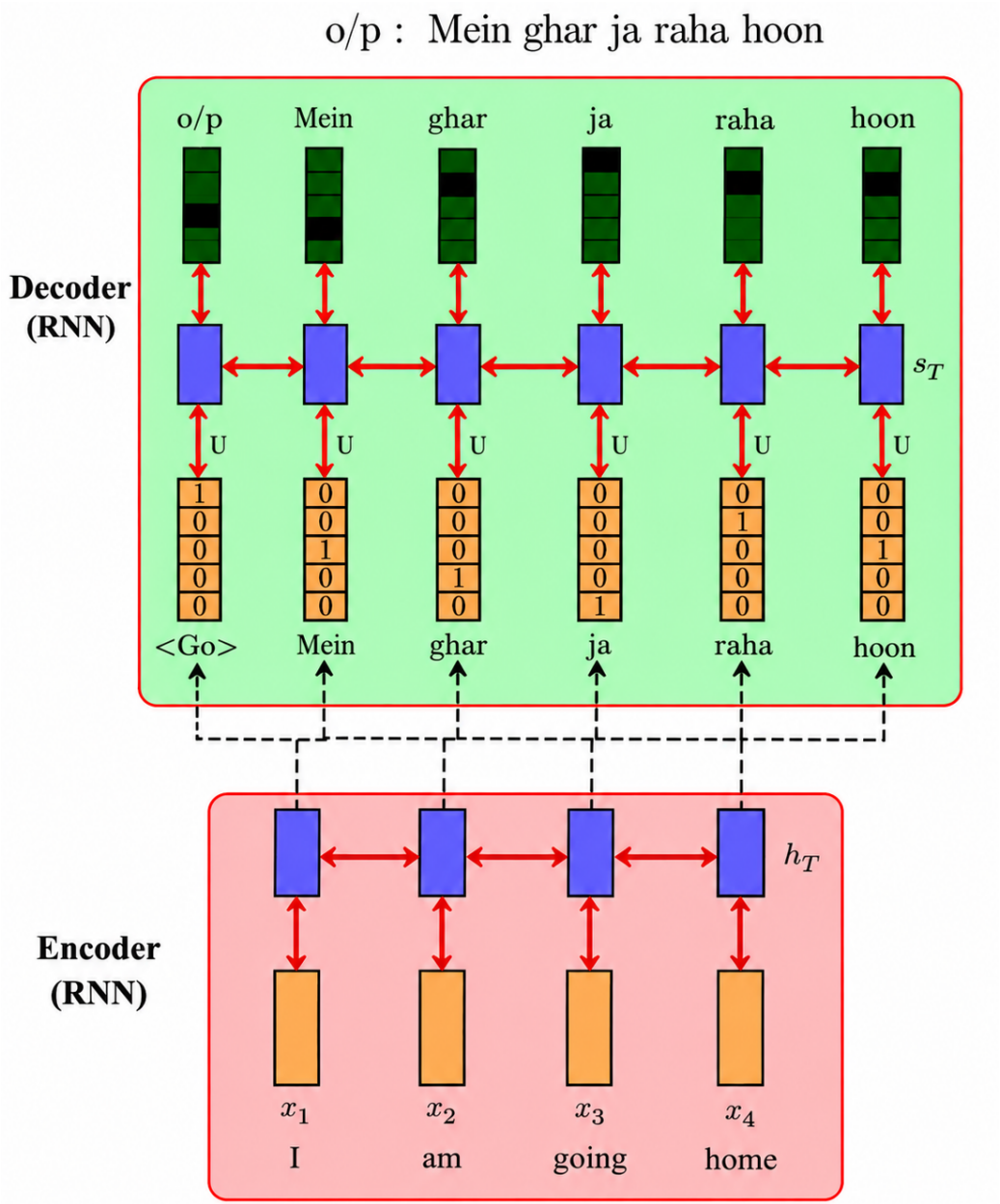
$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1})),$$

$$P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$

# Txt2Txt


o/p : Mein ghar ja raha hoon
Decoder (RNN)
o/p
Mein
ghar
ja
raha
hoon
$s_T$
U
<Go>
Mein
ghar
ja
raha
hoon
Encoder (RNN)
$h_T$
$x_1$
$x_2$
$x_3$
$x_4$
I
am
going
home

# Decoder: Worked Example

**Carrying over from the encoder:** $s_0 = h_T \approx \begin{bmatrix} 0.644 \\ 0.653 \end{bmatrix}$

**New toy values (decoder):**

$$e(\langle Go \rangle) = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad W_{dec} = \begin{bmatrix} 0.3 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}, \quad U_{dec} = \begin{bmatrix} 0.5 & -0.3 \\ 0.2 & 0.6 \end{bmatrix}, \quad b_{dec} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

**Compute** $s_1 = \sigma(W_{dec} s_0 + U_{dec}\, e(\langle Go \rangle) + b_{dec})$

1. $W_{dec}\, s_0$
2. $U_{dec}\, e(\langle Go \rangle)$
3. sum of the two
4. $+\, b_{dec}$
5. $\sigma(\cdot)$ elementwise

$$W_{dec} s_0 = \begin{bmatrix} 0.2585 \\ 0.1324 \end{bmatrix}$$

$$U_{dec}\, e(\langle Go \rangle) = \begin{bmatrix} 0.8000 \\ -0.4000 \end{bmatrix}$$

$$\text{sum} = \begin{bmatrix} 1.0585 \\ -0.2676 \end{bmatrix}$$

$$+\, b_{dec} = \begin{bmatrix} 1.1085 \\ -0.3176 \end{bmatrix}$$

$$s_1 \approx \begin{bmatrix} 0.752 \\ 0.421 \end{bmatrix}$$

# Decoder: Worked Example — First Predicted Word

**Toy output vocabulary** (3 words): Mein, ghar, $\langle stop \rangle$

$$V = \begin{bmatrix} 0.6 & -0.1 \\ 0.2 & 0.5 \\ -0.3 & 0.4 \end{bmatrix}, \quad c = \begin{bmatrix} 0.1 \\ -0.1 \\ 0.05 \end{bmatrix}$$

**Compute** $P(y_1 \mid x) = \text{softmax}(Vs_1 + c)$, using $s_1 \approx \begin{bmatrix} 0.752 \\ 0.421 \end{bmatrix}$

1. Logits: $Vs_1 + c$
2. Exponentiate each logit
3. Normalize (divide by the sum)

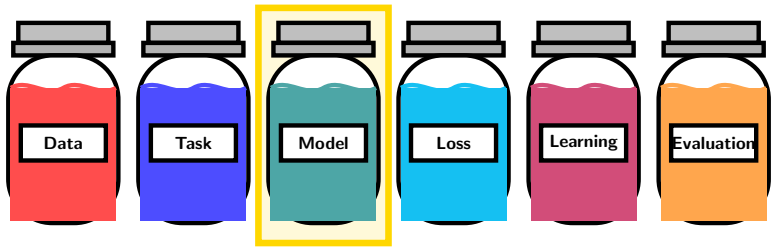
$$\text{logits} = \begin{bmatrix} 0.509 \\ 0.261 \\ -0.007 \end{bmatrix}$$

$$e^{(\cdot)} = \begin{bmatrix} 1.664 \\ 1.298 \\ 0.993 \end{bmatrix}, \text{ sum} = 3.955$$

$P(y_1 = \text{Mein}) \approx 0.421 \qquad P(y_1 = \text{ghar}) \approx 0.328 \qquad P(y_1 = \langle stop \rangle) \approx 0.251$

Highest probability $\Rightarrow$ predicted first word: **"Mein"**

# Txt2Txt




- **Task:** Machine translation
- **Data:** $\{x_i = source_i, y_i = target_i\}_{i=1}^{N}$
- **Model (Option 2):**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

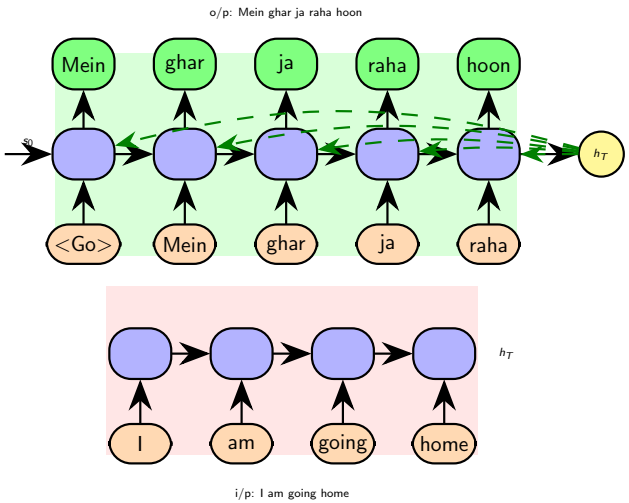
$$s_t = RNN(s_{t-1}, [h_T, e(\hat{y}_{t-1})])$$

$$P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$

# Txt2Txt

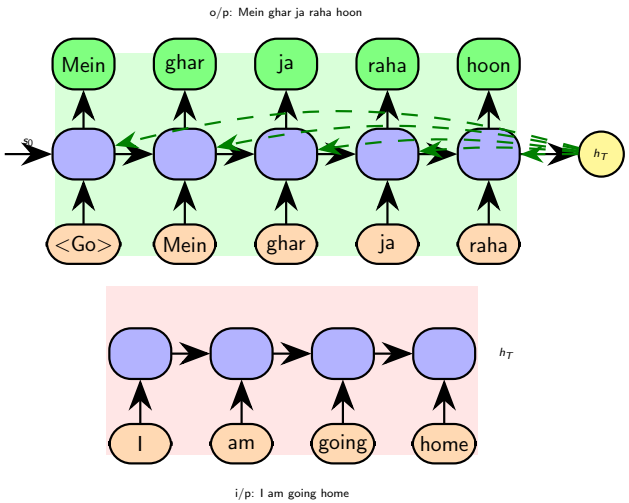



- **Task:** Machine translation
- **Model (Option 2):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

    $s_t = RNN(s_{t-1}, [h_T, e(\hat{y}_{t-1})])$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**
  $$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$
- **Algorithm:** Gradient descent with backpropagation

# Txt2Txt

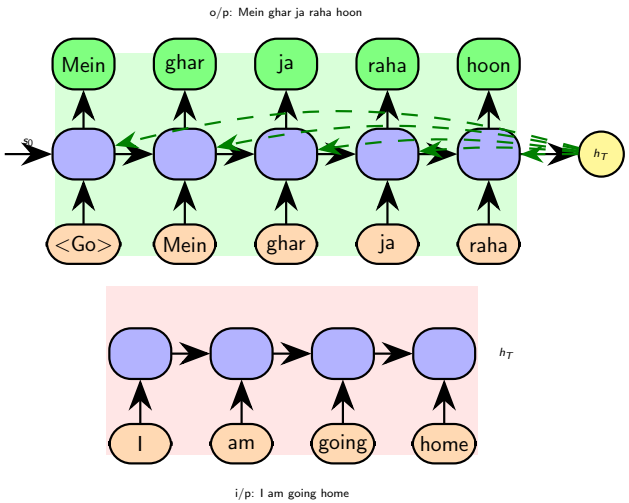



- **Task:** Machine translation
- **Model (Option 2):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

    $s_t = RNN(s_{t-1}, [h_T, e(\hat{y}_{t-1})])$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**
  $$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$
- **Algorithm:** Gradient descent with backpropagation

# Txt2Txt

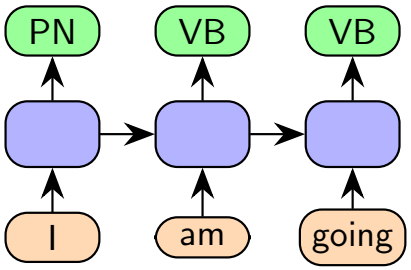



- **Task:** Machine translation
- **Model (Option 2):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

    $s_t = RNN(s_{t-1}, [h_T, e(\hat{y}_{t-1})])$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**
  $$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$
- **Algorithm:** Gradient descent with backpropagation

# Txt2Txt: Why Not Just Use Sequence Labeling?




i/p: I am going home

o/p: Mein ghar ja raha hoon

## Sequence labeling: perfectly aligned

- One output for every input, at the **same** position
- Output length always equals input length
- Word $t$'s tag depends mainly on word $t$ itself

## Machine translation: no such alignment

- Output length (5) $\neq$ input length (4)
- There's no single input word that "ja" lines up with position-for-position — it depends on the sentence as a whole
- Word order itself can change between languages

# Decoder (Option 2): Worked Example

**Carrying over from the encoder:** $s_0 = h_T \approx \begin{bmatrix} 0.644 \\ 0.653 \end{bmatrix}$

**Toy values:** $e(\langle Go \rangle) = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$,

$$W_{dec} = \begin{bmatrix} 0.3 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}, \quad U_{dec} = \begin{bmatrix} 0.5 & -0.3 & 0.2 & -0.1 \\ 0.2 & 0.6 & -0.2 & 0.3 \end{bmatrix}, \quad b_{dec} = \begin{bmatrix} 0.05 \\ -0.05 \end{bmatrix}$$

(note: $U_{dec}$ is now $2 \times 4$, since it multiplies the **concatenated** vector $[e(\hat{y}_{t-1}),\ h_T]$, length $2 + 2 = 4$)

**Compute** $s_1 = \sigma\big(W_{dec} s_0 + U_{dec}[e(\langle Go \rangle), h_T] + b_{dec}\big)$

1. Concatenate: $[e(\langle Go \rangle), h_T] = [1, -1, 0.644, 0.653]$
2. $W_{dec}\, s_0$
3. $U_{dec} \times$ (concatenated vector)
4. sum $+ b_{dec}$
5. $\sigma(\cdot)$ elementwise

$$W_{dec} s_0 = \begin{bmatrix} 0.2585 \\ 0.1324 \end{bmatrix}$$

$$U_{dec}[\cdot] = \begin{bmatrix} 0.8635 \\ -0.3329 \end{bmatrix}$$

$$\text{sum} + b_{dec} = \begin{bmatrix} 1.1720 \\ -0.2505 \end{bmatrix}$$

$$s_1 \approx \begin{bmatrix} 0.764 \\ 0.438 \end{bmatrix}$$

# Decoder (Option 2): Worked Example (continued)

Suppose $\hat{y}_1 =$ "Mein", with toy embedding $e(\text{Mein}) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

**Compute** $s_2 = \sigma\big(W_{dec}\, s_1 + U_{dec}[e(\text{Mein}),\, h_T] + b_{dec}\big)$, using $s_1 \approx \begin{bmatrix} 0.764 \\ 0.438 \end{bmatrix}$

1. Concatenate: $[e(\text{Mein}), h_T] = [0, 1, 0.644, 0.653]$
2. $W_{dec}\, s_1$
3. $U_{dec} \times$ (concatenated vector)
4. sum $+ b_{dec}$
5. $\sigma(\cdot)$ elementwise

$$W_{dec}\, s_1 = \begin{bmatrix} 0.2730 \\ 0.0224 \end{bmatrix}$$

$$U_{dec}[\cdot] = \begin{bmatrix} -0.2365 \\ 0.6671 \end{bmatrix}$$

$$\text{sum} + b_{dec} = \begin{bmatrix} 0.0865 \\ 0.6395 \end{bmatrix}$$

$$s_2 \approx \begin{bmatrix} 0.522 \\ 0.655 \end{bmatrix}$$

Notice: the **same** $h_T \approx [0.644, 0.653]$ was fed in **fresh** here again — unlike Option 1, where it only ever appeared once, in $s_0$.

# Txt2Txt: A Sequence-to-Sequence Generation Problem

## What makes a problem "sequence-to-sequence"?

- Input is a sequence: $x_1, x_2, \ldots, x_T$; output is **also** a sequence: $y_1, y_2, \ldots, y_L$
- Crucially, $L$ need **not** equal $T$, and there is no fixed rule tying output position $t$ to input position $t$

## Why machine translation fits this exactly

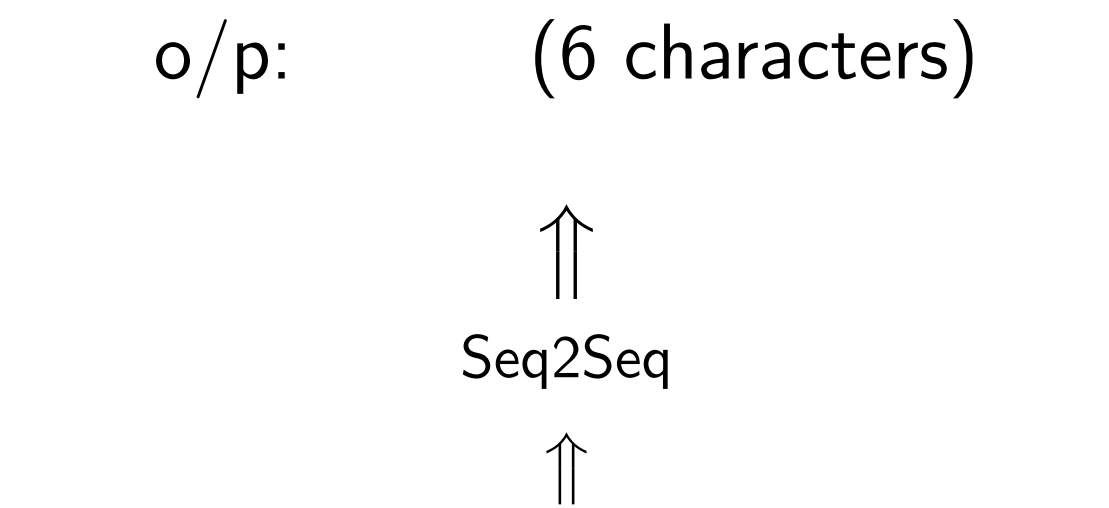
- i/p: I am going home ($T = 4$) $\rightarrow$ o/p: Mein ghar ja raha hoon ($L = 5$)
- Output is longer than input — no way to produce it one-tag-per-input-word
- Word order differs across languages, so output word $t$ can depend on **any** input word, not just input word $t$

## The consequence

We need a model that first **reads the entire input** (encoder), and only then **generates the entire output** (decoder) — one sequence in, one independently-shaped sequence out.

# Transliteration

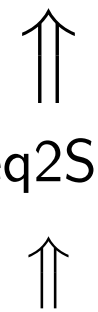
**Task:** Transliteration



# Transliteration

**Task:** Transliteration

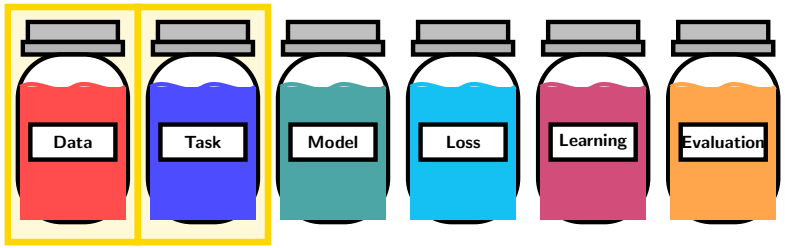
o/p: indiyaa (6 characters)



Seq2Seq

i/p: I N D I A (5 characters)

# Transliteration

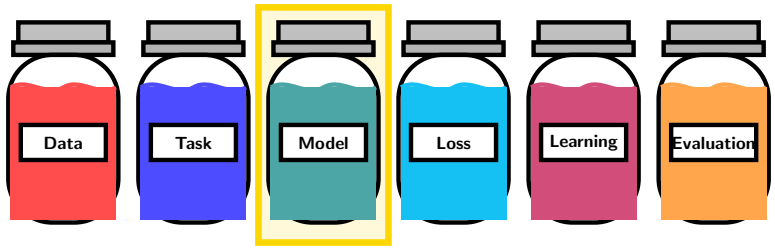



- **Task:** Transliteration
- **Data:** $\{x_i = srcword_i,\ y_i = tgtword_i\}_{i=1}^{N}$

**Source:** wikidata.org

| **E** | **H** |
|---|---|
| Mumbai | mumbal |
| ⋮ | ⋮ |
| ⋮ | ⋮ |
| ⋮ | ⋮ |

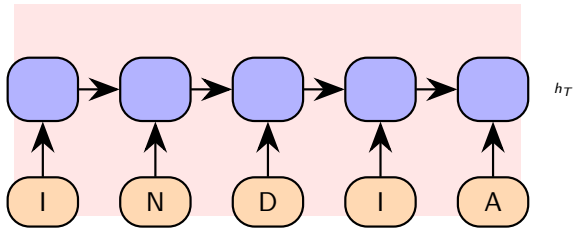
# Transliteration




- **Task:** Transliteration
- **Data:**
  $\{x_i = srcword_i, y_i = tgtword_i\}_{i=1}^{N}$
- **Model (Option 1):**
  - **Encoder:**

$$h_t = RNN(h_{t-1}, x_{it})$$

**implemented using LSTM**




i/p: I N D I A

# Transliteration Decoder: Getting Started

## The decoder's job here is different from Txt2Txt

- In Txt2Txt, the decoder generated one **word** at a time
- Here, the decoder generates one **character** at a time — the output "indiyaa" is built letter by letter

## Before generating anything, two things are borrowed / invented

- **Starting state:** the decoder doesn't start from scratch — it starts exactly where the encoder left off

$$s_0 = h_T$$

- **Starting input:** since no character has been predicted yet, we feed in a special placeholder symbol instead

$$\hat{y}_0 = \langle Go \rangle$$

# Transliteration Decoder: One Step at a Time

## The general rule

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

## In plain words

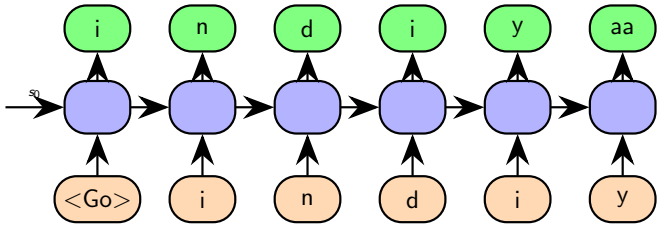
- Take the **previous hidden state** ($s_{t-1}$) — everything decided so far
- Take the **previous character we just wrote** ($\hat{y}_{t-1}$), turned into a vector via $e(\cdot)$
- Combine the two to get a new hidden state, $s_t$

## Then turn that hidden state into an actual character

$$P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$$

This gives a probability for **every** possible output character; we simply pick the most likely one.

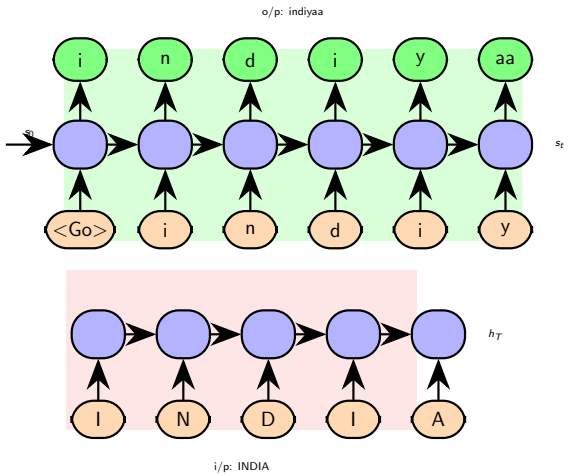
# Transliteration Decoder: Character by Character




## Reading the diagram

- Start: $\langle Go \rangle$ in, first character "i" out
- Each predicted character feeds right back in as the next input — same autoregressive pattern as before
- Repeat until all 6 characters of "indiyaa" have been generated

# Transliteration




- **Task:** Transliteration
- **Data:** $\{x_i = srcword_i, y_i = tgtword_i\}_{i=1}^{N}$
- **Model (Option 1):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:**

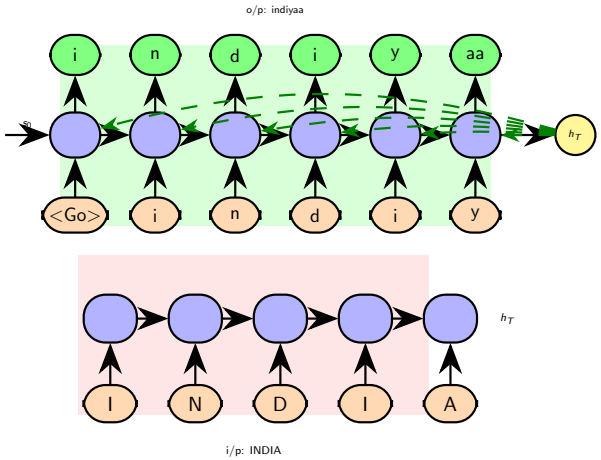
    $s_0 = h_T$ ($T$ is length of input)

    $s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^t$$

# Transliteration

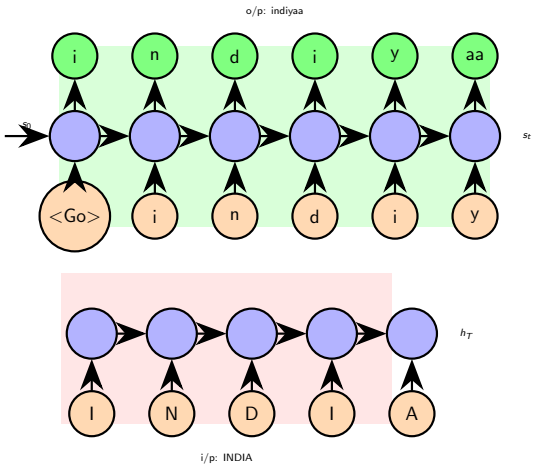



- **Task:** Transliteration
- **Model (Option 2):**
  - **Encoder:**
    $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

    $s_t = RNN(s_{t-1}, [e(\hat{y}_{t-1}), h_T])$

    $P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t +$
- **Parameters:**
  $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**
  $$\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$$
- **Algorithm:** Gradient descent with backpropagation

# Transliteration




- **Task:** Transliteration
- **Data:** $\{x_i = srcword_i, y_i = tgtword_i\}_{i=1}^{N}$
- **Model (Option 1):**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:**

$$s_0 = h_T \ (T \text{ is length of input})$$

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

$$P(y_t \mid y_1^{t-1}, x)$$

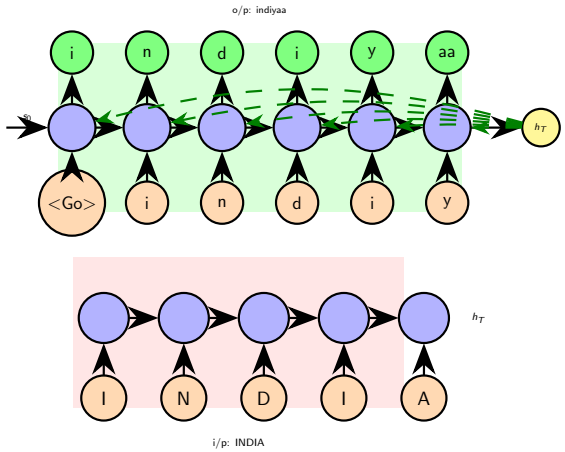
$$= \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta)$$

$$\sum_{t=1}^{T} -\log P(\quad \mid \quad^{t-1}, \quad)$$

# Transliteration




- **Task:** Transliteration
- **Model (Option 2):**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

$$s_t = RNN(s_{t-1}, [e(\hat{y}_{t-1}), h_T])$$

$$P(y_t \mid y_1^{t-1}, x)$$

$$= \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:** $\mathcal{L}(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$
- **Algorithm:** Gradient descent with backpropagation

# Transliteration: Option 1 vs. Option 2

## Option 1: $h_T$ used once

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

- $h_T$ sets $s_0$ and is never seen again
- After that, it must survive purely by being passed along through $s_1 \rightarrow s_2 \rightarrow \ldots$
- By the time we generate the 5th or 6th character, $h_T$'s influence has passed through several RNN hops and can fade

## Option 2: $h_T$ used every step

$$s_t = RNN(s_{t-1}, [e(\hat{y}_{t-1}), h_T])$$

- $h_T$ is fed in **fresh**, concatenated with the previous character, at **every** timestep
- The decoder never has to rely on $h_T$ surviving a long chain of hops
- Even the last character generated sees the **original** $h_T$ directly

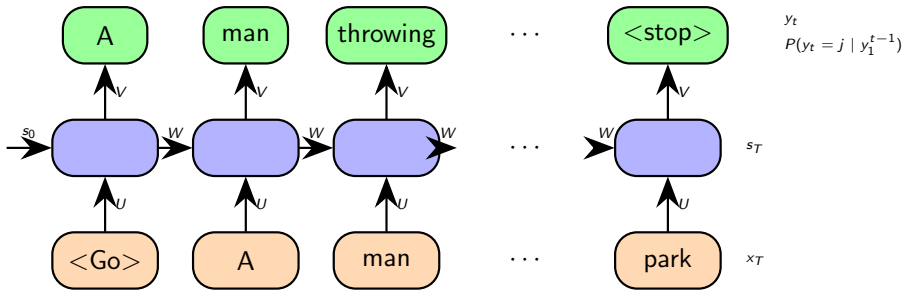
# Encoder Decoder Model for Image Captioning

- So far we have seen how to model the conditional probability distribution

$$P(y_t \mid y_1^{t-1})$$

- More informally, we have seen how to generate a sentence given previous words

# Encoder Decoder Model for Image Captioning



A man throwing
a frisbee in a park

# Encoder Decoder Model for Image Captioning

## Before: conditioned only on previous words

$$P(y_t \mid y_1^{t-1})$$

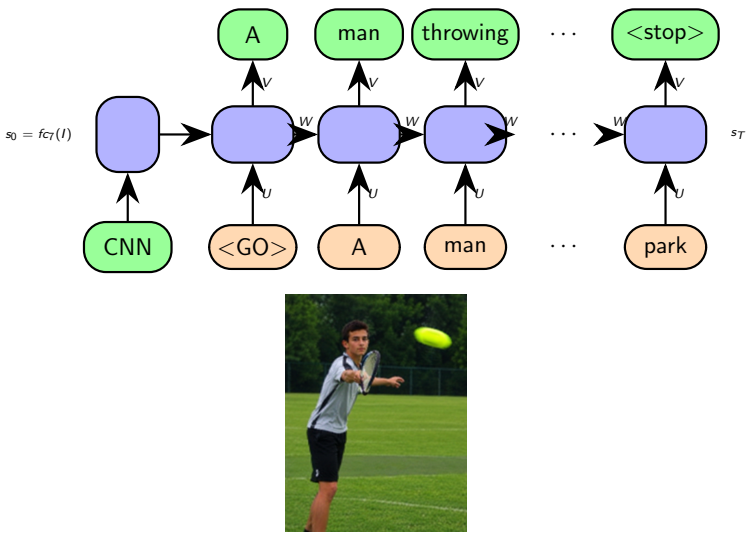
## Now: also conditioned on the image

$$P(y_t \mid y_1^{t-1}, I)$$

## An important difference from Txt2Txt

- In Txt2Txt, the context vector $h_T$ only became available **after** the encoder had read the entire input sentence
- Here, the image $I$ is available **immediately** — there's no word-by-word reading step for an image
- So $I$ is present from the very first timestep onward, influencing even the **first** generated word

# Encoder Decoder Model for Image Captioning




## Earlier

$$P(y_t \mid y_1^{t-1}) = P(y_t = j \mid s_t)$$

where $s_t$ was a state capturing all the previous words.

## Now

$$P(y_t = j \mid y_1^{t-1}, I) = P(y_t = j \mid s_t, fc_7(I))$$

where $fc_7(I)$ is the representation obtained from the $fc_7$ layer of a CNN.

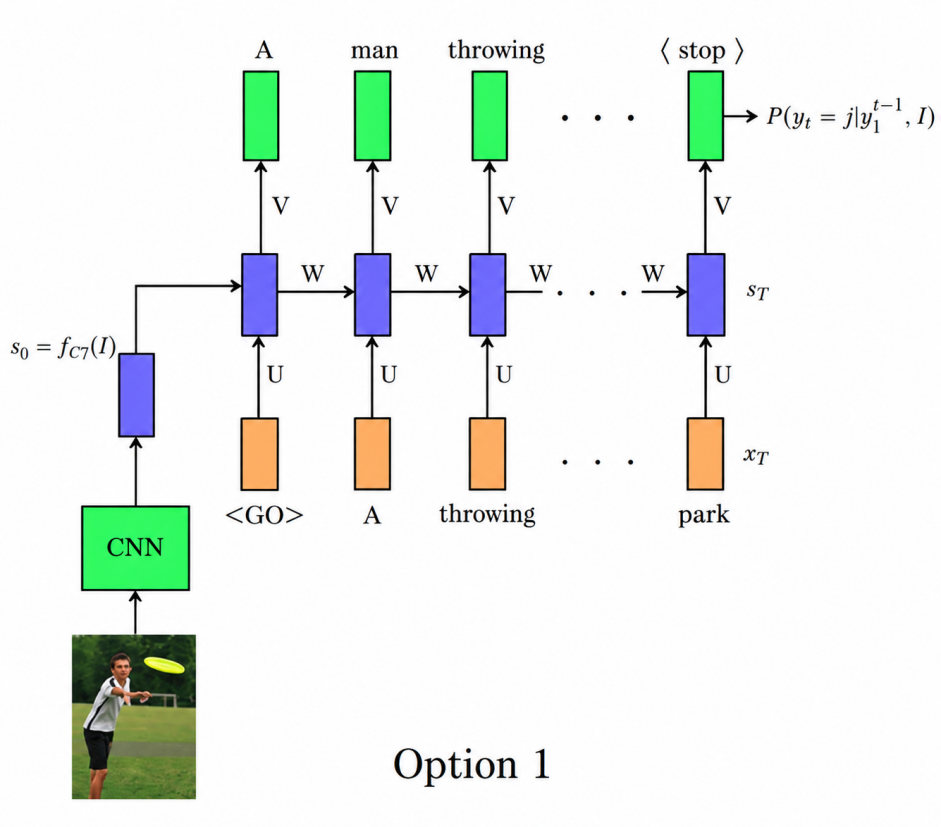
## What is $fc_7$?

The second-to-last fully connected layer of a classification CNN (e.g. AlexNet/VGG) — a fixed-size vector summarizing the whole

# Encoder Decoder Model for Image Captioning

There are many ways of making $P(y_t = j)$ conditional on $f_{c_7}(I)$

# Encoder Decoder Model for Image Captioning



# Encoder Decoder Model for Image Captioning

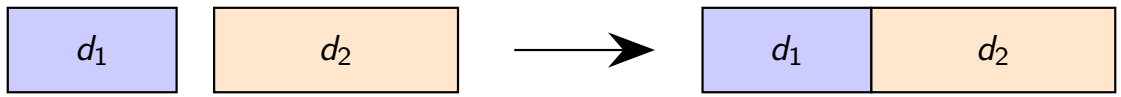
- **Option 2:** Another, more explicit way of doing this is to compute

$$s_t = RNN(s_{t-1}, [x_t, f_{c_7}(I)])$$

- In other words, we are explicitly using $f_{c_7}(I)$ to compute $s_t$ and hence $P(y_t = j)$

**What does the comma inside $[\cdot, \cdot]$ mean?**

It's **concatenation** — stacking two vectors end to end into one longer vector, not adding or combining them mathematically.




$[d_1, d_2]$ — length $d_1 + d_2$

# Encoder Decoder Model for Image Captioning

$$s_1 = \sigma(W \underbrace{s_0}_{\uparrow \text{CNN}} + U[\underbrace{x_1}_{\uparrow \text{one-hot of } \langle GO \rangle}, \underbrace{s_0}_{\uparrow \text{CNN}}] + b)$$

Here $s_0 = f_{c_7}(I)$ appears **twice**: once as the running hidden state coming in (via $W$), and once concatenated fresh alongside the input word (via $U$) — this second copy is what makes Option 2 "explicit".

# Encoder Decoder Model for Image Captioning

$$s_2 = \sigma(W \underbrace{s_1}_{\text{from previous step}} + U[\underbrace{x_2}_{\text{one-hot of "A"}}, \underbrace{s_0}_{\text{CNN}}] + b)$$

Notice: $s_0 = f_{c_7}(I)$ stays exactly the **same** fixed vector as before — the image doesn't change — while the word input advances from $x_1$ to $x_2$.

# Encoder Decoder Model for Image Captioning

- **Option 1:** Set $s_0 = f_{c_7}(I)$
- Now $s_0$ and hence all subsequent $s_t$'s depend on $f_{c_7}(I)$

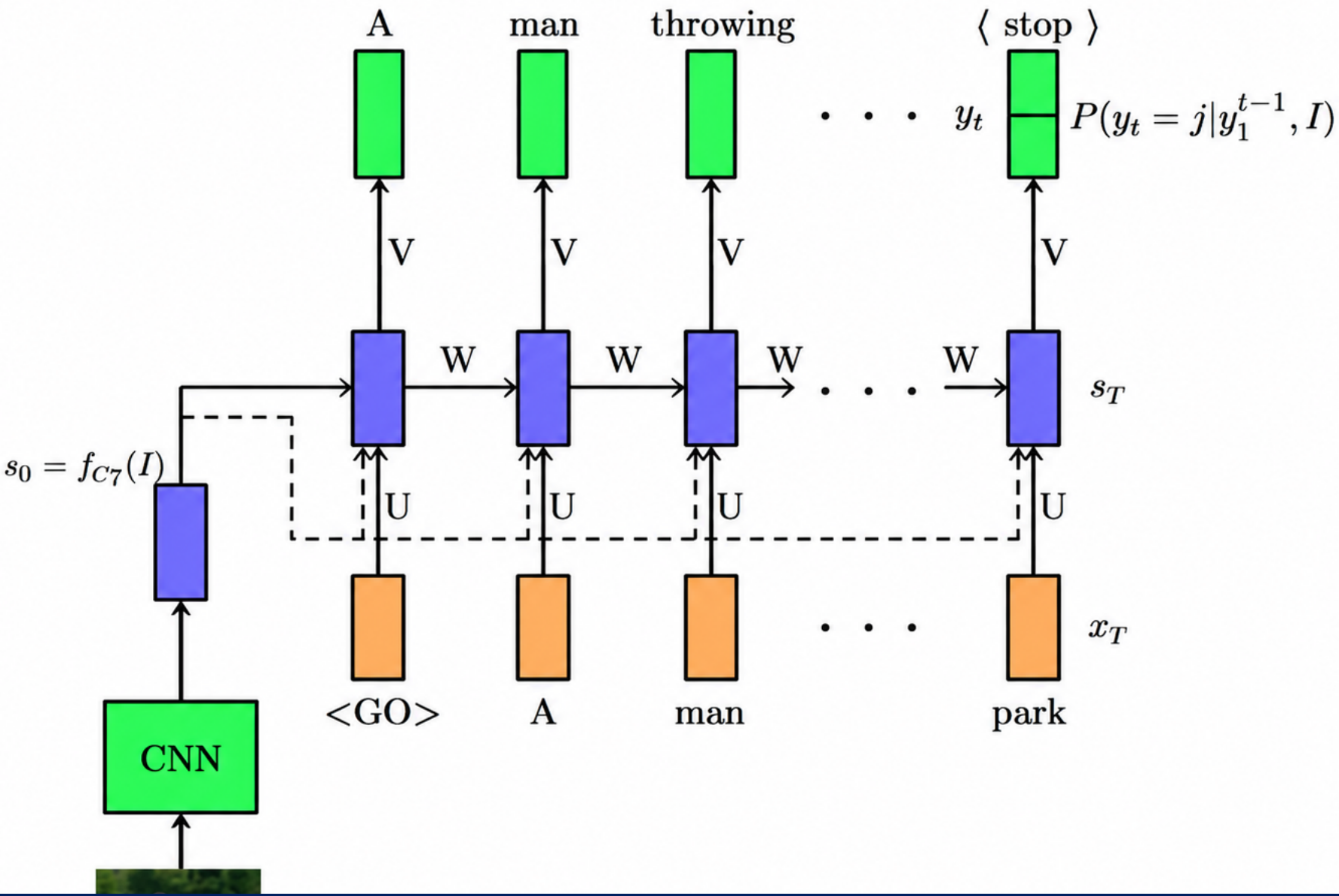
$$s_1 = \sigma(W \underbrace{s_0}_{\uparrow \text{CNN}} + U \underbrace{x_1}_{\uparrow \text{one-hot of } \langle GO \rangle} + b)$$

# Encoder Decoder Model for Image Captioning

$$s_2 = \sigma(Ws_1 + Ux_2 + b)$$

$$\hat{y}_2 = \text{softmax}(Vs_2 + c)$$

# Encoder Decoder Model for Image Captioning



# Encoder Decoder Model for Image Captioning

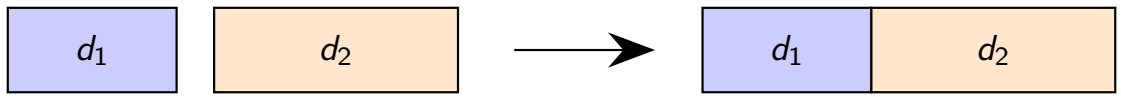
- **Option 2:** Another, more explicit way of doing this is to compute

$$s_t = RNN(s_{t-1}, [x_t, f_{c_7}(I)])$$

- In other words, we are explicitly using $f_{c_7}(I)$ to compute $s_t$ and hence $P(y_t = j)$

**What does the comma inside $[\cdot, \cdot]$ mean?**

It's **concatenation** — stacking two vectors end to end into one longer vector, not adding or combining them mathematically.




$[d_1, d_2]$ — length $d_1 + d_2$

# Encoder Decoder Model for Image Captioning

$$s_1 = \sigma(W \underbrace{s_0}_{\uparrow \text{CNN}} + U[\underbrace{x_1}_{\uparrow \text{one-hot of } \langle GO \rangle}, \underbrace{s_0}_{\uparrow \text{CNN}}] + b)$$

Here $s_0 = f_{c_7}(I)$ appears **twice**: once as the running hidden state coming in (via $W$), and once concatenated fresh alongside the input word (via $U$) — this second copy is what makes Option 2 "explicit".

# Encoder Decoder Model for Image Captioning

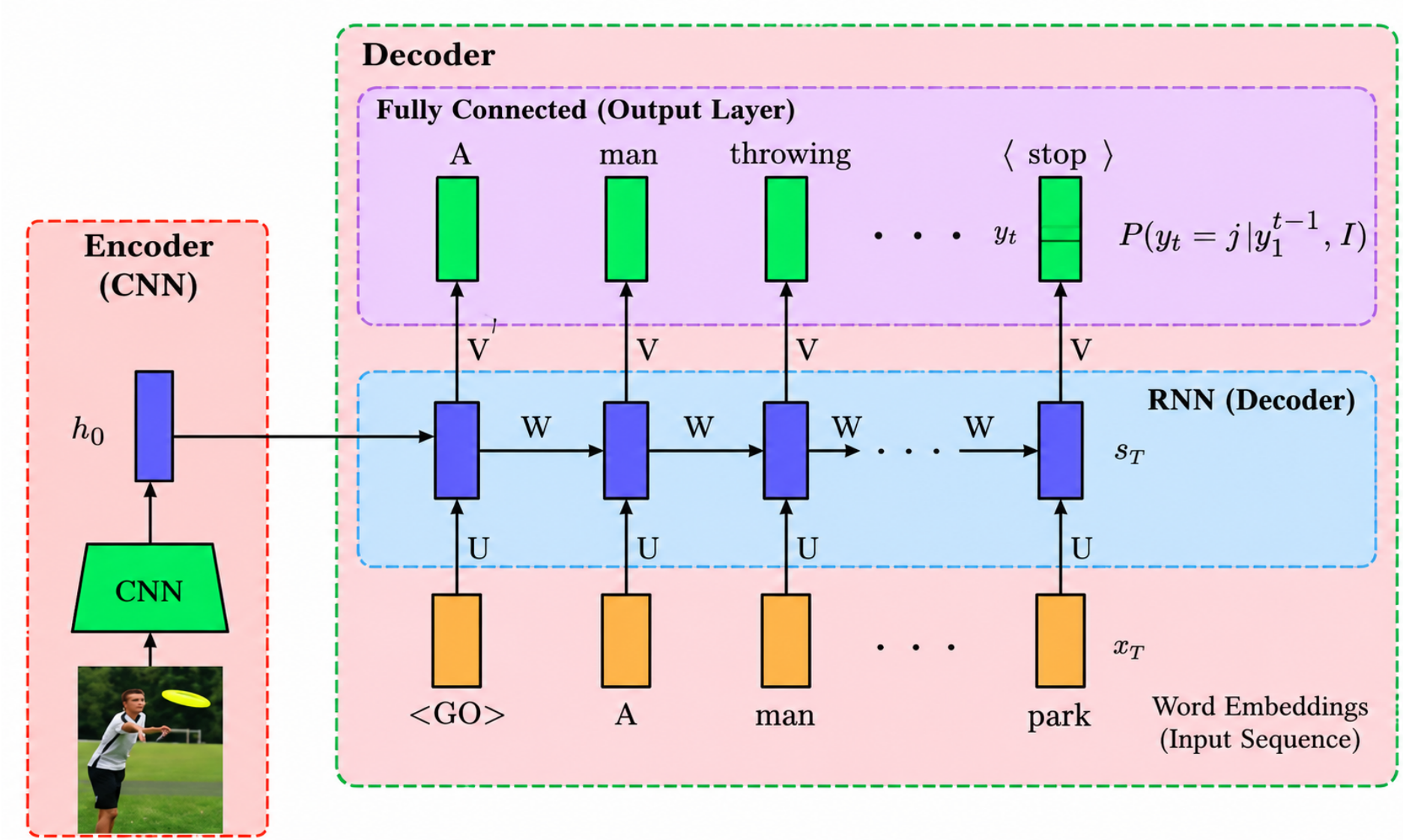
$$s_2 = \sigma(W \underbrace{s_1}_{\text{from previous step}} + U[\underbrace{x_2}_{\text{one-hot of "A"}}, \underbrace{s_0}_{\text{CNN}}] + b)$$

Notice: $s_0 = f_{c_7}(I)$ stays exactly the **same** fixed vector as before — the image doesn't change — while the word input advances from $x_1$ to $x_2$.

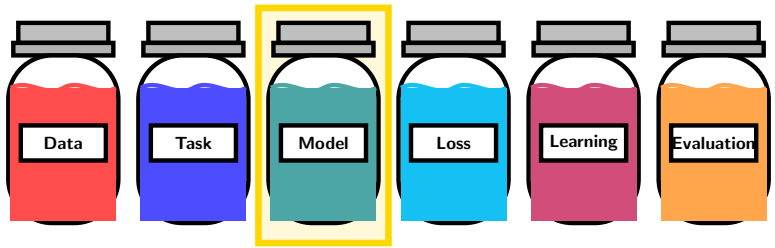
# Encoder Decoder Model for Image Captioning

- Let us look at the full architecture
- A CNN is first used to **encode** the image
- A RNN is then used to **decode** (generate) a sentence from the encoding
- This is a typical **encoder decoder architecture**

# Encoder Decoder Model for Image Captioning



# Encoder Decoder Model for Image Captioning




- **Task:** Image captioning
- **Data:** $\{x_i = image_i, y_i = caption_i\}_{i=1}^{N}$
- **Model:**
  - **Encoder:** $s_0 = CNN(x_i)$
  - **Decoder:**

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1}))$$

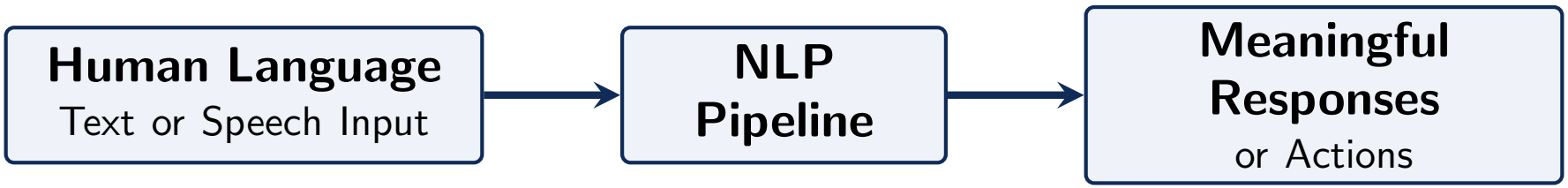
$$P(y_t \mid y_1^{t-1}, I) = \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $W_{conv}$, $b$
- **Loss:**

$$\mathcal{L}(\theta) = \sum_{t=1}^{T} \mathcal{L}_t(\theta) = -\sum_{t=1}^{T} \log P(y_t = \ell_t \mid y_1^{t-1}, I)$$

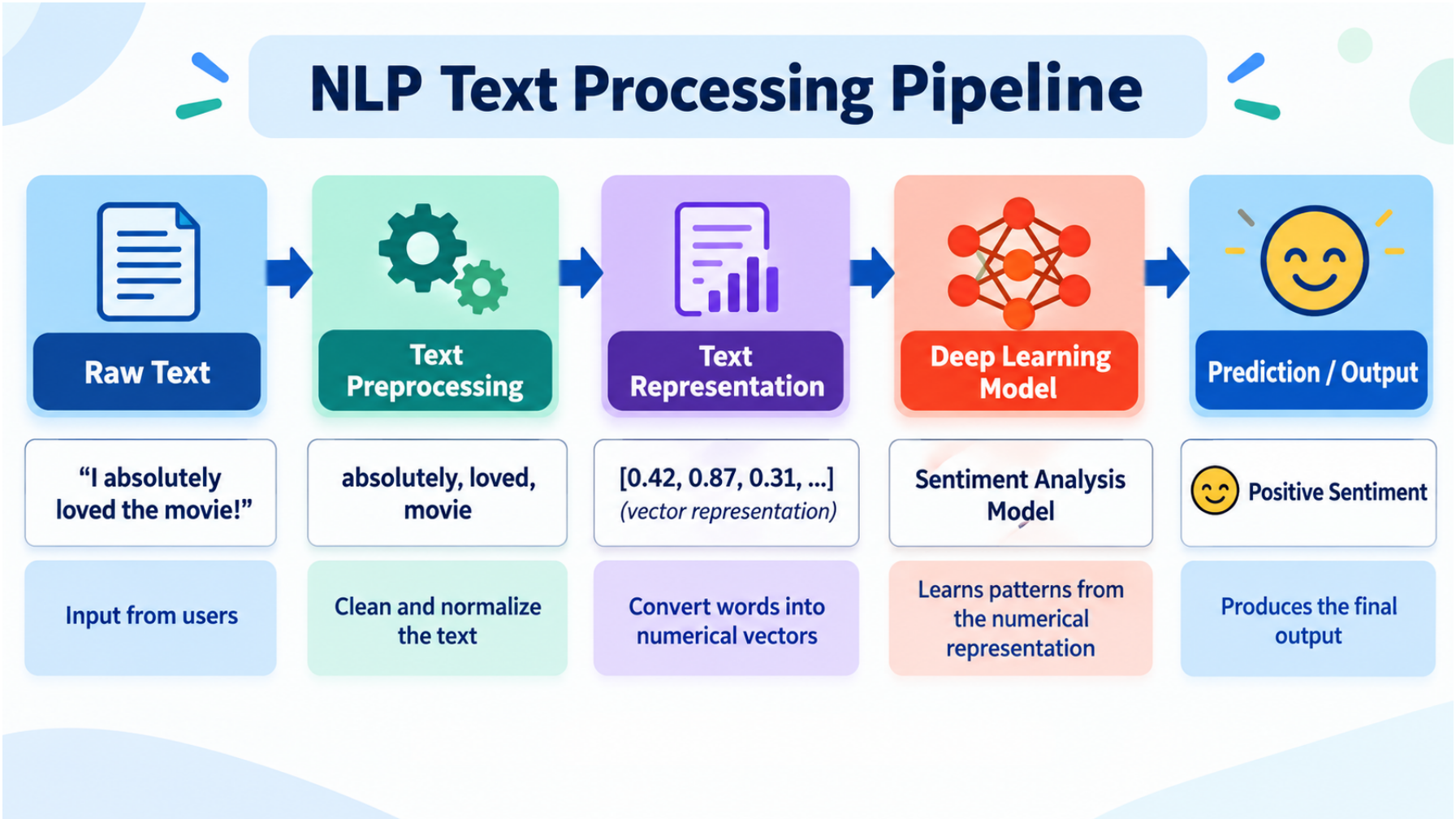
- **Algorithm:** Gradient descent with backpropagation

# NLP: From Language to Action



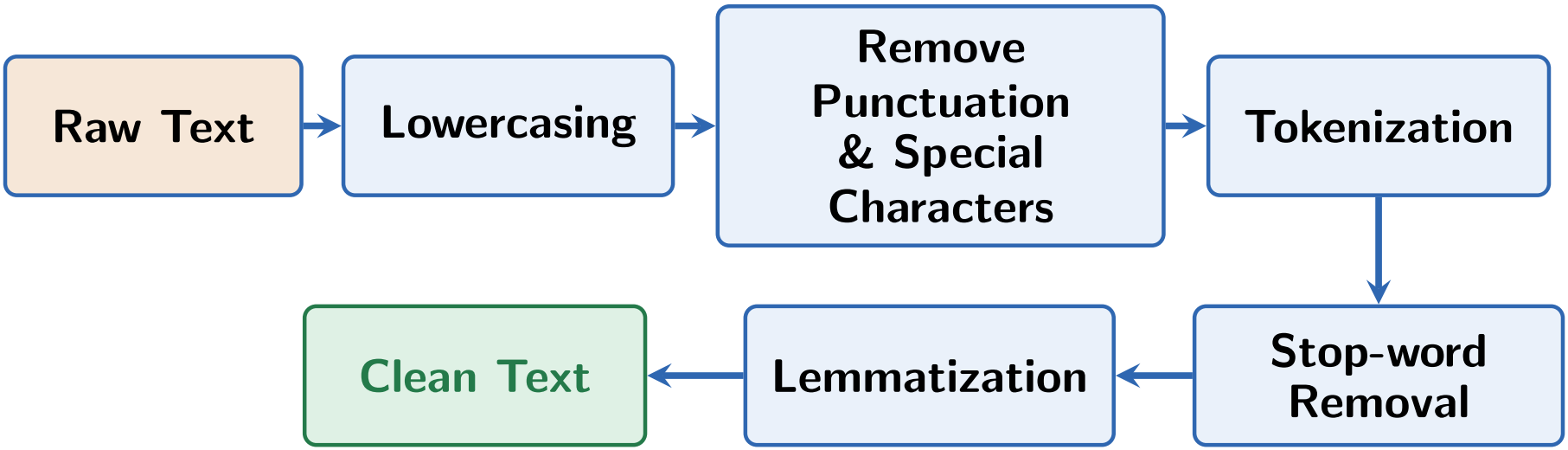

| Input | Output | Action |
|---|---|---|
| **"Is this email spam?"** | Yes | **Spam Detection** |
| **"The movie was amazing!"** | Positive Sentiment | **Sentiment Analysis** |
| **"Hello"** | "Hola" | **Machine Translation** |
| **"What is Generative AI?"** | Learns patterns from data and generates new content. | **Question Answering** |

# NLP Pipeline



# Text Preprocessing

Text preprocessing cleans and transforms raw text into a structured form that machine learning and deep learning models can analyze.



# Text Preprocessing Example

**Raw Text**

"The Quick Brown Foxes are Running!!!
Visit `https://example.com` :-)"

**After Preprocessing**

`quick brown fox run`

Tokenization divides the cleaned text into smaller units called **tokens**.

**quick** **brown** **fox** **run**

# Tokenization

- Tokenization splits text into smaller units called **tokens**.
- A token may be a word, sentence, subword, or character.

## Word Tokenization

"I love learning Natural Language Processing."

| I | love | learning | Natural | Language | Processing |
|---|---|---|---|---|---|

## Sentence Tokenization

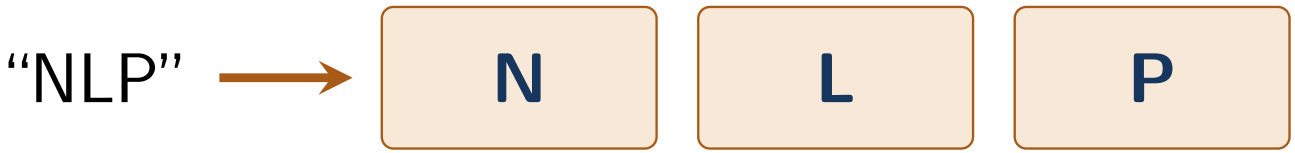
"NLP is fascinating. It enables computers to understand language. It powers ChatGPT."

| NLP is fascinating. | It enables computers to understand language. | It powers ChatGPT. |
|---|---|---|

# Tokenization

## Subword Tokenization

## Character Tokenization




**Key idea**

Tokenization converts raw text into smaller meaningful units that computers can process.

# Stop-word Removal

Stop words are frequent words such as **the**, **is**, **are**, **of**, and **and**.

**Original Sentence**

**The** students **are** learning Natural Language Processing.

**After Stop-word Removal**

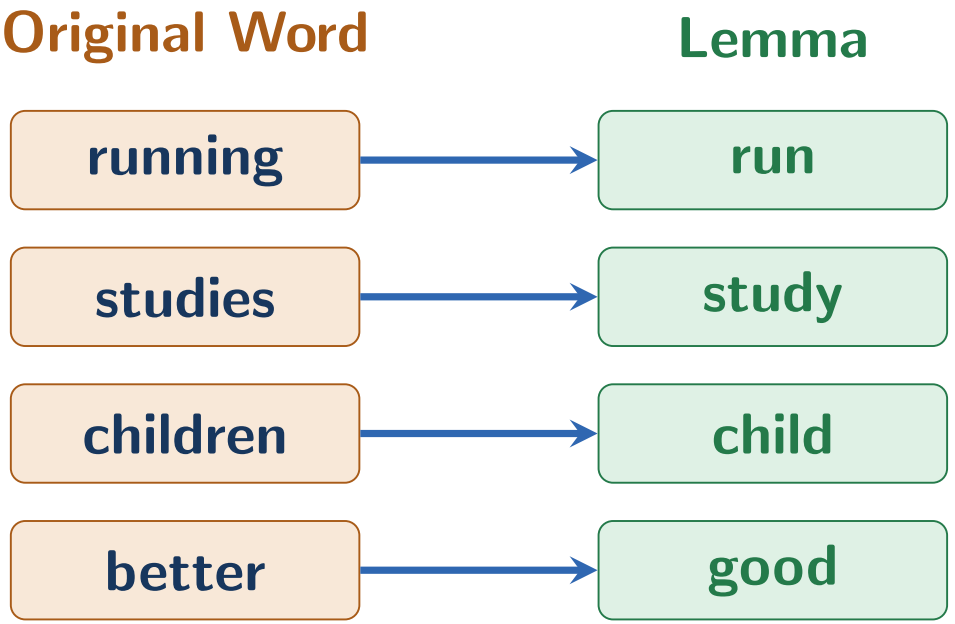
students learning Natural Language Processing

**Why is it useful?**

Stop words often carry little task-specific meaning. Removing them can reduce noise and vocabulary size.

# Lemmatization

Lemmatization converts a word to its base or dictionary form, called its **lemma**, while preserving its meaning.




**Why is it useful?**

Maps different forms of a word to one common base.

This helps the model treat related words as the same concept.

**After preprocessing, how can a computer represent and understand these words?**

# Text Representation

Machines process numbers rather than words. Text representation converts language into numeric vectors that a model can use.

**Traditional Techniques**

- ✓ One-hot encoding
- ✓ Bag of Words (BoW)
- ✓ TF-IDF

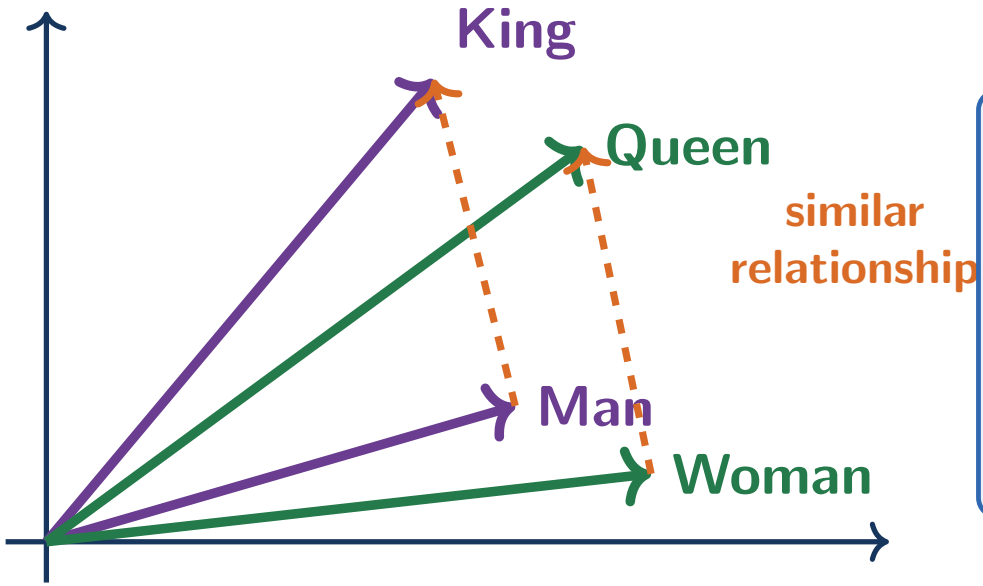
**Modern Techniques**

**Static embeddings**
Word2Vec and GloVe

**Contextual embeddings**
BERT and GPT

# Word2Vec Word Embeddings

- Words used in similar contexts receive similar vectors.
- Word2Vec learns these vectors from local context windows.




Related meanings often produce nearby vectors.

Similar directions can encode relations such as gender and royalty.

$$\vec{V}_{king} - \vec{V}_{man} + \vec{V}_{woman} \approx \vec{V}_{queen}$$

# Understanding the Word2Vec Analogy

**Word2Vec learns from nearby words**

**Semantic components**

King $\approx$ royalty + male

Man $\approx$ male

Woman $\approx$ female

**Vector operation**

1. Subtracting **Man** reduces the male-associated direction.
2. Adding **Woman** moves the result toward **Queen**.

$$\vec{V}_{king} - \vec{V}_{man} + \vec{V}_{woman} \approx \vec{V}_{queen}$$

This is an approximate pattern learned from the training data, not an exact rule.

# GloVe Word Embeddings

- GloVe learns dense word vectors from **global co-occurrence statistics**.
- Words that occur in similar contexts receive similar representations.

**Co-occurrence Matrix**

| Word | King | Queen | Man | Woman |
|---|---|---|---|---|
| **King** | 0 | 120 | **250** | 80 |
| **Queen** | 120 | 0 | 75 | **240** |
| **Man** | **250** | 75 | 0 | 110 |
| **Woman** | 80 | **240** | 110 | 0 |

**Reading the matrix**

Each value counts how often two words appear near each other. In this illustrative matrix, **king–man** and **queen–woman** have high co-occurrence counts.

# Challenges in NLP

| | |
|---|---|
| **Discrete Nature of Text** | Raw word symbols do not express similarity. For example, **sky and stars** form a related pair, as do **red and pink**. |
| **Evolving Language** | Language changes continually. New words and slang enter the vocabulary, for example, **rizz**. |
| **Ambiguity and Vagueness** | Meaning can depend on tone or context. Example: "I have never tasted a pizza like this before." |
| **Long-range Dependencies** | Resolving **it** requires distant context: "The animal did not cross the road because it was too **tired** / **wide**." Tired links **it** to the animal; wide links **it** to the road. |

# Text Summarization

*Given a long piece of text, can we automatically produce a short version that keeps the key meaning?*

## Example

**Document:** "The scientist made an important discovery in the lab today. Her team celebrated the breakthrough late into the night."

**Summary:** "Scientist makes breakthrough discovery."

## Two flavors

- **Extractive:** pick out important sentences/phrases directly from the document
- **Abstractive:** generate **new** sentences that capture the meaning, possibly using words not in the original — this is the seq2seq version we focus on here

# Text Summarization: A Seq2Seq Problem

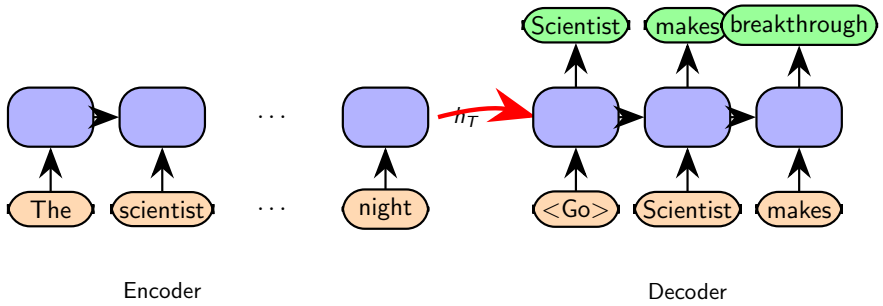
## Same shape as Txt2Txt

- Input is a sequence: the document, $x_1, x_2, \ldots, x_T$
- Output is a sequence: the summary, $y_1, y_2, \ldots, y_L$
- Crucially, $L \ll T$ — the output is **much shorter** than the input

## A harder version of the same bottleneck problem

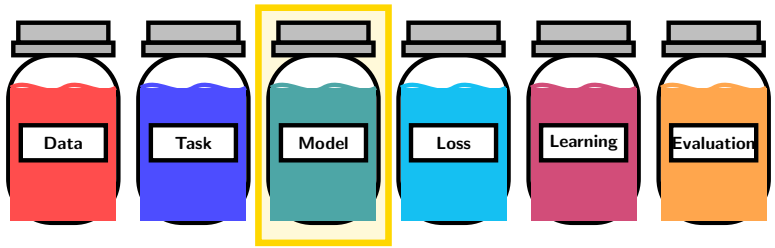
- In Txt2Txt, input and output were roughly similar in length
- Here the encoder must compress a **long** document into the **same** fixed-size $h_T$ that worked for short sentences
- This makes the single-context-vector bottleneck (Option 1) even more strained — a strong motivation for what comes later with Attention

# Text Summarization: Encoder-Decoder




Exactly the Txt2Txt architecture — the encoder just has to read a much longer input before handing off $h_T$.

# Text Summarization




- **Task:** Text summarization (abstractive)
- **Data:** $\{x_i = document_i, y_i = summary_i\}_{i=1}^{N}$
- **Model:**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1})), \quad P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:** $\mathcal{L}(\theta) = -\sum_{t=1}^{L} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$
- **Algorithm:** Gradient descent with backpropagation

# Question-Answer Bots

*Given some context and a question about it, can we generate the answer?*

## Example

**Context:** "The Eiffel Tower is located in Paris, France. It was completed in 1889."

**Question:** "Where is the Eiffel Tower located?"

**Answer:** "Paris, France"

Unlike Txt2Txt or Summarization, there are now **two** separate pieces of input text (context and question) that must both influence the output.

# Question-Answer Bots: Encoding Two Inputs

## The question

How does one encoder-decoder handle **two** separate input sequences?

## A simple approach: concatenate

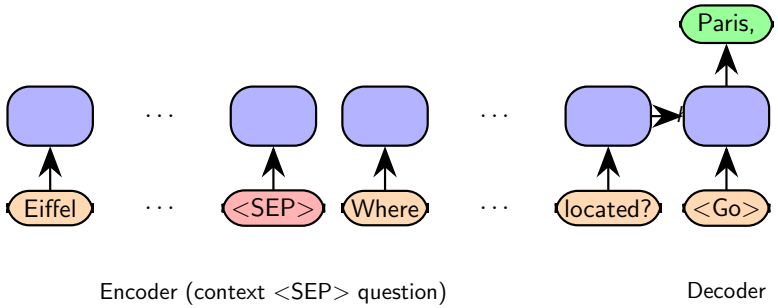
- Join context and question into a single sequence, with a separator token

  `[context] <SEP> [question]`

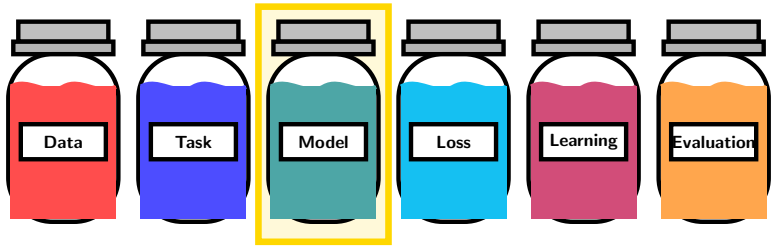
- Feed this combined sequence through a single encoder, exactly as before
- The encoder's final state $h_T$ now summarizes **both** the context and the question together

This keeps the exact same architecture as Txt2Txt — only the **input construction** changes.

# Question-Answer Bots: Encoder-Decoder



# Question-Answer Bots




- **Task:** Question answering
- **Data:** $\{x_i = [context_i, \langle SEP \rangle, question_i], y_i = answer_i\}_{i=1}^{N}$
- **Model:**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T$

$$s_t = RNN(s_{t-1}, e(\hat{y}_{t-1})), \quad P(y_t \mid y_1^{t-1}, x) = \text{softmax}(Vs_t + b)$$

- **Parameters:** $U_{dec}$, $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:** $\mathcal{L}(\theta) = -\sum_{t=1}^{L} \log P(y_t = \ell_t \mid y_1^{t-1}, x)$
- **Algorithm:** Gradient descent with backpropagation

# Txt2Img

*Given a text description, can we generate an image that matches it?*

## Example

**Text:** "a red circle on a white background"

**Image:** (generated)

## Task:

Text-to-image generation

# Txt2Img: A Different Kind of Decoder

## So far, every decoder generated a sequence of discrete tokens

- Words (Txt2Txt), characters (Transliteration) — a softmax over a fixed vocabulary at every step

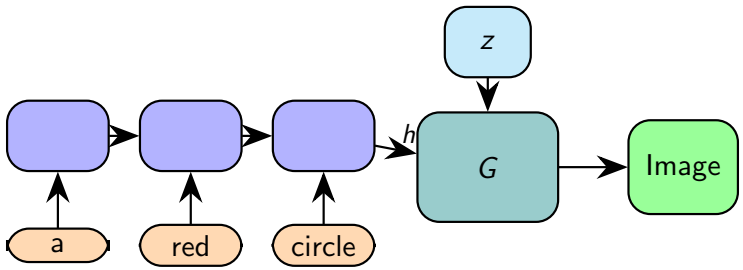
## An image is not a sequence of discrete tokens

- It's a grid of continuous-valued pixels — there's no "vocabulary" of possible images to choose from
- A softmax-based decoder simply doesn't apply here

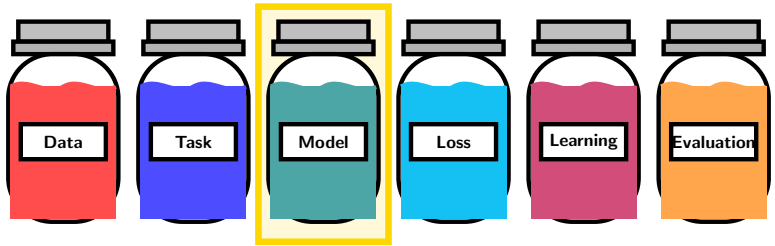
## The fix: reuse the Generator from GANs

- **Encoder** (RNN): reads the text, produces an embedding $h_T$
- **Decoder** (Generator): takes $h_T$ (as the conditioning input) together with noise $z$, and generates an image — exactly the Conditional GAN setup from before

# Txt2Img: Encoder + Conditional Generator



# Txt2Img




- **Task:** Text-to-image generation
- **Data:** $\{x_i = text_i, y_i = image_i\}_{i=1}^{N}$
- **Model:**
  - **Text Encoder:** $h_t = RNN(h_{t-1}, x_{it})$, giving text embedding $h_T$
  - **Generator:** $\hat{y} = G(z, h_T)$
  - **Discriminator:** $D(y, h_T) \in [0, 1]$ — real image + matching text vs. fake
- **Parameters:** $U_{enc}, W_{enc}$ (text encoder); $\theta_G$ (Generator); $\theta_D$ (Discriminator)
- **Loss:** adversarial minimax objective (as in Conditional GAN), conditioned on $h_T$
- **Algorithm:** alternating gradient descent (Generator vs. Discriminator)

# Txt2Voice

*Given text, can we generate the corresponding spoken audio?*

## Example

**Text:** "Hello"

**Audio:** a sequence of sound frames that, played back, say "Hello"

## Task:

Text-to-speech (TTS) generation

# Txt2Voice: A Different Kind of Decoder Output

## So far, the decoder always produced discrete tokens

- A softmax over a fixed vocabulary of words or characters, one choice per timestep

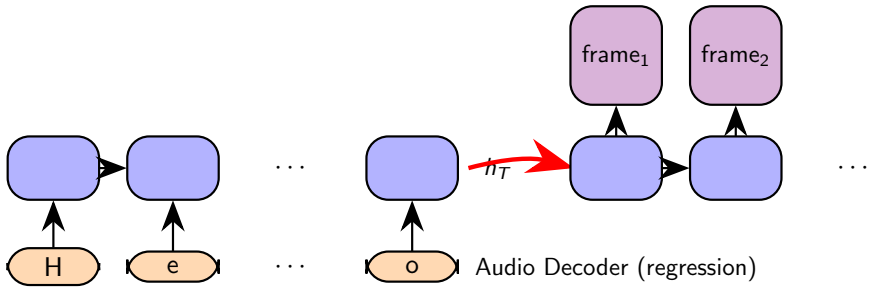
## Audio is continuous, not discrete

- Each output timestep is a **spectrogram frame** — a vector of continuous numbers describing sound at that instant
- There's no fixed "vocabulary" of sounds to pick from with softmax

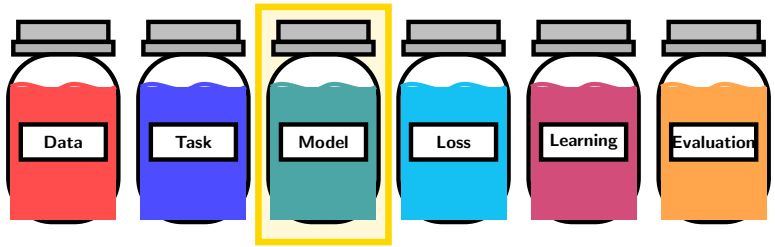
## The fix: swap softmax for direct regression

- Decoder structure stays the same (RNN, $s_t = RNN(s_{t-1}, \hat{y}_{t-1})$)
- Output layer becomes linear instead of softmax: $\hat{y}_t = Vs_t + b$
- Loss becomes mean squared error instead of cross-entropy

# Txt2Voice: Encoder-Decoder



# Txt2Voice




- **Task:** Text-to-speech generation
- **Data:** $\{x_i = text_i, y_i = audio_i\}_{i=1}^{N}$, where $audio_i$ is a sequence of spectrogram frames
- **Model:**
  - **Encoder:** $h_t = RNN(h_{t-1}, x_{it})$
  - **Decoder:** $s_0 = h_T, \quad s_t = RNN(s_{t-1}, \hat{y}_{t-1}), \quad \hat{y}_t = Vs_t + b$ (linear, no softmax)
- **Parameters:** $V$, $W_{dec}$, $U_{enc}$, $W_{enc}$, $b$
- **Loss:** $\mathcal{L}(\theta) = \sum_{t=1}^{T} \|y_t - \hat{y}_t\|^2$ (mean squared error)
- **Algorithm:** Gradient descent with backpropagation